



Republic of Algeria Democratic and Popular

Ministry of Higher Education and Scientific Research

University of Abbes Laghrour - khenchela -

Faculty of Science and Technology

Computer Science Department

Field of study: Artificial Intelligence

**Comparative Study of DQN, Double DQN, and Dueling DQN
with Parameter Sharing for Adaptive Traffic Signal Control in
Urban Multi-Intersection Environments**

Supervised by:

Dr. Chergui Wahid

Presented by:

Chahat Rima

The jury members:

President:

Examiner:

June 2026

Dedication

I am pleased to dedicate this work to:

My dear father:

For your support, love, and sacrifice in ensuring that nothing hinders my studies. Your constant presence by my side is my source of strength in facing all challenges.

My dear mother:

For her effort and hard work in ensuring that nothing hinders my studies. Your endless love, patience, and encouragement have been my greatest motivation.

My close friend Mouna:

To thank her for all the beautiful and inspiring moments we shared, and for her constant support and encouragement throughout this journey.

Acknowledgments

First and foremost, I thank God for granting me the ability
to complete this humble work.

I extend my sincere gratitude to my supervisor, **Dr. Chergui Wahid**,
for his guidance, patience, and invaluable advice throughout this research.

I also thank everyone who contributed, in one way or another,
to the completion of this work, whether through information, ideas,
or moral support.

Abstract

This thesis presents a rigorous comparative study of three value based deep reinforcement learning architectures (Classical Deep Q-Network (DQN), Double DQN, and Dueling DQN) for adaptive traffic signal control in a multi-intersection urban environment. Motivated by the absence of controlled comparative evaluations in the existing literature, all three architectures are trained and evaluated under a strictly unified experimental framework using the SUMO microscopic traffic simulator, ensuring that observed performance differences are attributable exclusively to architectural distinctions rather than experimental confounds.

A multi agent architecture based on parameter sharing and a shared Prioritized Experience Replay buffer is implemented, enabling a single neural network to control four signalized intersections simultaneously through decentralized execution. All models are trained on a rotating curriculum of low, medium, and mixed traffic demand scenarios and evaluated on an unseen high demand scenario generating 1,800 vehicles per hour, providing a genuine test of out of distribution generalization. A paired evaluation design assigns identical stochastic demand realizations to all four models across 20 independent episodes, eliminating demand variability as a confounding factor in the comparison.

Results demonstrate that all three DRL variants outperform the fixed time baseline on all eight evaluation metrics without exception. Dueling DQN achieves the strongest mean performance across all metrics, reducing average waiting time by 15.2%, average delay by 16.1%, and improving average speed by 21.7% relative to the fixed time baseline, with the narrowest confidence intervals among all models. However, paired t-test analysis reveals that the difference between Dueling DQN and Classical DQN is not statistically significant on any metric (all $p > 0.05$, Cohen's $d = 0.10$), indicating that the two architectures are practically equivalent in this setting. Double DQN consistently underperforms relative to both, with a meaningful performance gap of approximately 4.9% on average waiting time and a Cohen's d of 0.27 relative to Classical DQN, attributable to a switch bias mechanism introduced by the overestimation correction in binary action spaces. An important methodological finding concerning the measurement of throughput in episodic simulation is documented and resolved, with implications for future simulation based traffic control studies.

Keywords: adaptive traffic signal control, deep reinforcement learning, Deep Q Network, Double DQN, Dueling DQN, Prioritized Experience Replay, multi agent systems, parameter sharing, SUMO simulation, urban traffic management.

Résumé

Ce mémoire présente une étude comparative rigoureuse de trois architectures d'apprentissage par renforcement profond à base de valeur (le réseau Q profond classique (DQN), le Double DQN et le Dueling DQN) appliquées au contrôle adaptatif des feux de signalisation dans un environnement urbain à intersections multiples. Motivée par l'absence d'évaluations comparatives contrôlées dans la littérature existante, les trois architectures sont entraînées et évaluées dans un cadre expérimental strictement unifié utilisant le simulateur de trafic microscopique SUMO, garantissant que les différences de performance observées sont attribuables exclusivement aux distinctions architecturales plutôt qu'à des facteurs expérimentaux confondants.

Une architecture multi agents basée sur le partage de paramètres et un tampon d'expérience rejouée priorisée partagé est mise en œuvre, permettant à un réseau de neurones unique de contrôler simultanément quatre intersections à feux par exécution décentralisée. Tous les modèles sont entraînés sur un curriculum rotatif de scénarios de demande de trafic faible, moyen et mixte, puis évalués sur un scénario à forte demande non rencontré pendant l'entraînement, générant 1 800 véhicules par heure, constituant un test authentique de généralisation hors distribution. Un protocole d'évaluation apparié assigne des réalisations de demande stochastique identiques aux quatre modèles sur 20 épisodes indépendants, éliminant la variabilité de la demande comme facteur confondant dans la comparaison.

Les résultats démontrent que les trois variantes d'apprentissage par renforcement profond surpassent le contrôle à temps fixe sur l'ensemble des huit métriques d'évaluation sans exception. Le Dueling DQN obtient les meilleures performances moyennes sur toutes les métriques, réduisant le temps d'attente moyen de 15,2 %, le délai moyen de 16,1 % et améliorant la vitesse moyenne de 21,7 % par rapport au contrôle à temps fixe, avec les intervalles de confiance les plus étroits parmi tous les modèles. Cependant, l'analyse par test t apparié révèle que la différence entre le Dueling DQN et le DQN classique n'est statistiquement significative sur aucune métrique ($p > 0,05$, d de Cohen = 0,10), indiquant que les deux architectures sont pratiquement équivalentes dans ce contexte. Le Double DQN présente systématiquement des performances inférieures aux deux autres variantes, avec un écart de performance d'environ 4,9 % sur le temps d'attente moyen et un d de Cohen de 0,27 par rapport au DQN classique, attribuable à un mécanisme de biais vers la commutation introduit par la correction de surestimation dans les espaces d'action binaires. Une observation méthodologique importante concernant la mesure du débit dans les simulations épisodiques est documentée et résolue, avec des implications pour les futures études de contrôle du trafic par simulation

Mots-clés : contrôle adaptatif des feux de signalisation, apprentissage par renforcement profond, réseau Q profond, Double DQN, Dueling DQN, expérience rejouée priorisée, systèmes multi agents, partage de paramètres, simulation SUMO, gestion du trafic urbain.

Table of Contents

Dedication	I
Acknowledgments	II
Abstract	III
Résumé	IV
Table of Contents	V
List of tables	1
List of figures	2
General Introduction	3
Chapter 1: Theoretical Foundations	8
1. Introduction	8
2. Urban Traffic and Congestion	8
2.1. Traffic congestion	9
2.1.1. Definition	9
2.1.2. Primary Causes	9
2.1.3. Significant Impacts	10
2.2. Key Performance Indicators (KPIs)	10
3. Traffic Signal Control Systems	11
3.1. Role of Traffic Signals in Intersection Control	11
3.2. Traffic Signal Control Fundamentals	11
3.3. Traffic Signal Control Methods	13
3.3.1. Fixed-Time Control	13
3.3.2. Actuated Control	13
3.3.3. Adaptive Control	13
3.4. Limitations of Traditional Traffic Signal Control Methods	14
4. Intelligent Transportation Systems -ITS	15
4.1. Components of Intelligent Transportation Systems	16
4.2. Role of ITS in Reducing Traffic Congestion	17
4.3. Limitations of ITS without Artificial Intelligence	17
5. Machine Learning Fundamentals	18
5.1. Supervised learning	18
5.2. Unsupervised learning	18
5.3. Reinforcement learning	19
6. Reinforcement Learning	19
7. Markov Decision Process (MDP)	20
8. Q-Learning	21
9. Deep Q-Network (DQN)	22
10. Chapter 1 conclusion	23
Chapter 2 : State of Art	24

1. Introduction	24
2. Traditional traffic signal control methods	25
3. Intelligent and Optimization-Based Approaches	26
3.1. Fuzzy Logic Control	26
3.2. Optimization-Based Approaches (Genetic Algorithm)	27
3.3. Hybrid Approaches (Fuzzy Logic + Genetic Algorithm)	27
3.4. Discussion and Transition	28
4. Reinforcement Learning for Traffic Signal Control	28
5. Deep Reinforcement Learning	29
6. Deep Q Network (DQN)	30
6.1. Classical Deep Q-Network (baseline DQN)	30
6.2. Double Deep Q-Network (Double DQN)	31
6.3. Dueling Deep Q-Network (Dueling DQN)	32
7. Multi-Agent Reinforcement Learning	34
8. Limitations of Existing Work	35
9. Research Gap	37
Chapter 3: Methodology and System Design	38
1. Introduction	38
2. General Overview of the Proposed Solution	38
3. Description of the Retained Network	40
3.1. General Topology	40
3.2. State Vector Dimension	40
3.3. Justification for the Comparative Study	40
4. Problem Formulation as a Markov Decision Process	40
4.1. MDP Definition	40
4.2. State Representation	41
4.3. Action Space	43
4.4. Reward Function	44
5. Multi-Agent Architecture and Parameter Sharing	45
5.1. Decentralized Execution	45
5.2. Parameter Sharing Strategy	46
5.3. Shared Experience Replay	47
6. DQN Architectures	48
6.1. Classical DQN Architecture	48
6.2. Double DQN	49
6.3. Dueling DQN	51
6.4. Comparative Summary	52
7. Evaluation Metrics	52
8. Comparison Scenarios	54
9. Chapter Summary	55
Chapter 4: Implementation and Experimentation	56
1. Introduction	56
2. Hardware and Software Environment	56
3. The SUMO Simulator	57
4. Construction of the Simulated Network	57
4.1. Cluster Junctions	58

4.2. Traffic Generation	59
5. Implementation of the Three Methods	59
5.1. Shared Components	59
5.2. Classical DQN	59
5.3. Double DQN	60
5.4. Dueling DQN	60
6. Prioritized Experience Replay parameters	61
6.1. TD Error Priority	61
6.2. Importance Sampling Correction	63
6.3. Implementation Details	64
7. Training Setup and Hyperparameters	65
7.1. General Configuration	65
7.2. Epsilon Decay Strategy	67
7.3. Checkpoint Mechanism	67
8. Evaluation Protocol	69
8.1. Evaluation Scenario and Experimental Design	69
8.2. Greedy Policy Evaluation	69
8.3. Fixed-Time Baseline	69
8.4. Statistical Analysis	71
9. Reproducibility and Fairness of Comparison	72
9.1. Unified Random Seed	72
9.2. Identical Environment Configuration	72
9.3. Controlled Variation	72
9.4. Shared Metric Infrastructure	72
10. Chapter 4 Summary	73
Chapter 5: Results and Discussion	76
1. Introduction	76
2. Training Results	76
2.1. Exploration Scheduling: Epsilon Decay	76
2.2. Replay Buffer Growth	77
2.3. Beta Annealing for Importance Sampling Correction	78
2.4. Training Loss Convergence	79
3. Summary of Results	80
3.1. Performance Summary Table	80
3.2. Overview	81
4. Comparison with Fixed Time Baseline	81
4.1. Improvement Percentages	81
4.2. Bar Chart Analysis	82
4.3. Throughput Measurement Note	83
5. Metric by Metric Analysis	84
5.1. Congestion Metrics: Waiting Time, Queue Length, and Delay	84
5.2. Traffic Flow: Speed and Throughput	84
5.3. Environmental Metrics: CO ₂ and Fuel	85
5.4. Stopped Vehicle Ratio	86
6. Comparative Analysis Between DQN Variants	86
6.1. Classical DQN versus Double DQN: A Consistent Performance Gap	86
6.2. Dueling DQN: Consistent Superior Performance	87
6.3. Performance Ranking Summary	88

7. Distribution Analysis	88
7.1. Box Plot Analysis	88
7.2. Consistency Analysis	89
8. Radar Chart Summary	90
9. Statistical Significance Analysis	91
9.1. Motivation and Method	91
9.2. Results	91
9.3. Interpretation	92
10. Principal Findings	92
10.1. All DQN Variants Outperform Fixed-Time Control	92
10.2. Dueling and Classical DQN Achieve Equivalent Performance	92
10.3. Double DQN Underperforms Relative to the Other Two Variants	93
10.4. Training Dynamics Validate the Experimental Protocol	93
11. Limitations	94
12. Future Work	94
13. Chapter Summary	95
General Conclusion	97
Bibliographical References	105

List of tables

Table1. Fixed Time ,Actuated and Adaptive Control Comparison.....	
Table2. Traffic Signal Interpretation with (MDP).....	
Table3. Comparison of the methods used in each study.....	
Table4. Comparison of the methods.....	
Table5. The difference between the four scenarios.....	
Table6. The versions of software tools.....	
Table7. The key hyperparameters of the Prioritized Experience Replay implementation used across all three model variants.....	
Table8. Complete set of shared hyperparameters used across all three models.....	
Table9. Summary of Shared and Variant-Specific Components Across the Three DQN Architectures...	
Table10. Mean $\pm$ CI ₉₅ performance over 20 evaluation episodes on the high traffic scenario. Bold values indicate the best result for each metric.....	
Table11. Percentage improvement over fixed-time baseline. Positive values indicate improvement in all cases.....	
Table12. Standard deviation of waiting time and throughput across 20 evaluation episodes.....	

List of figures

Figure01. A simple representation of ITSs information chain.....	
Figure02. Providentia distribution locations	
Figure03. The Classical DQN policy network structure.....	
Figure04. The difference between Classical DQN and double DQN architecture.....	
Figure05. The dueling DQN policy network structure.....	
Figure06. SUMO's (NETEDIT) network editor.....	
Figure07. Simulated Road Network Constructed in SUMO for the Experimental Study.....	
Figure08. Epsilon-greedy Decay Schedule.....	
Figure09. Replay Buffer Growth	
Figure10. PER Beta Annealing Importance correction schedule.....	
Figure11. Training Loss Convergence	
Figure12. The signed percentage improvement of each DQN variant over the fixed time baseline for each metric.....	
Figure13. The absolute mean values with confidence interval error bars for all four models across all eight metrics	
Figure14. The episode level distribution of each model's performance across the 20 paired evaluation episodes for all eight metrics.....	
Figure15. Normalized multi dimensional view of the performance of all four Models.....	

General Introduction

1. Context and Motivation

Urban mobility has become one of the defining challenges of the twenty first century. Rapid population growth and the sustained expansion of vehicle ownership have placed increasing pressure on road infrastructure, particularly at signalized intersections, which represent the primary bottlenecks in dense urban networks. Traffic congestion at these intersections generates substantial economic losses through wasted fuel and lost productivity, contributes to harmful atmospheric emissions, and degrades the quality of life of urban populations. The management of traffic signals is therefore a critical lever for improving the overall efficiency of transportation systems.

Conventional traffic signal control strategies most notably fixed time control, which applies predetermined timing plans regardless of actual traffic conditions have been widely deployed due to their operational simplicity. However, their inability to respond to the dynamic and stochastic nature of real traffic demand makes them fundamentally inadequate for modern urban environments, where demand fluctuates unpredictably across the day and across different intersections. The gap between the rigidity of fixed time control and the complexity of real traffic has motivated decades of research into adaptive and intelligent signal control strategies.

The emergence of deep reinforcement learning (DRL) has opened a new and promising direction in this field. By enabling a control agent to learn optimal decision policies directly from interaction with a traffic simulation environment without requiring an explicit model of traffic dynamics DRL offers the potential for adaptive signal control that continuously improves through experience and generalizes across a wide range of traffic conditions. Among DRL approaches, value based methods built on the Deep Q Network framework and its architectural variants have attracted particular attention for their demonstrated effectiveness in sequential decision-making problems.

2. Problem Statement

Despite the growing body of research on DRL based traffic signal control, a critical gap persists in the existing literature. Most studies evaluate a single algorithm in isolation, without conducting a systematic and controlled comparison between architectural variants of the same base framework. In particular, three prominent variants of the Deep Q Network (Classical DQN, Double DQN, and Dueling DQN) have been applied to traffic signal control in separate studies, but under different experimental

conditions, different network topologies, different state representations, and different evaluation metrics. This lack of standardized comparative evaluation makes it impossible to draw reliable conclusions about which variant is most suitable for adaptive traffic signal control, and why.

A second gap concerns the multi-intersection setting. Many of the existing studies are conducted on isolated single intersections, which does not capture the coordination dynamics that emerge when multiple agents operate simultaneously in a shared traffic network. Scalable learning strategies such as parameter sharing, which could address this limitation efficiently, have received limited attention in comparative evaluations.

This thesis addresses both gaps through the following central research question: among Classical DQN, Double DQN, and Dueling DQN, all equipped with Prioritized Experience Replay and applied within a unified multi-intersection experimental framework, which architecture delivers the best adaptive traffic signal control performance under high demand urban conditions, and what are the algorithmic reasons for the observed differences?

3. General Objective

The general objective of this thesis is to conduct a rigorous, fair, and comprehensive comparative evaluation of three value based deep reinforcement learning architectures (Classical DQN, Double DQN, and Dueling DQN) applied to adaptive traffic signal control in a multi-intersection urban environment, with the aim of identifying the best performing architecture and providing a theoretically grounded explanation for its superiority.

4. Specific Objectives

This general objective is pursued through four specific objectives:

Objective 1: Design and implement a unified experimental framework in which all three DQN variants are trained and evaluated under strictly identical conditions same environment, same state representation, same reward function, same hyperparameters, and same evaluation metrics ,so that any observed performance difference can be attributed exclusively to architectural differences.

Objective 2: Evaluate each controller against a fixed time baseline across eight complementary performance metrics covering traffic efficiency, network throughput, and environmental impact, and quantify the improvement delivered by each DRL variant over conventional signal control.

Objective 3: Conduct a comparative analysis of the three DQN variants to determine whether the architectural innovations of Double DQN and Dueling DQN provide measurable performance benefits over Classical DQN in the specific context of binary action traffic signal control.

Objective 4: Assess the out of distribution generalization capability of the learned policies by evaluating on a high demand scenario that was deliberately withheld from the training curriculum, and determine whether DRL based controllers trained on moderate demand can transfer effectively to unseen high demand conditions.

5. Hypothesis

This thesis is guided by the following research hypothesis: the architectural decomposition introduced by Dueling DQN which separates the estimation of state value from the estimation of action advantage provides a representational advantage that is particularly well suited to the traffic signal control problem, where the quality of a traffic state and the differential benefit of phase switching are conceptually distinct quantities. This advantage is expected to manifest as consistently superior performance across all evaluated metrics and greater policy consistency across different demand realizations, compared to Classical DQN and Double DQN.

6. General Methodology

The methodology adopted in this thesis is organized around a controlled comparative experimental design. A **four intersection urban traffic network** is implemented in the **SUMO microscopic traffic simulator**. The traffic signal control problem is formalized as a Markov Decision Process, with a carefully designed state representation encoding local traffic conditions at each intersection, a binary action space, and a multi objective reward function. A multi agent architecture with parameter sharing is adopted, in which all agents share a single neural network and contribute to a common Prioritized Experience Replay buffer.

Three instantiations of this framework are implemented, one for each DQN variant. All three are trained over **700,000 simulation steps** on a rotating curriculum of low, medium, and mixed traffic demand scenarios. They are then evaluated exclusively on a high demand scenario withheld from training, **over 20 independent paired episodes**, against each other and against a fixed time baseline. Performance is assessed across eight metrics and analyzed through statistical comparison including confidence intervals, standard deviations, and episode level distribution analysis.

7. Structure of the Thesis

This thesis is organized into five chapters, preceded by this general introduction and followed by a general conclusion.

Chapter 1 establishes the theoretical and conceptual foundations of the work. It introduces the key characteristics of urban traffic and congestion, the role of traffic signal control systems and their classical methods, the Intelligent Transportation Systems framework, and the machine learning foundations underpinning the proposed approach, including reinforcement learning, Markov Decision Processes, Q learning, and Deep Q Networks.

Chapter 2 provides a comprehensive review of the existing literature on traffic signal control, progressing from classical fixed time and actuated methods through intelligent and optimization based approaches to the deep reinforcement learning methods at the core of this thesis. It documents the limitations of existing work and identifies the specific research gap ,the absence of a rigorous controlled comparison between DQN variants under unified experimental conditions that directly motivates this study.

Chapter 3 presents the design of the comparative framework proposed in this thesis. It covers the general architecture of the proposed solution, the description of the retained road network, the formal problem formulation as a Markov Decision Process with detailed specification of the state representation, action space, and reward function, the architectural descriptions of the three DQN variants, the performance metrics used for evaluation, and the training and evaluation scenarios.

Chapter 4 describes the concrete implementation and experimental execution of the framework. It documents the hardware and software environment, the construction of the simulated network in SUMO, the traffic generation procedure and training curriculum, the implementation of the three DQN variants and their shared components, the complete set of learning parameters, and the evaluation protocol including the fixed time baseline and the statistical analysis framework.

Chapter 5 presents the experimental results, analysis, and discussion. It reports the learning dynamics observed during training, the performance of all four controllers across all eight metrics under the high demand evaluation scenario, a metric by metric analysis, a comparative analysis between the three DQN variants, episode level distribution analysis, the limitations of the study, and the perspectives for future work.

The thesis closes with a **General Conclusion** that synthesizes the principal findings, answers the central research question, revisits the limitations of the work in their broader context, and outlines the most promising directions for future research.

Chapter 1: Theoretical Foundations

1. Introduction

Urbanization and the rapid growth of vehicle ownership have significantly increased the complexity of traffic systems in modern cities. As a result, traffic congestion has become one of the most critical challenges facing urban mobility, leading to increased travel delays, fuel consumption, and environmental pollution. Efficient traffic management, particularly at signalized intersections, is therefore essential to improving the overall performance of transportation networks.

Traditional traffic signal control methods, such as fixed time and actuated control, have been widely used due to their simplicity and ease of implementation. However, these approaches rely on predefined rules or limited local information, making them insufficient for handling the dynamic and stochastic nature of real world traffic conditions. Their inability to adapt in real time often results in suboptimal performance, especially in highly congested urban environments.

To overcome these limitations, Intelligent Transportation Systems (ITS) have been introduced as an integrated framework that leverages sensing, communication, and data processing technologies to improve traffic efficiency and safety. More recently, the integration of Artificial Intelligence (AI), particularly Reinforcement Learning (RL), has opened new perspectives for adaptive and data driven traffic signal control. These approaches enable systems to learn optimal control policies through interaction with the environment, allowing for real time and autonomous decision making.

In this context, this chapter provides the fundamental concepts required to understand intelligent traffic management systems. It first introduces the characteristics of urban traffic and congestion, followed by an overview of traffic signal control strategies. Then, it presents the basics of machine learning and reinforcement learning, including Markov Decision Processes (MDP), Q learning, and Deep Q Networks (DQN), which form the foundation of the proposed approach

2. Urban Traffic and Congestion

Urban traffic systems are characterized by complex and dynamic interactions between vehicles, infrastructure, and control mechanisms. With the continuous growth of urban populations and vehicle ownership, traffic demand has increasingly exceeded the available road capacity, leading to frequent congestion phenomena. Understanding the fundamental characteristics of traffic flow and the causes of congestion is essential for designing effective traffic management strategies.

In this section, the key parameters of traffic flow, including flow, speed, and density, are introduced, followed by an analysis of congestion and its main contributing factors. [3] [4] [5] [15] [16] [26]

Traffic flow: (or flow rate) it is the number of vehicles passing a specific point on a roadway during a given time interval, typically expressed in vehicles per hour (veh/h). It represents the rate at which traffic demand is served at a given location and is a fundamental parameter in traffic flow analysis.[16] [26]

Density: “The number of vehicles on a roadway segment averaged over space, usually expressed as vehicles per mile or vehicles per mile per lane. (see also: volume density, sometimes referred to as density timing)”. [3] [26]

Speed (Average Speed): Speed in traffic flow refers to the rate at which vehicles travel along a roadway and is typically expressed in kilometers per hour (km/h). In traffic analysis, the space mean speed is commonly used, as it represents the average speed of vehicles over a given road segment and provides a more accurate measure of traffic conditions.[26]

Space mean speed: It is defined as the average speed of all vehicles occupying a segment of roadway at a given time

2.1. Traffic congestion

2.1.1. Definition

Traffic congestion is a situation where the demand for traffic exceeds the available supply. It often occurs during peak hours (the period with the highest traffic density during the day).[18]

2.1.2. Primary Causes

Bottlenecks: Permanent physical restrictions like narrow bridges or merging lanes.

Traffic Incidents: Unpredictable events such as crashes, breakdowns, or debris.

Weather: Rain, snow, and fog that force drivers to slow down.

Work Zones: Construction activities that temporarily reduce lane availability.

Poor Signal Timing: Inefficient traffic light cycles that disrupt flow

2.1.3. Significant Impacts

Economic Loss: Wasted fuel and lost productivity cost billions annually.

Environmental Damage: Increased emissions from idling vehicles.

Public Health: Linked to stress, respiratory issues, and sedentary lifestyles.

Emergency Services: Delays in response times for fire trucks and ambulances.

2.2. Key Performance Indicators (KPIs)

These KPIs are commonly used to evaluate the performance of traffic signal control strategies.

Delay :

- “1. The additional travel time experienced by a driver, passenger, or pedestrian.
- 2 .A detector parameter typically used with stop-line, presence mode detection for turn movements from exclusive lanes”. [3]

Queue :”A line of vehicles, bicycles, or persons waiting to be served by the system in which the flow rate from the front of the queue determines the average speed within the queue. Slowly moving vehicles or people joining the rear of the queue are usually considered part of the queue. The internal queue dynamics can involve starts and stops. A faster moving line of vehicles is often referred to as a moving queue or a platoon”[3].

So **Queue length** refers to the number of vehicles waiting in line at an intersection during a red signal.

Waiting Time: it is the amount of time a vehicle spends stopped or moving very slowly while waiting for the traffic signal to change .

Throughput: it is the number of vehicles that successfully pass through an intersection within a given period of time.

In summary, urban traffic congestion results from the imbalance between traffic demand and road capacity, and is influenced by several interrelated factors such as flow dynamics, vehicle interactions, and infrastructure limitations. These challenges highlight the need for efficient traffic control strategies, particularly at signalized intersections, where congestion is most critical.

Therefore, the following section focuses on traffic signal control systems and the different methods used to regulate traffic flow and reduce congestion.

3. Traffic Signal Control Systems

3.1. Role of Traffic Signals in Intersection Control

Signalized intersections play a fundamental role in urban traffic management, as they regulate the movement of vehicles and pedestrians at points where traffic streams intersect. Without proper control, these intersections can become major sources of congestion, delays, and conflicts due to competing traffic flows.

Traffic signals are designed to assign the right of way to different traffic movements in a controlled and organized manner. By alternating between red, yellow, and green phases, they ensure safe and efficient utilization of intersection capacity while minimizing conflicts between vehicles from different directions.

At signalized intersections, traffic control strategies aim to optimize key performance indicators such as delay, queue length, and throughput. The effectiveness of these systems depends on how well the signal timing adapts to varying traffic conditions. Poorly designed or static signal plans can lead to increased waiting times and inefficient use of road capacity, especially under dynamic and congested traffic conditions.

Therefore, intersection control is considered a critical component of traffic management systems, and improving signal control strategies has become a major research focus in order to enhance traffic efficiency and reduce congestion in urban environments.

3.2. Traffic Signal Control Fundamentals

Recently, traffic congestion has become one of the most pressing challenges facing urban transport systems. Therefore, signalized intersections are a crucial component in solving this problem as traffic signal control systems are designed to coordinate traffic flow. In this area, we present important fundamental concepts that provide a basis for further analysis [3] [4] [5] [16].

Phase: “A controller timing unit associated with the control of one or more movements. The MUTCD defines a phase as the right of way, yellow change, and red clearance intervals in a cycle that are assigned to an independent traffic movement”[3]. So **signal phase** (commonly referred to as a phase) represents a portion of the signal cycle during which a specific traffic movement is given the right of way.

Red time: “The period, expressed in seconds, in the signal cycle during which, for a given phase or lane group, the signal is red”[3].

The yellow time: is the time interval between the green and red lights that warns drivers that the light is about to change.

Green time: “The duration, in seconds, of the green indication for a given movement at a signalized intersection.”[3].

Maximum Green: “The maximum length of time that a phase can be green in the presence of a conflicting call”[3].

Minimum Green: “The first timed portion of the green interval which may be set in consideration driver expectancy and the storage of vehicles between the detectors and the stop line when volume density or presence detection is not used”[3].

Occupancy: “The percent of time that a detector indicates a vehicle is present over a total time period.”[3].

Single-intersection control: it refers to traffic signal control strategies that operate independently at a single intersection. In this approach, signal timing decisions are optimized based solely on traffic conditions at that intersection, without considering interactions with adjacent intersections .

Network-level Control: it refers to coordinated traffic signal control strategies applied across multiple intersections in a traffic network. The objective is to optimize traffic performance at the network scale by coordinating signal timings between intersections to improve traffic flow and reduce congestion.

A route: it is a path leading from one place to another. It is characterized by its length, the number of lanes it has, and the directions of traffic flow within it.

An intersection: it is a point of convergence in a road system where two or more roads meet. To manage these intersections safely, the area is divided into specific zones.

1. **Conflict Zone:**The conflict zone is the area within an intersection where vehicle paths intersect, creating potential points of conflict and requiring coordinated control
2. **Storage Zone:**The storage zone is the upstream area before the conflict zone where vehicles queue and wait prior to entering the intersection
3. **Exit Zone:**The exit zone is the downstream area after the conflict zone where vehicles leave the intersection, enabling its clearance for subsequent traffic

3.3. Traffic Signal Control Methods

Traffic signal control systems rely on several strategies that vary in their level of adaptability to traffic conditions. Among the most commonly used methods are fixed time control, actuated control, and adaptive control. Each system reflects a different level of responsiveness to changes in traffic flow.

3.3.1. Fixed-Time Control

It is a traditional control system based on fixed schedules that are preset according to historical traffic studies and operate periodically without change during operation. Fixed Time Control does not rely on real time data. And does not include feedback (Open Loop Control). Timing is constant during the specified time period (Peak/Off Peak) and It depends on the following:

1. **Cycle Length:** The total time required to complete all phases of the signal once before repeating.”The time required for a complete sequence of signal indications”[3].
2. **Green Split:** The proportion of green time allocated to each phase compared to the total cycle length.
3. **Offset:** The time difference between the start of the green phase at a given junction and the start of the green phase at an adjacent junction within a coordinated network.

3.3.2. Actuated Control

A control system that uses sensors (Loop Detectors, Cameras) to detect the presence of vehicles, and adjusts the signal time based on actual demand at the intersection. It relies on closed loop local control And the green phase can be extended or shortened.

There are two main types .Semi Actuated and Fully Actuated

3.3.3. Adaptive Control

A dynamic control system that adjusts signaling strategies in real time based on a comprehensive analysis of the traffic network situation, and may use optimization algorithms or machine learning. It relies on real time data from multiple intersections and operates within an optimization based control framework.

Actuated Control can be used for Predictive models ,Mathematical optimization algorithms and Reinforcement Learning.

Property	Fixed-Time	Actuated	Adaptive
Data Type	It does not rely on real time data.	It relies on local sensors	It relies on real time data from the network.
Complexity	Low	Medium	High
Adaptability Level	None	Medium	High
Control field	Single intersection	One intersection	Intersection network
Efficiency in congestion	Limited	Relatively better	High

table 1: *Fixed Time ,Actuated and Adaptive Control Comparison*

3.4. Limitations of Traditional Traffic Signal Control Methods

Traditional traffic signal control methods, including fixed-time and actuated control systems, have been widely used due to their simplicity and ease of implementation. However, these approaches exhibit several limitations when applied to modern urban traffic environments characterized by high variability and uncertainty.

One of the primary limitations of fixed time control is its reliance on predefined timing plans based on historical traffic data. Such systems lack real time adaptability and cannot respond effectively to sudden fluctuations in traffic demand, leading to inefficient signal timing and increased delays.

Although actuated control systems improve upon fixed time strategies by using local sensors to detect vehicle presence, they remain limited in scope. These systems typically make decisions based on local information at a single intersection, without considering the broader traffic network. As a result, they may fail to achieve global optimization and can lead to suboptimal coordination between adjacent intersections.

Furthermore, both fixed time and actuated control methods struggle to handle the stochastic and dynamic nature of traffic flow. They are generally based on heuristic or rule based approaches, which lack the ability to learn and adapt from experience. This limitation becomes particularly significant in complex urban environments where traffic patterns are continuously changing.

Consequently, these shortcomings highlight the need for more advanced and adaptive traffic control strategies capable of leveraging real time data and learning optimal control policies. This has led to the development of intelligent and data driven approaches, particularly those based on Artificial Intelligence and Reinforcement Learning.

To overcome these limitations at a system wide level, Intelligent Transportation Systems (ITS) have been developed as a broader framework integrating sensing, communication, and control technologies.

4. Intelligent Transportation Systems -ITS

“Intelligent Transportation Systems, also known as ITS, are advanced software programmes that aim to offer cutting edge services relating to various modes of transportation and traffic management and facilitate users to be more knowledgeable and make safer, more organized, and smarter use of

transportation networks. With the aid of progressive innovations like GSM, WiFi, Cellular Data, GPS, and GPRS, the Intelligent Transport Systems offer transportation businesses a centralised way to schedule and monitor vehicles. Better public transportation services are made possible by the Intelligent Transport System by taking into account bus revenue, public safety, and protection .”[1]

It is defined as an integrated system of digital technologies that are integrated into the transportation infrastructure (roads, vehicles, traffic signals, control centers, etc.). This includes sensing, communications, data processing, control systems, and artificial intelligence within the transportation system, with the aim of improving traffic flow efficiency, enhancing safety, reducing congestion and emissions, and supporting decision making in traffic network management. These systems rely on real time data collection and analysis using advanced algorithms, including artificial intelligence techniques, to enable a dynamic response to changing traffic conditions.

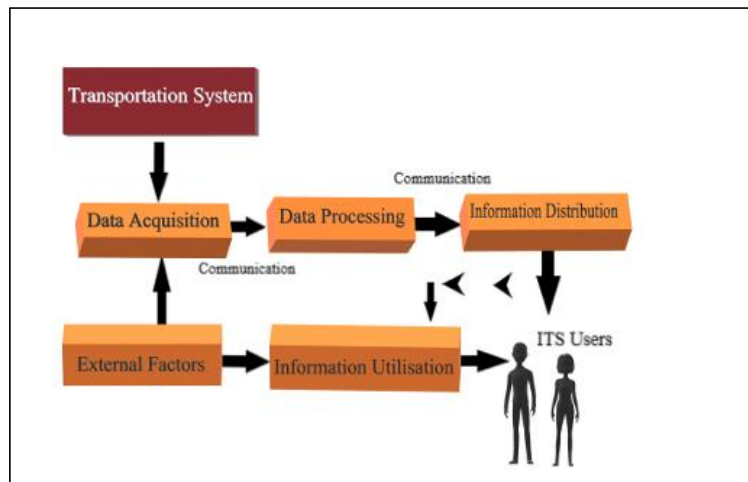


Figure.1: *A simple representation of ITSs information chain [1]*

As has been indicated, this system contributes to preventing approximately 400,000 to 600,000 accidents annually, and reducing travel time by about 42% in the United States alone [2]. The Providentia++ project can be considered a good example of intelligent transportation systems, and its deployment locations can be seen in the following figure.

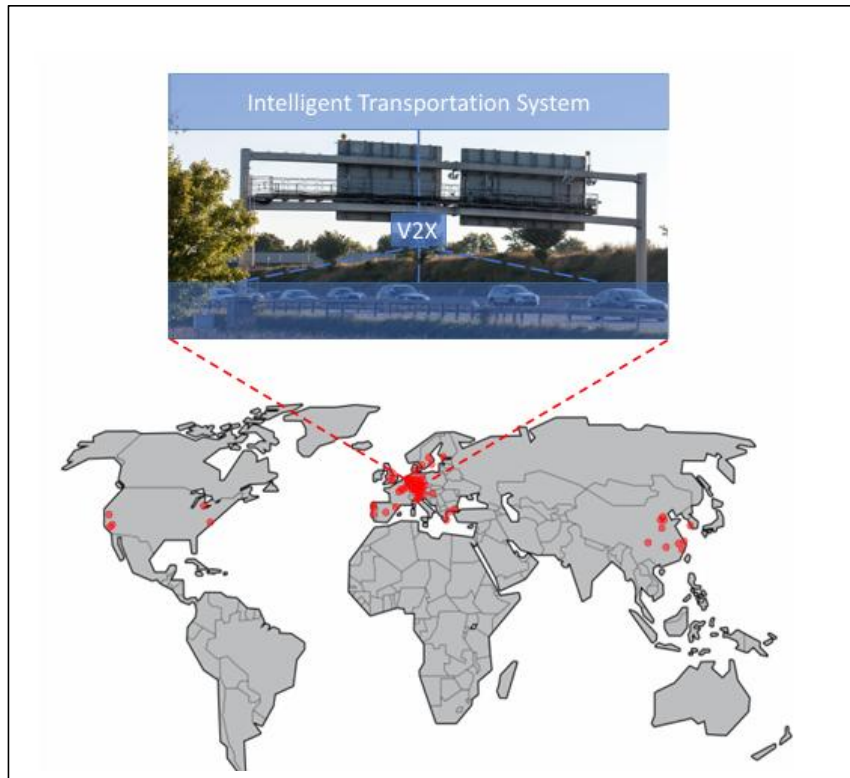


Figure.2: *Providentia distribution locations* [2].

4.1. Components of Intelligent Transportation Systems

Intelligent Transportation Systems (ITS) are composed of several interconnected components that work together to monitor, analyze, and control traffic conditions in real time. These components can be broadly categorized into sensing, communication, data processing, and control systems.

Sensing technologies are responsible for collecting real time traffic data. These include loop detectors, cameras, radar sensors, and GPS enabled devices, which provide information about vehicle count, speed, and occupancy.

Communication systems enable the transmission of data between different elements of the transportation network. Technologies such as wireless communication, cellular networks, and dedicated short range communication (DSRC) facilitate data exchange between vehicles, infrastructure, and control centers.

Data processing and management systems analyze the collected data using computational algorithms and models. These systems transform raw data into meaningful information, such as traffic state estimation and prediction.

Control systems utilize the processed information to make decisions and implement traffic management strategies. Examples include adaptive traffic signal control, route guidance systems, and incident management systems.

Together, these components enable **ITS** to provide a coordinated and data-driven approach to traffic management.

4.2. Role of ITS in Reducing Traffic Congestion

Intelligent Transportation Systems play a crucial role in mitigating traffic congestion by enabling real time monitoring, adaptive control, and informed decision-making. Unlike traditional systems, ITS leverage continuous data collection and analysis to respond dynamically to changing traffic conditions.

One of the key contributions of ITS is the optimization of traffic signal timing based on real time traffic flow. By adjusting signal phases and durations according to current demand, ITS can significantly reduce vehicle delays and queue lengths at intersections.

In addition, ITS support traffic management through advanced traveler information systems, which provide drivers with real time updates on traffic conditions, alternative routes, and incidents. This helps distribute traffic more efficiently across the network and prevents the formation of bottlenecks.

Furthermore, ITS enhance coordination between multiple intersections, allowing for network level optimization rather than isolated local control. This coordinated approach improves overall traffic flow and increases roadway capacity utilization.

As a result, ITS contribute to reducing congestion, improving travel time reliability, and enhancing the overall efficiency of urban transportation systems.

4.3. Limitations of ITS without Artificial Intelligence

Despite their significant advantages, traditional ITS implementations face several limitations when not integrated with advanced Artificial Intelligence techniques. Most conventional ITS rely on predefined rules, statistical models, or optimization methods that may not fully capture the complexity and dynamic nature of urban traffic systems.

One major limitation is the lack of learning capability. Traditional ITS systems do not adapt their behavior based on past experiences, which restricts their ability to improve performance over time in evolving traffic conditions.

Additionally, these systems often depend on simplified assumptions and models that may not accurately represent real world traffic dynamics. This can lead to suboptimal decision making, especially in highly congested or unpredictable environments.

Another challenge is scalability. As traffic networks grow in size and complexity, conventional ITS approaches may struggle to process large volumes of data and coordinate decisions across multiple intersections efficiently.

Furthermore, without AI, ITS systems have limited capability to perform predictive and proactive control. They primarily react to current conditions rather than anticipating future traffic states.

These limitations highlight the need for integrating Artificial Intelligence, particularly data driven and learning based approaches, to enhance the adaptability and performance of ITS in modern traffic management.

Among these AI techniques, Machine Learning provides the foundational tools for building adaptive and learning based traffic control systems, as introduced in the following section.

5. Machine Learning Fundamentals

Machine learning is a branch of artificial intelligence that is gaining increasing importance in solving complex real world problems by learning from data and interactions with the environment. These techniques can support intelligent decision making in dynamic systems such as traffic networks.[6]

5.1. Supervised learning : This branch of machine learning relies on learning from a dataset containing both inputs and their target outputs. The goal is to build a model capable of predicting the correct outputs of new, unseen data.

5.2. Unsupervised learning: In this other branch ,the algorithm attempts to analyze unlabeled data in order to discover hidden patterns and identify similarities or clusters in the data without predefined classifications or targeted outputs.

5.3. Reinforcement learning: is a machine learning paradigm in which an agent learns to make decisions by interacting with an environment. The agent receives rewards or penalties based on its actions and gradually learns a strategy that maximizes long term rewards.

Urban traffic systems are known for their dynamic nature, with traffic flows constantly changing and becoming less predictable over time. Reinforcement learning is well suited to such environments because it allows systems to adapt their decisions based on real time interactions with the environment.

Given the dynamic and uncertain nature of urban traffic, Reinforcement Learning is particularly well suited, as it enables a controller to learn optimal decisions through direct interaction with the traffic environment.

The following section details the key components of the RL framework.

6. Reinforcement Learning

The reinforcement learning framework contains several key components that are defined in this section. [6]

Agent: The learner or controller is the agent who interacts with the environment, observes and determines the action that should be taken.

Environment: It is the external system that includes all the external elements with which the agent interacts and which affect the outcome of its actions, from which he receives feedback in the form of rewards and state transitions.

State (s): Represents the current situation of the environment and contains all the relevant information necessary for decision making.

Action (a): represents a decision taken by the agent that affects the future state of the environment.

Reward: A numerical feedback signal that guides the learning process provided by the environment to evaluate the quality of the agent's action.

In traffic signal control applications, the reward function can be defined using various performance related parameters that reflect the current traffic conditions. These may include measures such as **waiting time**, **queue length**, delay, or more advanced metrics. Among these, **pressure** is commonly used and is defined as the difference between incoming and outgoing traffic queues. It captures the imbalance in traffic flow at an intersection and is often employed to guide the agent toward selecting signal phases that reduce congestion and improve overall traffic efficiency [17].

Policy (π): the strategy used by the agent to select actions based on the current state and the objective of the agent is to learn an optimal policy that maximizes cumulative reward.

Value Function: It estimates how the long term benefit of being in a given state while following a specific policy based on their expected future reward.

Q-Function: It measures how beneficial it is to perform a certain action in a particular state

Return: It is defined as the sum of rewards collected over time

Discount Factor (γ): It determines the importance of future rewards compared to immediate rewards.

To formally define this interaction between the agent and the traffic environment, the Markov Decision Process (MDP) framework is introduced in the following section.

7. Markov Decision Process (MDP)

In reinforcement learning, the interaction between the agent and the environment is commonly formalized using a Markov Decision Process, which provides a mathematical framework for sequential decision making under uncertainty. [6] [7]

$$\text{MDP} = (S, A, P, R, \gamma)$$

A Markov Decision Process is defined by a tuple (S, A, P, R, γ) , where :

- S = State
- A = Action
- P = Transition Probability
- R = Reward Function
- Γ = Discount Factor

Transition Probability : The probability of transitioning from one state to another after a certain action is performed.

Markov Property: The Markov property states that the future state depends only on the current state and the chosen action, and not on the sequence of past states.

To improve traffic signals, the Markov Decision Process (MDP) formulation allows the control problem to be expressed as a sequential decision making process. This table illustrates how:

MDP Component	Traffic Signal Interpretation
State	queue length, waiting time, signal phase
Action	switch phase / keep phase
Reward	negative queue length or waiting time
Transition	traffic flow dynamics

table2: Traffic Signal Interpretation with (MDP)

8. Q-Learning

It is a value based reinforcement learning method in which the agent learns the expected return of taking a given action in a specific state.

This technique estimates the optimal value function of action $Q(s,a)$, which represents the long term reward expected when executing action a in state s [6] [7] .

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

The update rule adjusts the previous Q value by adding a fraction (α) of the difference between the new target value $r + \gamma \max_{a'} Q(s',a')$ and the current estimate $Q(s,a)$.

Temporal Difference (TD): Temporal Difference represents the error between the predicted value of a state and a new estimate obtained after observing the next state and reward.

$$\delta = r + \gamma \max_{a'} Q(s',a') - Q(s,a)$$

the discrepancy between the current value estimate and the updated target. this means it is the difference between the value expected and the value discovered after interacting with the environment.

Bootstrapping: It refers to the updating value of a state using the estimated value of the next state. rather than waiting for the final outcome.

Off-Policy Learning: It refers to learning the optimal policy independently from the policy used to explore the environment which the agent can learn the value of the optimal policy while following a different behavior policy.

In traffic signal control, Q Learning enables the agent to learn the signal control policy that minimizes congestion and waiting times.

However, Q Learning relies on a tabular representation of the Q function, which becomes infeasible when the state space is large or continuous, as is the case in real world traffic environments. This limitation is addressed by Deep Q Networks, presented in the following section.

9. Deep Q-Network (DQN)

When the state space becomes large or continuous, storing Q values in a table becomes impractical.

Deep Q Networks are an extension of Q Learning which addresses this limitation by approximating the Q function using a neural network. [6] [7]

Neural Network Approximation: In Deep Q Networks, a neural network is used to approximate the Q function $Q(s,a)$.

Which:

- **Input :** state
- **Outputs :** estimated Q values for all possible actions.

Experience Replay: A mechanism that retains the agent's past transition experiences

(s, a, r, s') in a replay buffer, from which mini batches are sampled uniformly at random during training.

Target Network: It is a copy of the main Q network whose parameters are updated periodically. using a separate target network reduces oscillations and improves the stability of Q value updates.

ϵ -greedy Exploration: The ϵ -greedy strategy balances exploration and exploitation .With probability ϵ the agent explores the environment randomly, while with probability $1-\epsilon$ it selects the action with the highest estimated Q value.

In traffic signal control, Deep Q Networks allow the agent to learn effective signal control strategies to reduce queue lengths and waiting times.

10. Conclusion

This chapter has laid the theoretical groundwork for understanding intelligent traffic signal control systems. It began by introducing the fundamental characteristics of urban traffic flow and the growing challenge of congestion, highlighting the gap between traffic demand and road capacity. Key performance indicators such as delay, queue length, and throughput were presented as essential metrics for evaluating control strategies.

The limitations of traditional signal control methods (fixed time and actuated control) were examined, showing their inability to adapt to the dynamic and stochastic nature of real world traffic. This motivated the need for more intelligent approaches, leading to the introduction of Intelligent Transportation Systems (ITS) as a comprehensive framework integrating sensing, communication, and data processing technologies.

The chapter then presented the foundational concepts of machine learning, with particular emphasis on Reinforcement Learning as the most suitable paradigm for adaptive traffic control in dynamic environments. The Markov Decision Process (MDP) formalism was introduced to model the traffic signal control problem as a sequential decision making task. Q Learning was discussed as a value based RL method, and its extension, the Deep Q Network (DQN), was presented as a solution to the scalability limitations of tabular approaches.

Together, these concepts form the theoretical basis upon which the proposed intelligent traffic signal control system is built. Building on this foundation, the following chapter reviews the existing literature and state of the art approaches in RL based traffic signal control, identifying the main research trends and open challenges that motivate the contributions of this work.

Chapter 2: State of the Art

1. Introduction

Traffic congestion has become one of the most critical challenges in modern urban environments, driven by rapid population growth, urbanization, and the increasing number of vehicles. This phenomenon leads to significant economic losses, increased fuel consumption, environmental pollution, and reduced quality of life. Among the various components of transportation systems, traffic signal control plays a central role in regulating traffic flow and mitigating congestion, particularly in dense urban networks with multiple intersections.

Traditional traffic signal control methods, such as fixed time and vehicle actuated strategies, have been widely deployed due to their simplicity and ease of implementation. However, these approaches rely on predefined rules or limited real time information, which makes them insufficient for handling highly dynamic and complex traffic conditions. As a result, their adaptability and overall performance remain limited, especially in large scale urban environments.

To address these limitations, a wide range of intelligent and data driven approaches has been proposed. These include fuzzy logic controllers, evolutionary optimization techniques, and hybrid methods that combine multiple paradigms to improve decision making. While these methods offer improved adaptability, they often require manual design, extensive parameter tuning, or lack scalability when applied to complex traffic networks.

More recently, reinforcement learning (RL) has emerged as a promising framework for traffic signal control, enabling agents to learn optimal control policies through interaction with the environment. Building on this paradigm, deep reinforcement learning (DRL) integrates neural networks to handle high dimensional state spaces, making it particularly suitable for real world traffic scenarios. Among DRL approaches, value based methods such as Deep Q Network (DQN) and its advanced variants, including Double DQN and Dueling DQN, have shown strong potential in improving learning stability and control performance.

In this context, this chapter provides a comprehensive review of the existing literature on traffic signal control, progressing from classical methods to advanced deep reinforcement learning techniques. The chapter is organized as follows:

- **Section 2** presents traditional traffic signal control methods.
- **Section 3** reviews intelligent and optimization based approaches.
- **Section 4** introduces reinforcement learning for traffic signal control.

- **Section 5** discusses deep reinforcement learning.
- **Section 6** details DQN and its main variants, including Double DQN and Dueling DQN.
- **Section 7** explores multi agent reinforcement learning and parameter sharing strategies.
- **Section 8** summarizes the limitations of existing work.
- **Section 9** identifies the research gap addressed in this thesis.

2. Traditional traffic signal control methods

Traditional traffic signal control methods represent the earliest approaches used to regulate traffic flow at intersections. These methods are generally based on predefined rules and limited real-time information, making them simple to implement but often insufficient in dynamic traffic environments. The most common classical approaches include fixed time control and vehicle actuated control.

Fixed time control relies on predetermined signal timing plans that are designed based on historical traffic data and remain unchanged regardless of real time traffic conditions. In contrast, vehicle actuated control adjusts signal phases dynamically using real time data collected from sensors such as inductive loop detectors, allowing for more responsive traffic management.

In a study conducted in 2025 at the University of Tirana, Albania, entitled “*Microsimulation-Based Evaluation of Fixed-Time and Vehicle-Actuated Traffic Signal Control Strategies Using PTV Vissim in an Urban Corridor*”, the researchers compared fixed time control with actuated control at multiple urban intersections. To achieve this, they used PTV Vissim, a microscopic traffic simulation software, and conducted 50 simulation runs during peak hours to ensure statistical reliability. The actuated system utilized inductive loop sensors to detect vehicle presence and adjust signal phases accordingly. Performance was evaluated using metrics such as average speed, vehicle delay, queue length, and travel time[8].

The results demonstrated that vehicle actuated control significantly improved traffic performance. Specifically, vehicle delay was reduced by approximately 39%, average speed increased by about 33%, and travel time decreased by around 25%, with overall efficiency improvements ranging between 30% and 60%.

Similarly, a study published in 2023, entitled “*Efficiency of Vehicle Actuated Traffic Control on Isolated Intersection*”, evaluated these two approaches at a major intersection in Tashkent, Uzbekistan. The authors used PTV Vissim for simulation and TRL TRANSYT 15 to design fixed time signal plans based on intersection characteristics and traffic flow. The evaluation metrics included **average travel**

time and **queue length**. The results confirmed the superiority of vehicle actuated control, with queue lengths reduced by up to 65% and travel time reduced by up to 51%.[10] .

Despite these improvements, both studies highlight important limitations of classical approaches. Vehicle actuated systems rely heavily on physical sensors, which require installation, calibration, and maintenance, leading to increased operational costs. Moreover, these methods remain limited in their ability to handle highly complex and large scale traffic networks, as most evaluations are conducted on isolated intersections or specific corridors.

Overall, while classical traffic signal control methods are effective in relatively simple and stable traffic conditions, they lack the adaptability and scalability required for modern urban environments. These limitations have motivated the development of more advanced intelligent and data driven approaches, which are discussed in the next section.

3. Intelligent and Optimization-Based Approaches

To overcome the limitations of traditional traffic signal control methods, several intelligent and optimization based approaches have been proposed. These methods aim to improve adaptability and decision making by incorporating either rule based reasoning or optimization techniques.

In general, these approaches can be classified into three main categories:

- Rule based intelligent methods (e.g., Fuzzy Logic)
- Optimization based methods (e.g., Genetic Algorithms)
- Hybrid approaches combining both techniques

3.1. Fuzzy Logic Control

Traffic signal control using fuzzy logic is a widely adopted intelligent approach. Instead of relying on fixed thresholds, fuzzy controllers use linguistic variables such as “**high traffic density**” or “**short queue length**” to dynamically adjust traffic signal timing.[9]

A study published in 2019, entitled “*A Two-Stage Fuzzy Logic Adaptive Traffic Signal Control for an Isolated Intersection Based on Real Data Using SUMO Simulator*”, investigated the effectiveness of a fuzzy logic based control system. The proposed approach consists of two modules: a phase selection module to determine the next green phase, and a green phase extension module to adjust the duration of the green signal [11].

The system was evaluated using the SUMO traffic simulator based on traffic indicators such as queue length, traffic flow rate, and average vehicle waiting time. The results showed that the fuzzy logic controller outperformed the fixed time system by approximately 61.36%.

Despite these positive results, the study was conducted on a single intersection, which limits its generalization to large scale traffic networks. In addition, fuzzy logic systems require manual design of rules and membership functions, which can be difficult to optimize for complex environments.

3.2. Optimization-Based Approaches (Genetic Algorithm)

Another class of intelligent approaches is based on optimization techniques. Genetic Algorithms (GA) are widely used to search for optimal traffic signal timing plans by exploring a large space of possible solutions.

A study published in 2013, entitled “*Simulation and Optimization of an Isolated Intersection with Genetic Algorithms*”, aimed to reduce congestion and improve traffic flow using GA. In this approach, signal timing configurations are encoded as candidate solutions, which are iteratively improved using evolutionary operations such as selection, crossover, and mutation. [12]

The performance of each solution was evaluated using traffic simulation metrics, including average vehicle delay and traffic flow efficiency. The results demonstrated that the genetic algorithm can effectively identify signal timing strategies that outperform traditional methods.

However, this approach is computationally expensive and may require a large number of iterations to converge. Furthermore, similar to fuzzy logic approaches, the study was limited to a single intersection, which restricts its scalability.

3.3. Hybrid Approaches (Fuzzy Logic + Genetic Algorithm)

To combine the advantages of rule based reasoning and optimization, hybrid approaches have been proposed. These methods aim to enhance fuzzy logic controllers by optimizing their parameters using evolutionary algorithms.

A study published in 2016, entitled “*Fuzzy Control of Intersection Signal Based on Optimized Genetic Algorithm*”, proposed a hybrid traffic signal control strategy that integrates fuzzy logic and genetic algorithms. In this approach, fuzzy logic is used to make control decisions based on traffic conditions, while the genetic algorithm is employed to optimize the controller parameters [13].

The system was evaluated using traffic simulation based on indicators such as vehicle queues and traffic flow. The experimental results showed that the hybrid approach reduced average vehicle delay and improved overall traffic efficiency compared with traditional methods.

Despite these improvements, the study was still limited to isolated intersections, which raises concerns about its applicability to large-scale traffic networks.

3.4. Discussion and Transition

Overall, intelligent and optimization-based approaches provide improved adaptability compared to traditional traffic signal control methods. However, they still rely on predefined rules or offline optimization processes, which limits their ability to continuously learn and adapt in real time.

Moreover, their evaluation is often restricted to simplified environments, such as single intersections, which does not fully capture the complexity of real world traffic networks.

These limitations motivate the use of learning based approaches that can automatically adapt their behavior through interaction with the environment. Reinforcement learning, in particular, has emerged as a powerful framework for traffic signal control and is discussed in the next section.

4. Reinforcement Learning for Traffic Signal Control

Reinforcement Learning (RL) is a learning paradigm in which an agent learns to make optimal decisions through interaction with an environment by maximizing cumulative reward. Unlike traditional traffic signal control methods that rely on predefined rules or fixed timing plans, RL enables adaptive and data driven decision making based on real time traffic conditions.

In the context of traffic signal control, the problem is typically modeled as a sequential decision making process. The traffic signal controller is considered as the agent, while the traffic environment represents the system dynamics. The state usually includes traffic related features such as queue length, vehicle density, or waiting time. The action corresponds to selecting or switching traffic signal phases, and the reward is designed to reflect traffic performance objectives, such as minimizing vehicle delay, reducing queue length, or improving traffic flow efficiency.

This formulation allows the controller to learn optimal policies directly from interaction with traffic data, making RL particularly suitable for dynamic and uncertain environments. As a result, RL has been increasingly applied to traffic signal control problems, showing promising improvements over traditional approaches.

Several studies have explored the application of RL to traffic signal control. For example, a study entitled “**A Comparison of Reinforcement Learning Agents Applied to Traffic Signal Optimisation**” demonstrated that RL based approaches can learn adaptive signal control policies that outperform traditional fixed time methods in simulated environments .[19]

However, several challenges limit the effectiveness of classical RL methods in real world traffic systems. First, traffic environments are highly dynamic and non stationary, especially in multi intersection scenarios. Second, the state space can be large and complex, making it difficult for traditional RL methods to learn efficient policies. Third, the design of an appropriate reward function remains a critical issue, as it directly influences the learning behavior of the agent.

5. Deep Reinforcement Learning

Classical reinforcement learning methods rely on tabular representations of the value function, which become impractical when the state space is large or continuous. In traffic signal control, the state often includes high dimensional information such as queue lengths across multiple lanes, vehicle positions, or even visual observations from cameras. This complexity makes it difficult for traditional RL algorithms to scale effectively.

Deep Reinforcement Learning (DRL) addresses this limitation by integrating neural networks as function approximators to estimate value functions or policies. Instead of storing values in tables, DRL uses a neural network that takes the state as input and outputs either action values or action probabilities. This allows the model to generalize across similar traffic states and handle complex, high dimensional environments.

The main difference between classical RL and DRL lies in how the value function is represented. Classical RL methods use discrete tables and are suitable for small scale problems, whereas DRL replaces these tables with neural networks, enabling learning in large scale and continuous domains. As a result, DRL is more suitable for real world traffic signal control scenarios, where the environment is dynamic, complex, and data rich.

A representative example is the study “**Traffic Light Control Using Deep Policy-Gradient and Value-Function Based Reinforcement Learning**”, which proposes a DRL based approach for adaptive traffic signal control. In this work, the authors employ deep neural networks to process visual representations of the intersection and learn control policies using both policy gradient and value based

methods. The experiments, conducted using the SUMO simulator, show that the DRL based controller improves traffic flow efficiency and reduces vehicle delay compared to traditional approaches.[14]

Despite these advantages, DRL introduces additional challenges, including increased computational cost, training instability, and sensitivity to hyperparameter selection. These issues motivate the development of more robust architectures, such as Deep Q Networks (DQN) and their variants, which are discussed in the next section.

6. Deep Q-Network (DQN)

6.1. Classical Deep Q-Network (baseline DQN)

Deep Q Network (DQN) is an extension of the classical Q learning algorithm designed to handle high dimensional state spaces. Instead of using a tabular representation of the Q function, DQN employs a deep neural network to approximate the action value function $Q(s,a)$, enabling the agent to generalize across a large number of traffic states.

In the context of traffic signal control, the neural network takes the current traffic state as input, such as queue lengths, vehicle density, or waiting times, and outputs Q values for all possible actions, where each action corresponds to a traffic signal phase decision. The agent then selects the action with the highest Q value, aiming to maximize long term traffic performance.

One of the main challenges in combining Q learning with neural networks is training instability. To address this issue, DQN introduces two key mechanisms:

Experience Replay and Target Network.

- **Experience Replay** stores past interactions (s, a, r, s') in a replay buffer and samples them randomly during training. This breaks the correlation between consecutive samples and improves learning stability and efficiency.
- **Target Network** is a separate neural network used to compute target Q values. Its parameters are updated periodically from the main network, which reduces oscillations and divergence during training.

“Deep Reinforcement Q-Learning for Intelligent Traffic Signal Control with Partial Detection” (Ducrocq, 2021). In this work, the author proposes a DQN based traffic signal controller capable of operating under partial vehicle detection, which reflects realistic traffic sensing conditions. The

approach was evaluated using traffic simulation and compared with traditional actuated control systems. The results demonstrated that the DQN based controller can improve traffic performance even with limited detection capabilities. In addition, the study provides an open source implementation, which enhances reproducibility and practical applicability.[20]

Despite its success, DQN suffers from a known limitation: it tends to overestimate Q values due to the use of the same network for both action selection and evaluation. This overestimation can negatively affect learning performance and policy quality. To address this issue, improved variants such as Double DQN and Dueling DQN have been proposed, which are discussed in the following sections.

6.2. Double Deep Q-Network (Double DQN)

Double Deep Q Network (Double DQN) is an extension of the standard DQN algorithm designed to address the overestimation bias present in Q value estimation. In Classical DQN, the same neural network is used both to select and evaluate actions, which can lead to overly optimistic value estimates and suboptimal policies.

Problem Addressed

The main limitation of DQN lies in its tendency to overestimate Q values. This occurs because the max operator is applied on noisy value estimates generated by the same network. As a result, the agent may incorrectly favor certain actions, leading to unstable learning and degraded performance, especially in complex environments such as traffic signal control.

Core Idea

The key idea behind Double DQN is to decouple action selection from action evaluation. Instead of using a single network for both tasks, Double DQN uses two networks:

- the **online network** (main network) for selecting the best action
- the **target network** for evaluating that action

This separation reduces overestimation and leads to more accurate value estimation.

Mechanism

In Double DQN, the target Q value is computed using two steps:

1. The **online network** selects the best action for the next state
2. The **target network** evaluates the Q value of that selected action

This modification ensures that the evaluation is less biased, improving learning stability and convergence.

Application to Traffic Signal Control

In traffic signal control, Double DQN can be used to select the optimal signal phase based on current traffic conditions. By reducing overestimation, the agent avoids consistently favoring suboptimal signal phases, leading to more balanced traffic flow and improved overall intersection performance.

Supporting Study

Several studies have demonstrated the benefits of Double DQN in traffic signal control. For instance, the study “*Double Deep Q-Network with a Dual-Agent for Traffic Signal Control*” (Gu et al., 2020) proposes a Double DQN based approach combined with a dual agent architecture to improve traffic signal decision making at signalized intersections.[21]

In this work, two coordinated agents are employed to manage different aspects of the traffic signal phases, while the Double DQN mechanism is used to reduce the overestimation bias inherent in standard DQN. The model is evaluated using the SUMO traffic simulator, and the results show improvements in traffic capacity and reductions in congestion compared to conventional control strategies. These findings highlight the effectiveness of Double DQN in enhancing learning stability and overall traffic performance in dynamic environments.

6.3. Dueling Deep Q-Network (Dueling DQN)

Dueling Deep Q Network (Dueling DQN) is an advanced variant of DQN that improves learning efficiency by modifying the network architecture. Instead of directly estimating the Q value for each action, Dueling DQN decomposes it into two separate components: the state value and the advantage of each action.

Problem Addressed

In many decision making scenarios, including traffic signal control, the choice of action may have little impact in certain states. For example, when traffic is light, switching or keeping the current signal phase may result in similar outcomes. In such cases, standard DQN wastes learning capacity trying to estimate separate Q values for each action, even when their differences are negligible.

Core Idea

The key idea of Dueling DQN is to separate:

- **State Value** $V(s)$: how good it is to be in a given state
- **Advantage** $A(s,a)$: how beneficial a specific action is compared to others

By learning these two components independently, the model can better understand which states are important and which actions truly make a difference.

Mechanism

The neural network in Dueling DQN is divided into two streams:

1. A **value stream** that estimates $V(s)$
2. An **advantage stream** that estimates $A(s,a)$

These two outputs are then combined to produce the final Q value:

$$Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,a')))$$

This architecture allows the agent to generalize better across actions and focus learning on the most relevant decisions.

Application to Traffic Signal Control

In traffic signal control, Dueling DQN can identify whether a traffic state is inherently good or congested, regardless of the selected action. This helps the agent prioritize learning important traffic conditions while efficiently distinguishing between useful and less impactful signal phase decisions.

Supporting Study

Several recent studies have demonstrated the effectiveness of dueling based architectures in traffic signal control. For example, the study “*Carbon Dioxide Emission Reduction-Oriented Optimal Control of Traffic Signals in Mixed Traffic Flow Based on Deep Reinforcement Learning*” (2023) proposes a D3QN based approach that combines Double DQN and Dueling DQN to optimize traffic signal timing.

The model is designed not only to improve traffic efficiency but also to reduce fuel consumption and carbon emissions. Experimental results show that the proposed method outperforms conventional control strategies in terms of delay reduction and environmental impact, highlighting the ability of dueling based architectures to learn more informative state representations and make more efficient decisions in complex traffic environments [22].

7. Multi-Agent Reinforcement Learning

In large scale urban traffic networks, multiple traffic signals operate simultaneously and interact indirectly through shared traffic dynamics. When a signal at one intersection reduces congestion, the resulting improvement in traffic flow directly affects the arrival rates at downstream intersections. This interdependency motivates the use of multi agent reinforcement learning (MARL) frameworks, in which multiple agents interact within a common environment.

In standard MARL formulations, agents may rely on explicit communication or the exchange of neighboring state information to coordinate their decisions. However, such approaches increase system complexity and are often difficult to scale. An alternative is the **decentralized execution** strategy, in which each agent observes only its local state and selects actions independently, without direct communication with other agents. Coordination emerges implicitly through the shared traffic environment rather than through explicit message passing.

To improve learning efficiency and scalability in this setting, **parameter sharing** is commonly introduced. In this approach, all agents share a single neural network representing a common policy, while each agent interacts with a different part of the environment using its own local observations. This design significantly reduces the total number of trainable parameters and allows the shared model to generalize across different intersection contexts.

In addition, a **shared experience replay buffer** can be employed, allowing all agents to contribute their interaction experiences to a common memory. Sampling from this shared buffer exposes the network to a wider variety of traffic states, improving data efficiency and accelerating convergence.

This combination of decentralized execution, parameter sharing, and shared replay constitutes a scalable and computationally efficient framework for multi-intersection traffic signal control, and directly reflects the architecture adopted in this work.

Supporting Study

Several studies have explored MARL with parameter sharing for large scale traffic signal control. For example, the study "*CoLight: Learning Network-Level Cooperation for Traffic Signal Control*" (Wei et al., 2019) proposes a cooperative MARL framework in which multiple intersections are controlled simultaneously through a shared learning architecture. The approach models inter-intersection dependencies using a graph based representation, enabling agents to learn coordinated behaviors from traffic dynamics. Experimental results demonstrate improvements in traffic efficiency over both traditional and independent learning baselines, highlighting the importance of shared learning in multi-intersection environments [23].

8. Limitations of Existing Works

Although numerous traffic signal control methods have been proposed in the literature, several important limitations remain.

First, many studies evaluate their approaches on isolated intersections or limited traffic scenarios, which restricts the generalization of results to large scale urban traffic networks where multiple intersections interact dynamically.

Second, several methods rely on physical sensing infrastructure or complex system configurations, which may require significant installation cost, maintenance effort, and operational overhead. In addition, advanced deep learning based approaches often demand high computational resources, making real world deployment challenging.

Third, reinforcement learning and deep reinforcement learning approaches suffer from issues such as training instability, sensitivity to hyperparameter tuning, and low sample efficiency. These challenges can significantly affect the robustness and convergence of learned policies.

Furthermore, many existing studies focus on a single algorithm without conducting comprehensive comparisons between different variants, particularly among value based methods such as DQN, Double

DQN, and Dueling DQN. This lack of comparative analysis makes it difficult to determine which approach is most suitable for traffic signal control.

Finally, scalability remains a major challenge, especially in multi intersection environments, where interactions between traffic signals introduce non stationarity and increase the complexity of the learning process

Study	Method	Authors and Year	Strengths	Limitations	Ref
1	Vehicle Actuated Control	Syla et al., 2026	Low implementation complexity, responsive to real time traffic	Sensor dependency, limited adaptability	[8]
2	Vehicle Actuated Control	Ćiprovski et al., 2023	Improves delay and flow	Requires infrastructure, not scalable	[10]
3	Fuzzy Logic	Mahmood et al., 2019	Handles uncertainty, adaptive decisions	Manual rule design, limited generalization	[11]
4	Genetic Algorithm	Benkheris et al., 2013	Optimizes signal timing	High computational cost, slow convergence	[12]
5	Fuzzy + GA	Cao & Wang, 2016	Combines adaptability and optimization	Complex tuning, increased model complexity	[13]
6	DRL (Policy + Value)	Mousavi et al., 2017	Learns policies automatically	Tested on isolated intersections	[14]
7	RL Comparison Study	Louw et al., 2022	Compares multiple RL strategies	Limited focus on DQN variants	[19]
8	DQN	Ducrocq, 2021	Handles partial detection, open source implementation	Still affected by overestimation, limited large-scale evaluation	[20]
9	Double DQN	Gu et al., 2020	Reduces overestimation, improves stability	Evaluated on limited scenarios, increased model complexity	[21]
10	D3QN (Double + Dueling)	Wang et al., 2023	Improves efficiency and reduces emissions	Higher computational cost, requires careful tuning	[22]
11	MARL (Graph-based - CoLight)	Wei et al., 2019	Captures intersection , dependencies improves coordination between traffic signals	High complexity, requires modeling inter-agent interactions	[23]

Table3: Comparison of the methods used in each study

9. Research Gap

Despite the significant progress in traffic signal control using reinforcement learning and deep reinforcement learning, several research gaps remain.

First, although value based methods such as Deep Q Network (DQN) have been widely applied, most existing studies focus on a single algorithm without conducting a rigorous and systematic comparison between its advanced variants, particularly Double DQN and Dueling DQN. This limits the understanding of their relative performance, stability, and suitability for traffic signal control tasks.

Second, many studies evaluate their approaches under different experimental settings, including varying traffic scenarios, simulation environments, and evaluation metrics. This lack of standardized evaluation makes it difficult to draw fair and consistent conclusions regarding the effectiveness of different methods.

Third, most existing works focus on isolated intersections or simplified traffic environments, without fully addressing the complexity of multi-intersection systems where interactions between traffic signals significantly affect overall performance.

Finally, while some multi agent approaches have been proposed, limited attention has been given to scalable learning strategies such as parameter sharing, which can improve learning efficiency and generalization across multiple agents operating in a shared environment.

Therefore, there is a clear need for a comprehensive and controlled comparative study of DQN, Double DQN, and Dueling DQN under unified experimental conditions, with a focus on realistic traffic scenarios and scalable learning strategies.

Chapter 3: Methodology and System Design

1. Introduction

This chapter presents the methodology adopted in this thesis to design, implement, and evaluate adaptive traffic signal control systems based on deep reinforcement learning. Building on the theoretical foundations established in Chapter 1 and the review of existing approaches in Chapter 2, this chapter bridges the gap between concepts and experimentation by detailing the key design decisions made throughout the work.

The primary objective of this thesis is to conduct a rigorous and controlled comparative study of three value based deep reinforcement learning algorithms (Classical DQN, Double DQN, and Dueling DQN) applied to adaptive traffic signal control in a multi-intersection urban environment. As highlighted in the research gap identified in Chapter 2, most existing studies either evaluate a single algorithm in isolation or compare methods under different experimental conditions, making fair conclusions difficult to draw. To address this limitation, all three models in this work are trained and evaluated under a strictly unified experimental framework, sharing the same environment, hyperparameters, training protocol, and evaluation metrics.

The chapter is organized as follows. Section 2 provides a general overview of the proposed solution, outlining the decentralized multi agent architecture and the parameter sharing strategy. Section 3 describes the road network topology retained for this study. Section 4 formally defines the traffic signal control problem as a Markov Decision Process, specifying the state representation, action space, and reward function adopted in this work. Section 5 presents the multi agent architecture and the parameter sharing strategy employed to handle multiple intersections efficiently. Section 6 details the three DQN variants and their architectural differences. Section 7 defines the eight performance metrics used to evaluate all controllers. Section 8 describes the traffic scenarios used for training and evaluation. Finally, Section 9 summarizes the key design decisions established in this chapter and outlines how they are translated into a concrete implementation in Chapter 4.

2. General Overview of the Proposed Solution

The solution proposed in this thesis addresses the traffic signal control problem through a multi agent deep reinforcement learning framework in which each signalized intersection in the network is treated as an independent decision making agent. Rather than designing a centralized controller that observes the entire network state and issues coordinated commands to all intersections simultaneously an approach that scales poorly with network size and is difficult to deploy in practice the proposed

framework adopts a decentralized execution strategy in which each agent observes only the local traffic conditions at its own intersection and selects phase switching decisions autonomously, without any direct communication with the other agents.

Coordination between intersections emerges implicitly through the shared traffic environment: when one agent successfully reduces congestion at its intersection, the resulting improvement in vehicle flow propagates to adjacent intersections through the road network, where it is observed by neighboring agents through their own local state vectors and incorporated into their subsequent decisions. In this way, global traffic efficiency is improved through the aggregated effect of locally rational decisions, without requiring any form of centralized coordination.

To make this framework computationally tractable and statistically efficient, all agents share a single neural network through a **parameter sharing** strategy. Rather than maintaining one independent network per intersection, a single policy network and a single target network are used across the entire system. This design exploits the structural homogeneity of the problem: because all intersections share the same state representation format, the same binary action space, and the same reward function definition, a single shared network can learn a general policy applicable across all of them. The benefits are threefold: reduced trainable parameters, multiplied data collection rate, and more generalizable policies less prone to overfitting.

The shared experience buffer is implemented as a **Prioritized Experience Replay (PER)** buffer. Under uniform sampling, critical transitions (those involving sudden congestion onset, poorly timed phase switches, or effective interventions) are rare and would be selected with the same low probability as routine transitions. PER corrects this by assigning each transition a priority proportional to its TD error magnitude, ensuring the network focuses its training capacity on the most informative experiences. An importance sampling correction with a linearly annealed exponent compensates for the statistical bias introduced by non-uniform sampling.

Three distinct instantiations are implemented and compared, each corresponding to a different DQN architecture: **Classical DQN**, **Double DQN**, and **Dueling DQN**. All three share identical hyperparameters, training scenarios, evaluation protocols, and metric computation infrastructure, ensuring that any observed performance difference is attributable exclusively to their architectural distinctions.

3. Description of the Retained Network

3.1. General Topology

The road network consists of a multi-intersection urban grid containing **four signalized intersections**, reflecting a realistic urban topology extracted from an actual street layout. It comprises bidirectional roads where each road segment consists of a single lane in each direction, with turning movements permitted at each intersection. Every road segment has a corresponding opposite direction edge, forming a symmetric bidirectional structure throughout the grid.

The presence of four independent control agents allows the study of implicit interactions between neighboring intersections in a realistic multi agent setting. The symmetric structure (with an identical number of incoming lanes at all intersections) enables parameter sharing: the shared neural network receives state vectors of uniform dimension from all agents without requiring padding or variable length input handling.

3.2. State Vector Dimension

Since all road segments consist of a single lane per direction, the state vector dimension for each agent is fixed at initialization according to:

$$\text{STATE_DIM} = 2 + 4 \times \text{N_lanes}$$

where N_lanes is the number of incoming lanes at that intersection. The symmetric network design ensures all four intersections produce state vectors of identical length, which considerably simplifies the parameter sharing architecture.

3.3. Justification for the Comparative Study

The retained network satisfies criteria that make it suitable for this comparative study. Its topology ensures practical relevance. Four interdependent intersections allow evaluation of congestion propagation management across the network. The structural symmetry enables fair comparison of the three DQN variants under strictly identical training conditions.

4. Problem Formulation as a Markov Decision Process

4.1. MDP Definition

Traffic signal control at a multi-intersection network is a sequential decision making problem in which a controller must repeatedly choose signal phase actions over time in order to optimize traffic

flow. This type of problem is formally captured by the Markov Decision Process (MDP) framework, which was introduced in Chapter 1 and is here instantiated in the specific context of this work.

An MDP is defined by the tuple (S, A, P, R, γ) , where S is the state space, A is the action space, P is the transition probability function, R is the reward function, and γ is the discount factor. In the multi-intersection setting adopted in this thesis, the network contains multiple signalized intersections, each controlled by an independent agent. All agents share the same MDP structure, meaning that the definitions of state, action, and reward apply uniformly to every intersection in the network. This design choice is consistent with the parameter sharing strategy described in Section 5.2, and ensures that the learned policy generalizes across different intersection contexts rather than being tied to a specific location.

The transition function $P(s' | s, a)$ captures the stochastic dynamics of the traffic environment. Because vehicle arrivals, speeds, and routing decisions are inherently random, the exact next state cannot be predicted deterministically from the current state and action alone. The SUMO simulator serves as an implicit model of these dynamics, generating state transitions through microscopic vehicle level simulation. Rather than estimating P explicitly, the agents interact with the simulator directly and learn from observed transitions, which is the standard model free reinforcement learning approach.

The discount factor γ is set to 0.97 in this work, reflecting a strong but not exclusive preference for long term traffic performance. A value close to 1 encourages the agent to consider the downstream consequences of its phase decisions for instance, the fact that holding a green phase too long at one intersection may relieve immediate pressure but cause congestion to build at adjacent intersections. This temporal horizon is particularly important in a multi intersection environment where local decisions propagate through the network over time.

4.2. State Representation

The state observed by each agent at every decision step is a fixed length vector that encodes the current traffic conditions at its associated intersection. Formally, for a given traffic light controller tl with N incoming lanes, the state vector is defined as:

$$s = [\text{phase_norm}, \text{tip_norm}, q_1, \dots, q_n, w_1, \dots, w_n, d_1, \dots, d_n, v_1, \dots, v_n]$$

where the vector has dimensionality $2 + 4N$, and each component is described below.

Phase index (phase_norm). The current signal phase index, normalized by dividing by 10, yielding a value in $[0, 1]$. This encodes which traffic movement currently has the right of way, allowing the agent to distinguish between the two green phases and make temporally coherent decisions. division by 10 rather than by the maximum phase index ensures consistent normalization regardless of the specific phase configuration, accommodating potential future extensions to the signal program

Time in phase (tip_norm). The elapsed time since the last phase switch, clipped to the maximum green duration $\text{MAX_GREEN} = 60$ seconds and normalized to $[0, 1]$. This component provides the agent with an implicit sense of how long the current phase has been active, which is essential for learning when to switch and when to hold.

Queue length per lane (q_i). The number of halting vehicles on each incoming lane, divided by 20 as a normalization constant. Halting vehicles are those whose speed falls below a threshold defined by SUMO. Queue length is one of the most direct indicators of congestion and is therefore a core component of the state.

Waiting time per lane (w_i). The cumulative waiting time of vehicles on each incoming lane, normalized by dividing by 100. This component captures not only how many vehicles are queued but also how long they have been waiting, penalizing persistent congestion that queue length alone may not fully reflect.

Vehicle density per lane (d_i). The number of vehicles present on each incoming lane divided by the lane length in meters. This captures the spatial occupancy of each lane and complements queue length by accounting for moving vehicles as well as stationary ones.

Speed ratio per lane (v_i). The mean speed of vehicles on each incoming lane, normalized by the maximum allowed speed of that lane. This yields a value in $[0, 1]$, where 1 indicates free flow conditions and 0 indicates complete stoppage. Speed ratio is a direct measure of traffic fluidity and provides the agent with information about the quality of current flow rather than just its quantity.

The choice of this state representation is motivated by the need to provide each agent with a comprehensive yet compact description of its local intersection. All components are normalized to comparable ranges, which stabilizes neural network training and prevents any single feature from dominating gradient updates. Importantly, only locally observable quantities are included, consistent

with the decentralized execution strategy described in Section 4, where each agent operates without access to information from other intersections.

4.3. Action Space

The action space for each agent is binary, consisting of two possible decisions at each decision step:

- **Action 0 (Keep):** maintain the current signal phase without any change.
- **Action 1 (Switch):** transition the current green phase to the subsequent yellow phase, initiating a phase change.

This two action formulation is intentionally simple. Rather than requiring the agent to select a specific target phase directly, the switch action triggers a natural phase progression that follows the phase sequence defined in the network configuration:

green $\rightarrow$ yellow $\rightarrow$ green $\rightarrow$ yellow

cycling between the two green phases. The yellow phase duration is fixed at 3 seconds as defined in the network file and is not subject to agent control, reflecting the physical constraint that yellow intervals serve as safety clearance periods.

A critical constraint imposed on the switch action is the minimum green time, **MIN_GREEN = 10** seconds. An agent may only execute a switch if the time elapsed since the last phase change is at least 10 seconds. If the agent selects action 1 but this condition is not met, the action is silently overridden and treated as a keep. This constraint reflects a real world physical requirement: switching a signal too frequently causes driver confusion, disrupts traffic flow, and is unsafe. Rather than penalizing the agent heavily for attempting early switches, the **MIN_GREEN** constraint enforces this boundary at the environment level, allowing the reward function to focus on higher level traffic optimization objectives.

It is worth noting that the effective action stored in the replay buffer is the action that was actually executed either 0 if the agent kept the phase or if the switch was blocked by **MIN_GREEN**, or 1 if the switch was successfully applied. This design prevents ambiguity in the Q value targets: the agent learns the value of what actually happened in the environment rather than what it intended to do.

4.4. Reward Function

The reward function is the primary mechanism through which the agent receives feedback about the quality of its decisions. Its design is one of the most consequential choices in any reinforcement learning application, as it directly shapes the behavior that the agent will learn to produce. In this work, the reward for each agent at each decision step is computed as follows:

$$r = -0.3 \cdot q_norm - 0.1 \cdot w_norm + 0.6 \cdot spd_norm - 0.6 \cdot pressure - 0.05 \cdot switched$$

Each term is described in detail below.

Queue penalty ($-0.3 \cdot q_norm$). The normalized queue term is computed as $q_norm = Q / (Q + 10)$, where Q is the total number of halting vehicles across all incoming lanes. This soft cap normalization is linear for small queues and saturates gradually for large ones, preventing extreme queue values from dominating the reward signal. **A weight of 0.3 reflects the importance of reducing stopped vehicles without making queue reduction the sole objective.**

Waiting time penalty ($-0.1 \cdot w_norm$). The normalized waiting term is computed as $w_norm = W / (W + 50)$, where W is the total cumulative waiting time across all incoming lanes. This term penalizes persistent congestion that persists over time. **Its weight is deliberately kept small at 0.1.** An earlier version of the reward function included a **delta_w** term, the change in waiting time between consecutive steps, but this was removed because it introduced a systematic bias toward always switching phases. Immediately after a phase switch, waiting time briefly drops as new vehicles enter the green phase, producing an artificial positive reward that encouraged premature and excessive switching. The static w_norm term avoids this bias because it changes slowly across a single decision step and does not create an immediate incentive to switch.

Speed reward ($+0.6 \cdot spd_norm$). The normalized speed term is the mean of the speed ratios across all incoming lanes, where each ratio is the lane mean speed divided by the lane maximum allowed speed. This term directly rewards traffic fluidity, providing a positive signal when vehicles are moving freely. **Its weight of 0.6 is the largest in the reward function**, reflecting the design philosophy that throughput and flow efficiency are the primary objectives of adaptive signal control. A high speed ratio indicates that the current phase allocation is serving traffic demand effectively.

Pressure penalty ($-0.6 \cdot pressure$). The pressure term is defined as :

$$\tanh((in_q - out_q) / (N_in + N_out))$$

where **in_q** is the total queue on incoming lanes and **out_q** is the total queue on outgoing lanes. This captures the imbalance between traffic arriving at the intersection and traffic departing from it. A high pressure value indicates that vehicles are accumulating faster than they are being discharged, which is a leading indicator of congestion spillback. **The tanh function bounds the pressure to $(-1, 1)$, preventing extreme values from destabilizing training. Its weight of 0.6 matches that of the speed reward, reflecting the equal importance of load balancing across the network.** It is worth noting that computing outgoing lane queues required special handling in this work: the standard SUMO function for retrieving outgoing lanes fails for cluster junctions, which are the type of junction used in the network topology.

This was resolved by extracting outgoing lanes from **getControlledLinks**, which returns the destination lane for each signal controlled movement and remains valid for cluster junctions.

Switch penalty $(-0.05 \cdot \text{switched})$. A small penalty is applied whenever the agent successfully executes a phase switch. This discourages unnecessary switching when traffic conditions do not justify a phase change. The penalty is kept intentionally small at **0.05** because the **MIN_GREEN** constraint already prevents the most disruptive form of over switching. The reward function is finally clipped to the range $[-50, 10]$ to prevent extreme values from producing destabilizing gradient updates during training.

5. Multi-Agent Architecture and Parameter Sharing

5.1. Decentralized Execution

The traffic network used in this work contains four signalized intersections, each requiring independent real time control decisions. A natural approach to this problem is multi agent reinforcement learning, in which each intersection is assigned a dedicated agent that observes its local environment and selects actions autonomously. This is the architecture adopted in this thesis.

Each agent is associated with a single traffic light controller identified by its SUMO ID. At every decision step, the agent observes the state vector of its own intersection constructed from the incoming lanes under its control as described in Section 4.2 and selects either a keep or switch action based on its current policy. Critically, no agent has access to the observations or actions of any other agent. There is no explicit communication channel between agents, and no global state is ever constructed or used during either training or evaluation.

This design follows the decentralized execution paradigm, in which coordination between agents emerges implicitly through the shared traffic environment rather than through direct information exchange. When one agent decides to extend a green phase and reduce the queue at its intersection, the resulting improvement in vehicle flow propagates downstream and affects the arrival rates at adjacent intersections. The neighboring agents then observe these changes through their own local state vectors and adapt their decisions accordingly. In this way, global traffic efficiency is improved through the aggregated effect of locally rational decisions, without requiring any form of centralized coordination or inter agent messaging.

The decentralized execution strategy offers several practical advantages that are particularly relevant in the context of traffic signal control. First, it scales naturally with the number of intersections: adding a new intersection to the network requires no modification to the control architecture beyond assigning it an agent with the same state and action interface. Second, it is robust to communication failures, since agents do not depend on receiving information from other agents to function correctly. Third, it closely mirrors real world deployment conditions, where traffic controllers at different intersections typically operate as independent units without reliable high bandwidth communication links between them.

5.2. Parameter Sharing Strategy

Although each agent operates independently during execution, all four agents in this work share a single neural network during both training and evaluation. Rather than maintaining four separate networks one per intersection a single policy network is used, and all agents query this shared network to compute their Q values and select actions. Similarly, a single target network is maintained for computing stable Q value targets during training.

This approach is known as parameter sharing, and its adoption in this work is motivated by both theoretical and practical considerations. From a theoretical perspective, parameter sharing is justified by the structural homogeneity of the agents: all intersections share the same state representation format, the same action space, and the same reward definition. Although the specific traffic conditions at each intersection differ, the underlying decision making problem has the same structure everywhere in the network. A shared network can therefore exploit this structural similarity and learn a general policy that applies across all intersections simultaneously.

From a practical perspective, parameter sharing offers three concrete benefits in the context of this work. First, it reduces the total number of trainable parameters by a factor of four compared to

maintaining separate networks. This significantly lowers the computational cost of both forward passes during action selection and backward passes during gradient updates, which is important given the large number of simulation steps required for training. Second, it accelerates convergence: because all agents contribute experience to the same network, the effective data throughput per gradient update is multiplied by the number of agents. The network is exposed to a much wider variety of traffic states per training step than would be possible with a single agent setup, leading to faster and more stable learning. Third, parameter sharing improves generalization: the shared network must learn representations that are useful across multiple intersection contexts, which discourages overfitting to the idiosyncrasies of any single location and produces more robust policies.

During action selection, each agent passes its own local state vector through the shared policy network and independently selects the action with the highest Q value output. The network therefore processes one state vector per agent per decision step, and each agent receives its own action independently. During training, gradients are computed from batches of transitions drawn from the shared experience replay buffer described in the following section, and the shared network parameters are updated once per decision step regardless of which agent contributed the sampled transitions.

5.3. Shared Experience Replay

In standard single agent DQN, a replay buffer stores transitions collected by a single agent interacting with a single environment. In the multi agent parameter sharing framework adopted in this work, all four agents contribute their transitions to a single shared replay buffer. Every time a decision step is completed, each of the four agents appends its own transition tuple ($s, a, r, s', done$) to the shared buffer, where the subscript i identifies the agent. This means that every decision step produces four new transitions in the buffer rather than one, resulting in a data collection rate that is four times higher than in a single-agent setup.

The shared replay buffer provides two important advantages beyond the increased data collection rate. First, it exposes the shared network during training to a diverse mixture of traffic states drawn from all four intersections simultaneously. Each minibatch sampled from the buffer contains transitions from different intersections, different phases of the traffic cycle, and different demand conditions depending on which training scenario was active at the time the transition was collected. This diversity acts as a form of data augmentation that improves the generalizability of the learned policy and reduces the risk of the network becoming overly specialized to the conditions of any single intersection. Second, the shared buffer smooths out the nonstationarity that is inherent in multi agent environments. Because all

agents are learning simultaneously and the behavior of each agent affects the traffic conditions observed by the others, the effective environment seen by any single agent is constantly changing. By pooling transitions from all agents and sampling them randomly during training, the shared buffer decorrelates the training data and reduces the instability that nonstationarity would otherwise introduce.

6. DQN Architectures

6.1. Classical DQN Architecture

The first model evaluated in this work is the Classical Deep Q Network, which serves as the baseline among the three learned controllers. As described in Chapter 2, DQN extends tabular Q learning by replacing the Q value table with a deep neural network that approximates the action value function $Q(s, a)$ for all possible actions simultaneously given a state input. This approximation enables the agent to generalize across the large and continuous state space that characterizes real-world traffic environments.

In this work, the policy network for the Classical DQN is implemented as a fully connected feedforward neural network with the following architecture:

Input $\rightarrow$ Linear(state_dim, 256) $\rightarrow$ ReLU $\rightarrow$ Linear(256, 256) $\rightarrow$ ReLU $\rightarrow$ Linear(256, 2) $\rightarrow$ Output

The following figure represents the Classical DQN policy network structure :

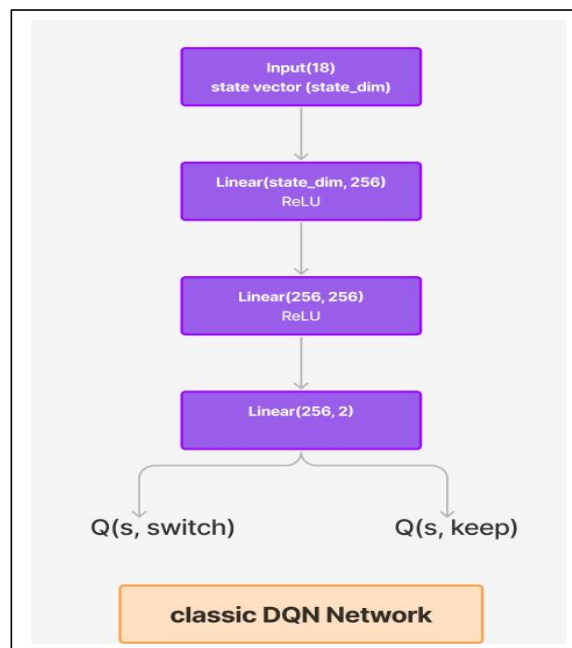


Figure03: *The Classical DQN policy network structure*

The output layer produces two Q values, one per action. Two copies are maintained : the **policy network**, updated at every training step through gradient descent, and the **target network**, a periodically synchronized copy used exclusively for computing stable Q value targets. The target is computed as:

$$y = r + \gamma \cdot \max_{\{a'\}} Q_target(s', a') \cdot (1 - done)$$

The Huber loss function is used rather than mean squared error, as it is less sensitive to outliers in the TD error distribution. The target network is synchronized with the policy network every 1000 training steps through a hard update.

Classical DQN suffers from a systematic **overestimation bias**: because the same network is used both to select and evaluate the best action in the next state, the max operator tends to select actions whose Q values are overestimated due to noise, feeding those overestimated values back as training targets.

6.2. Double DQN

Double DQN addresses the overestimation bias by **decoupling action selection from action evaluation** in the target computation. In Classical DQN:

$$y = r + \gamma \cdot Q_target(s', \operatorname{argmax}_{\{a'\}} Q_target(s', a'))$$

Both selection and evaluation are performed by the target network. In Double DQN:

$$y = r + \gamma \cdot Q_target(s', \operatorname{argmax}_{\{a'\}} Q_policy(s', a'))$$

The policy network identifies the best action in the next state, while the target network evaluates its value. Because the two networks have different parameters, the action that appears best according to one is evaluated by the other, which is less likely to assign it an inflated value. This breaks the self reinforcing overestimation cycle.

The following figure represents the difference between Classical DQN and double DQN architecture :

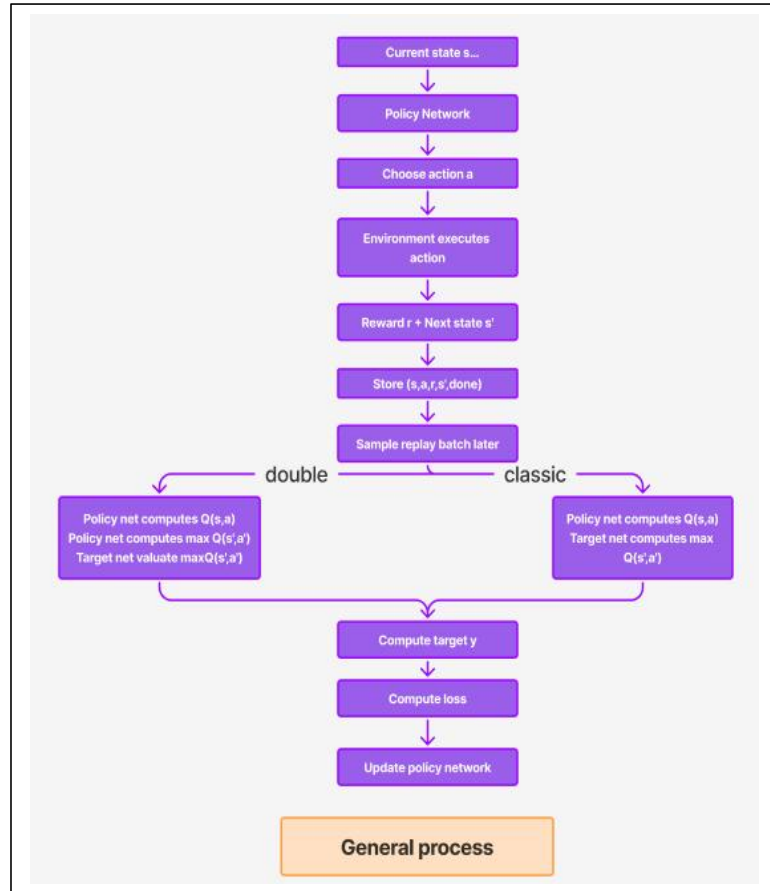


Figure04: The difference between Classical DQN and double DQN architecture

In the traffic signal control context, overestimation can cause the agent to consistently overvalue certain signal phases, leading to unbalanced phase allocation that does not reflect true traffic demand. In theory, the Double DQN system is expected to encourage more balanced and demand responsive switching decisions. This will be discussed in the results (chapter 5)

Every other component (network architecture, replay buffer, loss function, optimizer, target update schedule, and all hyperparameters) remains identical to Classical DQN, ensuring that any observed performance difference is attributable exclusively to the correction of the overestimation bias.

6.3. Dueling DQN

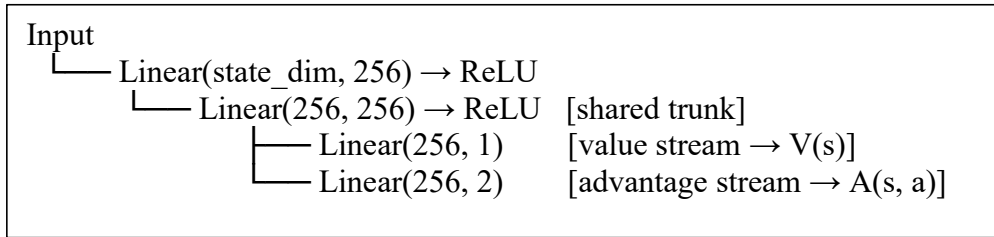
Dueling DQN introduces a fundamentally different approach by modifying the **internal architecture** rather than the target computation. Instead of directly outputting Q values, the network decomposes $Q(s, a)$ into two components:

- **V(s)**: the state value function = how beneficial it is to be in state s regardless of which action is taken
- **A(s, a)**: the advantage function = how much better action a is compared to the average action in state s

These are recombined using:

$$Q(s, a) = V(s) + (A(s, a) - \text{mean}_{\{a'\}} A(s, a'))$$

The subtraction of the mean advantage ensures a unique decomposition, forcing V(s) to reflect the true average value of the state. The architecture consists of a shared trunk followed by two parallel streams:



The following figure represents the dueling DQN policy network structure :

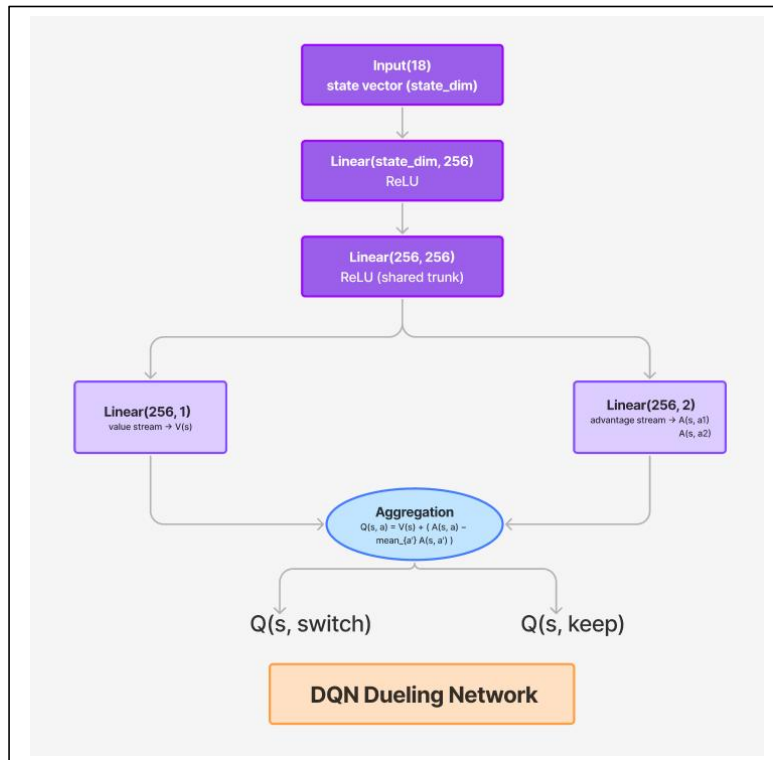


Figure05: The dueling DQN policy network structure.

In many traffic states (especially under low or moderate demand) the choice between keeping and switching may have little impact. What matters most is recognizing that the overall situation is benign, which corresponds to accurately estimating $V(s)$. Classical DQN wastes representational capacity estimating separate Q values for each action in every state, including those where action differences are negligible. Dueling DQN separates these concerns: the value stream learns the overall quality of traffic states, while the advantage stream focuses on when one action is meaningfully better than the other, leading to more data efficient learning.

6.4. Comparative Summary

Component	Classical DQN	Double DQN	Dueling DQN
Network architecture	3 fully connected layers	3 fully connected layers	Shared trunk + value and advantage streams
Q-value decomposition	Direct $Q(s,a)$ output	Direct $Q(s,a)$ output	$Q(s,a) = V(s) + A(s,a) - \text{mean}(A)$
Action selection (target)	Target network	Policy network	Target network
Action evaluation (target)	Target network	Target network	Target network
Overestimation correction	None	Decoupled selection and evaluation	Improves representation, not overestimation directly
Primary strength	Simplicity and stability	Reduced overestimation bias	More efficient state value learning
Primary limitation	Overestimation of Q values	Increased implementation complexity	Higher architectural complexity
Training pipeline changes	Baseline	Target computation only	Network architecture only

Table4: Comparison of the methods

7. Evaluation Metrics

Eight performance metrics are collected and analyzed across all evaluation episodes using a shared measurement infrastructure, ensuring strictly identical measurement conditions across all models.

Average Waiting Time measures the mean time, in seconds per vehicle, that vehicles spend stationary or moving at negligible speed while waiting for a green signal. It is one of the most direct indicators of driver discomfort and intersection inefficiency. A lower value indicates that the controller succeeds in reducing unnecessary stops and serving vehicles more promptly.

Average Queue Length represents the mean number of vehicles accumulated on incoming lanes at any given simulation step. Excessive queue lengths are symptomatic of supply demand imbalance at the intersection and can propagate upstream to affect the broader network. A well performing controller should keep queue lengths low and stable even under high traffic demand.

Average Delay captures the additional travel time experienced by each vehicle compared to its free flow travel time, expressed in seconds per vehicle. Unlike waiting time, delay also accounts for vehicles moving below their desired speed, making it a more complete measure of the performance degradation induced by congestion.

Throughput counts the total number of vehicles that successfully complete their journey through the network during a full evaluation episode. It reflects the capacity utilization of the intersection from a supply perspective. A higher throughput indicates that the controller manages to process more vehicles within the same simulation duration, which is particularly relevant under high demand conditions.

Average Speed reports the mean speed of all vehicles present in the network, expressed in meters per second. It serves as a proxy for overall traffic flow quality: a higher average speed indicates smoother and less interrupted vehicle progression, while a low value reflects frequent stops and congestion.

Total CO₂ Emissions aggregates the carbon dioxide produced by all vehicles throughout the episode, expressed in milligrams. Since emissions are directly correlated with idling and stop and go behavior, this metric captures the environmental cost of traffic signal inefficiency. Reducing CO₂ emissions is increasingly recognized as an important objective in urban traffic management alongside traditional efficiency metrics.

Total Fuel Consumption measures the cumulative fuel used by all vehicles during the episode, expressed in milliliters. Like CO₂ emissions, fuel consumption increases significantly during periods of idling and repeated acceleration, making it sensitive to the frequency and duration of vehicle stops induced by signal control decisions.

Stopped Vehicle Ratio expresses the proportion of vehicles that are completely halted at any given simulation step, as a fraction of the total number of vehicles present in the network. It provides a snapshot of how much of the traffic is immobilized at each moment and complements the queue length metric by capturing the network wide extent of stopping behavior rather than focusing solely on individual lanes.

These eight metrics cover three dimensions: **efficiency** (waiting time, queue length, delay, throughput, speed), **environmental impact** (CO₂, fuel), and **congestion severity** (stopped ratio).

8. Comparison Scenarios

Four traffic scenarios are defined, differing in vehicle demand level.

Scenario	Demand (veh/h)	Purpose
Low	300	Training
Medium	800	Training
Mixed	301–1799	Training
High	1800	Evaluation only

Table5: *The difference between the four scenarios*

The **Low** scenario represents light traffic conditions where congestion rarely develops, allowing agents to learn baseline behavior. The **Medium** scenario introduces more complex temporal dynamics requiring balance between competing demands. The **Mixed** scenario exposes agents to a broad and unpredictable range of intensities within a single episode, forcing the development of robust adaptive policies and preventing overfitting to a fixed demand level.

The **High** scenario, reserved exclusively for evaluation, represents heavily congested conditions pushing the network close to saturation. Because agents have never encountered this demand level during training, their performance on this scenario provides a direct measure of out of distribution generalization the most practically relevant test of an adaptive controller.

During training, the three scenarios are cycled in a rotating order (low → medium → mixed → low → ...), ensuring balanced exposure to all demand levels and preventing bias toward any single scenario.

10. Chapter Summary

This chapter has established the complete design of the comparative framework proposed in this thesis. The traffic signal control problem was formalized as a Markov Decision Process with a carefully designed state representation, a binary action space, and a multi objective reward function balancing throughput, queue reduction, load balancing, and switching cost. The decentralized multi agent architecture was presented, along with the parameter sharing strategy that makes it computationally tractable and statistically efficient. The three DQN variants (Classical, Double, and Dueling) were described and their architectural distinctions clearly delineated, ensuring that any observed performance

difference can be attributed exclusively to their specific algorithmic innovations. Prioritized Experience Replay was introduced as a shared mechanism that focuses training capacity on the most informative experiences. Finally, the evaluation metrics and comparison scenarios were defined, establishing a clear and objective basis for assessing the performance of each controller.

All design decisions documented in this chapter are motivated by the dual requirement of realism and fairness: the framework must reflect practical urban traffic conditions while ensuring that the three variants are compared under strictly identical conditions. The road network, the MDP formulation, the shared training infrastructure, and the evaluation protocol together constitute a controlled experimental environment in which differences in results can be meaningfully interpreted.

The framework designed here now requires concrete implementation. The next chapter details how each component described in this chapter was realized in practice from the construction of the simulated network in SUMO, to the coding of the three DQN variants in Python, to the execution of the training and evaluation experiments, translating every design decision into a functioning experimental pipeline.

Chapter 4: Implementation and Experimentation

1. Introduction

This chapter addresses the complementary question of how the design decisions established in Chapter 3 were translated into a functioning experimental system. Chapter 3 defined the problem formulation, the multi agent architecture, the three DQN variants, the shared training mechanisms, and the evaluation protocol at the conceptual and architectural level, specifying what the system does and why each design choice was made. The present chapter now documents how each of these elements was concretely realized in practice from the construction of the simulated network in SUMO, to the coding of the three DQN variants in Python, to the execution of the training and evaluation experiments translating every design decision into a functioning experimental pipeline.

The chapter is organized as follows. Section 2 describes the hardware and software environment in which all experiments were conducted. Section 3 presents the SUMO microscopic traffic simulator and its integration with the Python based learning framework through the TraCI interface. Section 4 details the construction of the simulated road network, including the technical handling of cluster junctions. Section 5 describes the traffic generation procedure and the scenario curriculum used during training. Section 6 presents the concrete implementation of the three DQN variants and their shared components. Section 7 documents the implementation of the Prioritized Experience Replay mechanism, including the TD error priority computation, the importance sampling correction, and the buffer management details. Section 8 presents the complete training setup and hyperparameter configuration, covering the general training procedure, the epsilon decay strategy, and the checkpoint mechanism. Finally, Section 9 details the evaluation protocol, including the greedy policy evaluation procedure, the fixed time baseline implementation, and the statistical analysis framework. Section 10 Reproducibility and Fairness of Comparison . Section 10 summary.

2. Hardware and Software Environment

All experiments are conducted on a standard computing environment. The software stack is built on **Python** as the primary programming language, **PyTorch** as the deep learning framework for neural network construction, training, and inference, and **SUMO (Simulation of Urban MObility)** as the microscopic traffic simulator. The interface between the Python based learning framework and the SUMO simulator is provided by **TraCI (Traffic Control Interface)**, a socket based API that allows an external program to observe and manipulate the simulation state in real time. This tight coupling forms the foundation of the training and evaluation pipeline.

Python version	Python 3.12.3
SUMO version	Eclipse SUMO 1.19.0
TraCI	1.26.0
PyTorch	2.1.0+cu130

Table6: The versions of software tools

3. The SUMO Simulator

SUMO is an open source, microscopic, and continuous road traffic simulation package developed by the German Aerospace Center (DLR). At the microscopic level, SUMO simulates each vehicle individually, modeling its position, speed, acceleration, and lane changing behavior at every simulation step, set to one second by default. Vehicle movements are governed by the **Intelligent Driver Model (IDM)** for longitudinal control and a lane changing model for lateral behavior, producing realistic traffic dynamics that emerge from individual vehicle interactions rather than aggregate flow equations.

Through TraCI, the learning agents retrieve traffic measurements from the simulation (queue lengths, waiting times, vehicle speeds, and signal phases) and apply control actions to the traffic lights at each decision step. Each simulation episode begins by launching a new SUMO process via TraCI and terminates either when the maximum number of simulation steps is reached or when the episode ends. At the start of each episode, the simulation is reset to its initial state with a fresh vehicle demand generated according to the scenario configuration.

4. Construction of the Simulated Network

The road network was constructed using **NETEDIT**, SUMO's network editor. All four signalized intersections are implemented as cluster junctions, each assigned a single traffic light controller with a unique ID.

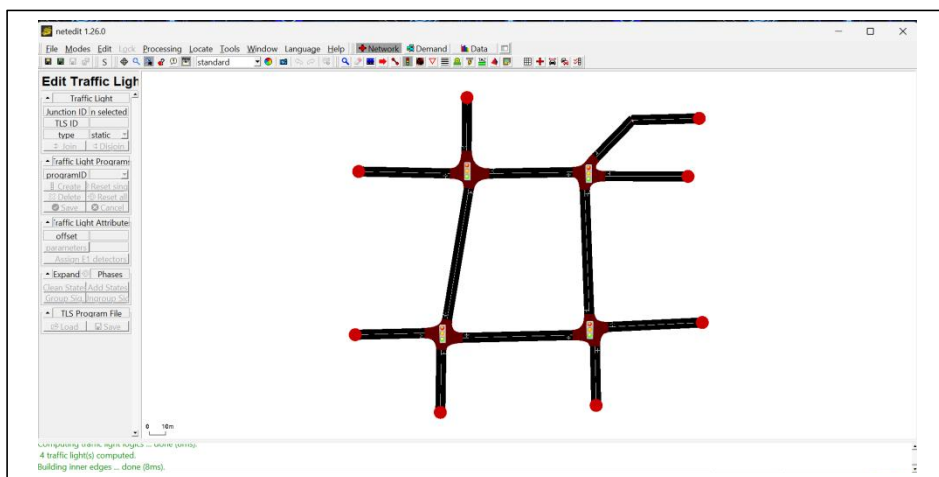


Figure06: SUMO's (NETEDIT) network editor

The state vector dimension for each agent is determined at initialization by querying the SUMO simulator for the controlled lanes of each traffic light controller:

$$\text{STATE_DIM} = 2 + 4 \times \text{N_lanes}$$

Because the network was designed with consistent road geometry in which all four intersections control the same number of incoming lanes, all agents produce state vectors of identical length. No padding, masking, or variable length input handling is required.

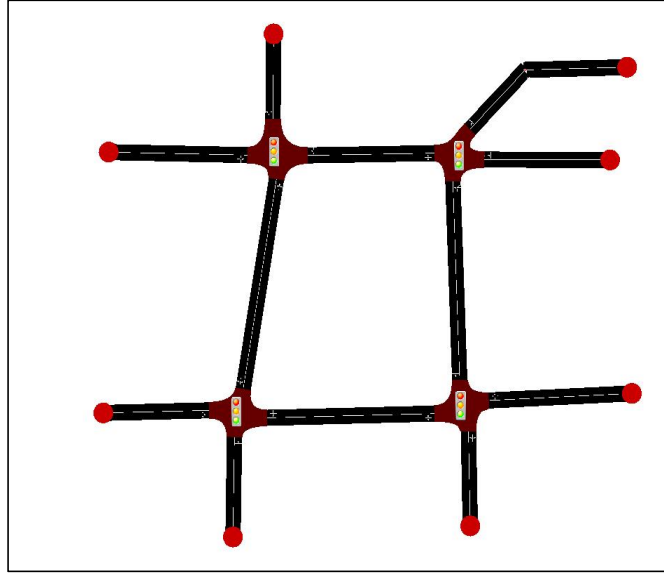


Figure07: *Simulated Road Network Constructed in SUMO for the Experimental Study*

4.1. Cluster Junctions

All four signalized intersections are implemented as **cluster junctions** compound nodes that merge several closely spaced physical junctions into a single logical traffic light controller. This representation is common in real urban networks where intersections are geometrically complex. Each intersection is assigned a single traffic light controller identified by a unique ID.

The use of cluster junctions introduced a technical challenge during state and reward computation. The standard SUMO function for retrieving outgoing lanes of a junction does not operate correctly on cluster junctions, consistently returning an empty set. This was resolved by extracting outgoing lanes using the **getControlledLinks** function, which returns the complete list of signal controlled movements at each intersection as (fromLane, toLane, viaLane) tuples. The destination lane (toLane) of each movement, excluding internal lanes whose identifiers begin with a colon character, is collected to form the outgoing lane set for that intersection.

4.2. Traffic Generation

Vehicle demand is generated according to four scenarios defined in SUMO configuration files. During training, the three training scenarios are cycled in a rotating order (low, medium, mixed, low, medium, mixed) with each episode using the next scenario in the sequence. This curriculum style rotation ensures balanced exposure to all demand levels.

Each training episode runs for a maximum of **MAX_STEPS = 3,600 simulation steps**, corresponding to 36,000 seconds of simulated traffic. The evaluation episode length is extended to **MAX_STEPS = 9,000 steps** (90,000 seconds) to provide a more comprehensive assessment of sustained performance under prolonged high demand conditions.

5. Implementation of the Three Methods

5.1. Shared Components

All three variants are implemented in Python using PyTorch and share the following components:

- **Optimizer:** Adam with learning rate $lr = 1 \times 10^{-4}$
- **Loss function:** Huber loss (SmoothL1Loss)
- **Gradient clipping:** maximum norm of 5.0
- **Target network update:** hard synchronization every **1,000 training steps**
- **Shared replay buffer:** Prioritized Experience Replay with capacity **100,000 transitions**

5.2. Classical DQN

Implemented as a three layer fully connected network. The target Q value for a transition is:

$$y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') \cdot (1 - \text{done})$$

Python:

```
class DQN(nn.Module):  
  
    def __init__(self, state_dim, action_dim):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(state_dim, 256), nn.ReLU(),  
            nn.Linear(256, 256), nn.ReLU(),  
            nn.Linear(256, action_dim)  
        )  
    def forward(self, x):
```

```
return self.net(x)
```

In `train_dqn(beta)` function:

```
next_q = target_net(next_states).max(1, keepdim=True)[0]
```

5.3. Double DQN

Identical to Classical DQN except for the target computation. Action selection uses the policy network; action evaluation uses the target network:

Python:

The same class of DQN but In `train_dqn(beta)` function:

```
next_actions = policy_net(next_states).argmax(1, keepdim=True)
# action selection
next_q = target_net(next_states).gather(1, next_actions)
# action evaluation
```

5.4. Dueling DQN

The network architecture replaces the direct Q value output with a shared trunk followed by two parallel streams. The aggregation formula:

$$Q(s, a) = V(s) + (A(s, a) - \text{mean}_{\{a'\}} A(s, a'))$$

is implemented in the network's forward pass. All other training components remain identical to Classical DQN.

```
class DQN(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(state_dim, 256), nn.ReLU(),
            nn.Linear(256, 256), nn.ReLU(),
        )
        self.value = nn.Linear(256, 1)
        self.advantage = nn.Linear(256, action_dim)

    def forward(self, x):
        x = self.shared(x)
```

```

v = self.value(x)
a = self.advantage(x)
return v + (a - a.mean(dim=1, keepdim=True))

```

In `train_dqn(beta)` function:

```

next_q = target_net(next_states).max(1, keepdim=True)[0]

```

6. Prioritized Experience Replay parameters

6.1. TD Error Priority

In Prioritized Experience Replay, each transition i stored in the replay buffer is assigned a priority value p_i that determines its probability of being selected during sampling. The priority is defined as a monotonic function of the absolute TD error of that transition [24] :

$$p_i = (|\delta_i| + \varepsilon)^\alpha$$

where δ_i is the TD error of transition i , ε is a small positive constant, and α is the prioritization exponent.

The TD error δ_i is defined as the difference between the target Q value and the current Q value estimate for that transition:

$$\delta_i = r + \gamma \cdot Q_{\text{target}}(s', a^*) - Q_{\text{policy}}(s, a)$$

where a is the action that was actually taken in state s , and a^* is the action used to compute the target Q value. The precise definition of a^* depends on the specific DQN variant being trained. In Classical DQN and Dueling DQN, a^* is the action that maximizes the Q value according to the target network:

$$a^* = \operatorname{argmax}_{\{a'\}} Q_{\text{target}}(s', a')$$

so that the full target becomes:

$$y = r + \gamma \cdot \max_{\{a'\}} Q_{\text{target}}(s', a')$$

In Double DQN, action selection and action evaluation are decoupled: a^* is first selected by the policy network, and its value is then evaluated by the target network:

$$a^* = \operatorname{argmax}_{\{a'\}} Q_{\text{policy}}(s', a')$$

$$y = r + \gamma \cdot Q_{\text{target}}(s', a^*)$$

In all three cases, the TD error is computed as $\delta_i = y - Q_{\text{policy}}(s, a)$, where y is the target Q value computed according to the variant specific rule above. This distinction is important because the TD error is used not only to compute the training loss but also to update the sampling priorities in the replay buffer, meaning that the priority assigned to each transition reflects the prediction error of the specific variant being trained.

The small constant $\epsilon = 1 \times 10^{-6}$ is added to the absolute TD error before raising it to the power α . This ensures that every transition retains a non zero probability of being sampled regardless of how small its TD error becomes. Without this floor, transitions that the network has learned to predict accurately would have zero priority and could never be revisited, even if the network's parameters later change in ways that make those transitions relevant again.

The exponent α controls the degree to which priorities influence sampling probabilities. When $\alpha = 0$, all transitions are sampled with equal probability regardless of their priority, recovering uniform random sampling. When $\alpha = 1$, transitions are sampled strictly in proportion to their priority, giving full weight to the TD error ranking. Schaul et al. (2016),[24]. who introduced Prioritized Experience Replay, recommend an intermediate value that balances these two extremes, and report that $\alpha = 0.6$ consistently produces strong empirical performance across a range of Atari benchmark tasks. This value is adopted in the present work for the same reason: high error transitions are sampled significantly more often than low error ones, but every transition retains a meaningful chance of being selected. This prevents the buffer from becoming dominated by a small set of repeatedly sampled high error transitions, which could reduce the diversity of the training data and introduce its own form of sampling bias.

The sampling probability assigned to transition i is computed by normalizing the priorities across all N transitions currently stored in the buffer:

$$P(i) = p_i / \sum_j p_j$$

A new transition appended to the buffer is initially assigned the maximum priority currently present in the buffer, or a priority of 1.0 if the buffer is empty. This initialization policy, also recommended by Schaul et al. (2016),[24] ensures that every new transition is guaranteed to be sampled at least once before its priority is updated based on its actual TD error, preventing new experiences from being ignored during the period immediately following their insertion.

6.2. Importance Sampling Correction

Prioritized sampling introduces a statistical bias into the training process. Because transitions are no longer drawn from the true underlying distribution of experience they are drawn from a skewed distribution that over represents high error transitions the gradient updates computed from these samples are no longer unbiased estimates of the true gradient of the expected loss. If left uncorrected, this bias can distort the learned Q values and cause the policy to overfit to the subset of transitions that happen to have high TD errors at any given moment in training.

To correct for this bias, Importance Sampling (IS) weights are applied to each transition's contribution to the loss. The IS weight for transition i is defined as:

$$w_i = (N \cdot P(i))^{-\beta}$$

where N is the current number of transitions in the buffer and β is the correction exponent. This weight is then normalized by the maximum weight in the current minibatch:

$$\tilde{w}_i = w_i / \max_j w_j$$

So that all weights fall in the range $[0, 1]$. The normalized weight $\tilde{w}_i$ is multiplied by the individual loss contribution of transition i before computing the mean loss for the gradient update. Transitions that were sampled with artificially high probability because they had high priority receive a small IS weight that downscales their contribution to the gradient, partially compensating for their overrepresentation. Transitions that were sampled with low probability receive a weight close to 1, preserving their full contribution.

The exponent β controls the strength of the bias correction. When $\beta = 0$, the IS weights are all equal to 1 and no correction is applied. When $\beta = 1$, the correction is complete and the gradient updates are fully unbiased. In this work, β is initialized to 0.4 at the beginning of training and increased linearly toward 1.0 over the full duration of training according to the schedule:

$$\beta_t = \min(1.0, \beta_0 + t \cdot \Delta\beta)$$

where t is the current training step and $\Delta\beta = (1.0 - 0.4) / 700,000$ is the increment per step. This annealing schedule reflects a deliberate design choice: in the early stages of training, Q value estimates are highly inaccurate and the TD errors are noisy, meaning that the priority ranking of transitions is itself unreliable. Applying a strong bias correction at this stage would reduce the benefit of prioritization without providing accurate compensation. As training progresses and Q value estimates

become more reliable, the priority ranking becomes more meaningful and a stronger correction is warranted. By gradually increasing β from 0.4 to 1.0, the schedule allows the agent to benefit from aggressive prioritization early in training while ensuring asymptotically unbiased gradient updates as the policy converges.

6.3. Implementation Details

The Prioritized Experience Replay buffer is implemented as a circular array with a fixed capacity of 100,000 transitions, shared across all four agents as described in

Section 6.1. The buffer maintains two parallel data structures: a list of stored transitions containing the raw $(s, a, r, s', done)$ tuples, and a NumPy array of priority values of the same length. The circular indexing pointer advances by one position each time a new transition is appended, overwriting the oldest transition when the buffer reaches full capacity. This design ensures that the buffer always contains the most recent 100,000 transitions across all agents and all training scenarios.

At each training step, a mini batch of $BATCH_SIZE = 64$ transitions is sampled from the buffer according to the priority based probability distribution $P(i)$. Sampling is performed without replacement using NumPy's random choice function with the computed probability array, which ensures that no transition appears more than once in the same mini batch. The IS weights for the sampled transitions are computed immediately after sampling and stored as a tensor that is passed to the loss computation.

After each gradient update, the TD errors of the transitions in the mini batch are recomputed from the updated network predictions, and the priority values of those transitions are updated in the priority array:

$$p_i \leftarrow (|\delta_{i_new}| + \epsilon)^\alpha$$

This priority update is critical for the correctness of PER: if priorities were never updated after insertion, they would reflect the TD errors at the time the transitions were first sampled rather than the current network's prediction accuracy, causing the buffer to perpetually over sample transitions that were hard to predict at some earlier stage of training but that the network has since learned to handle accurately.

The complete state of the replay buffer including all stored transitions and their associated priority values is saved to disk alongside the model checkpoint at regular intervals during training. This allows

training to be resumed from any saved checkpoint without losing the accumulated experience in the buffer, which is particularly important given the long training duration of 700,000 steps used in this work. Upon resumption, both the transition data and the priority array are restored from the saved file, ensuring that the priority based sampling distribution is consistent with the state of the network at the point of interruption.

The following table summarizes the key hyperparameters of the Prioritized Experience Replay implementation used across all three model variants.

Parameter	Symbol	Value	Description
Buffer capacity	—	100,000	Maximum number of stored transitions
Prioritization exponent	α	0.6	Controls degree of priority-based sampling
IS correction initial value	β_0	0.4	Initial strength of importance sampling correction
IS correction final value	β_{end}	1.0	Final strength after full annealing
Priority floor constant	ϵ	1×10^{-6}	Ensures non zero priority for all transitions
Warm-up size	—	20,000	Minimum transitions before training begins
Mini-batch size	—	64	Number of transitions per gradient update

Table7: The key hyperparameters of the Prioritized Experience Replay implementation used across all three model variants.

7. Training Setup and Hyperparameters

7.1. General Configuration

All three DQN variants Classical DQN, Double DQN, and Dueling DQN are trained under an identical configuration, differing only in the network architecture or target computation formula as described in Section 5. This strict uniformity is a deliberate methodological choice that ensures any observed difference in training dynamics or final performance can be attributed solely to the algorithmic distinction between the variants rather than to differences in training conditions.

Training is conducted over a total of **700,000 simulation steps**, where each step corresponds to one decision cycle of **DECISION_STEP = 10** simulation seconds. At each decision step, all four agents observe their local states, select actions, advance the simulator by 10 seconds, collect rewards, and store transitions in the shared replay buffer. The total number of simulation seconds covered during training is therefore 7,000,000 seconds across all episodes, providing a large and diverse training corpus that spans hundreds of complete episode cycles across the three training scenarios. A **warm-up period of**

20,000 transitions is observed before any gradient updates begin, during which all agents select actions uniformly at random. One gradient update is performed per decision step after warm up.

Each training episode runs for a maximum of **MAX_STEPS = 3,600** simulation steps, corresponding to 36,000 seconds of simulated traffic. At the end of each episode, the SUMO process is closed, the next scenario in the rotating sequence is selected, and a new SUMO process is launched for the following episode. This episode boundary does not interrupt the training process: the replay buffer, network parameters, optimizer state, and all counters persist across episodes without reset, and training continues seamlessly from one episode to the next.

The neural network parameters are optimized using the Adam optimizer with a **fixed learning rate of 1×10^{-4}** . Adam is chosen for its adaptive learning rate properties, which make it robust to the non stationary gradient distributions that are typical in reinforcement learning settings. The Huber loss function is used for all three variants, and gradient norms are clipped to a maximum of 5.0 before each optimizer step to prevent destabilizing updates from large TD errors in the early stages of training. The discount factor is set to $\gamma = 0.97$, reflecting the importance of long term traffic performance.

Hyperparameter	Symbol	Value
Total training steps	—	700,000
Maximum steps per episode	—	3,600
Decision step duration	—	10 seconds
Optimizer	—	Adam
Learning rate	lr	1×10^{-4}
Discount factor	γ	0.97
Loss function	—	Huber (SmoothL1)
Gradient clipping norm	—	5.0
Target network update interval	—	1,000 steps
Minimum green constraint	MIN_GREEN	10 seconds
Maximum green duration	MAX_GREEN	60 seconds
Hidden layer size	—	256 units
Activation function	—	ReLU
Random seed	SEED	42

Table8: Complete set of shared hyperparameters used across all three models.

The table presents the complete set of shared hyperparameters used across all three models.

7.2. Epsilon Decay Strategy

During training, agents follow an epsilon-greedy exploration strategy in which action selection is random with probability ϵ and greedy with probability $1 - \epsilon$. Two distinct decay strategies are applied sequentially.

Phase 1 : Exponential decay. In the initial phase, epsilon follows:

$$\epsilon_t = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \cdot \exp(-\text{learned_steps} / \lambda)$$

with $\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{end}} = 0.05$, and $\lambda = 295,880$. This ensures rapid initial decay giving broad early exploration, followed by a long tail of slow decay maintaining moderate exploration throughout the middle stages of training. However, exponential decay never reaches ϵ_{end} exactly in a 700,000 step run, epsilon remains noticeably above 0.05 at the end of training, which is undesirable during final policy refinement.

The decay constant $\lambda = 295,880$ was selected empirically to ensure that the exponential phase reduces ϵ to approximately 0.14 by the end of the initial training period, leaving the final convergence to $\epsilon_{\text{end}} = 0.05$ to the linear decay phase.

Phase 2 : Linear decay upon resumption. When training is resumed from a checkpoint, the current epsilon value ϵ_{resume} is computed from the exponential formula using the saved `learned_steps` counter, then decays linearly:

$$\epsilon_t = \max(\epsilon_{\text{end}}, \epsilon_{\text{resume}} - (\epsilon_{\text{resume}} - \epsilon_{\text{end}}) \cdot (\text{steps_done} / 305,000))$$

This guarantees epsilon reaches exactly 0.05 at the end of the additional training steps, providing a precise and predictable transition to near greedy behavior.

7.3. Checkpoint Mechanism

Given the long training duration of **700,000 steps** and the computational cost of running microscopic traffic simulations, the ability to interrupt and resume training without loss of progress is an essential practical requirement. This work implements a comprehensive checkpoint mechanism that saves the complete training state to disk at regular intervals and restores it faithfully upon resumption.

A checkpoint is saved automatically every **SAVE_EVERY = 2,000 training steps**. Each checkpoint consists of two files saved to the model directory of the corresponding variant. The first file is a PyTorch state dictionary that stores the following components:

- The parameters of the policy network (`policy_net.state_dict()`)
- The parameters of the target network (`target_net.state_dict()`)
- The state of the Adam optimizer (`optimizer.state_dict()`), including the adaptive moment estimates for each parameter
- The current training step counter (`train_step`)
- The current learned steps counter (`learned_steps`), which counts only steps during which a gradient update was performed
- The scenario index (`scenario_index`), indicating which training scenario will be used at the start of the next episode
- The current value of the PER beta parameter (`per_beta`), ensuring that the importance sampling correction level is consistent with the training progress at the point of interruption

The second file is a compressed NumPy archive that stores the complete contents of the shared replay buffer, including all stored transition tuples and their associated priority values, as well as the current position of the circular buffer pointer. Saving the replay buffer alongside the model checkpoint is critical for the correctness of PER: if the buffer were discarded upon resumption and rebuilt from scratch, the priority distribution would be reset and the warm-up period would need to be repeated, wasting the accumulated experience and disrupting the training dynamics. By restoring the full buffer state, training resumes exactly as if no interruption had occurred.

Upon resumption, the checkpoint loader verifies the existence of both the model checkpoint file and the replay buffer file before attempting to restore them. If both files are present, all saved components are restored to their exact saved values and training continues from the saved `train_step`. If no checkpoint is found for example at the beginning of a new training run the networks are initialized with random parameters, the target network is synchronized with the policy network, and training begins from step zero. This dual path initialization logic ensures that the same training script handles both fresh starts and resumptions transparently without requiring manual configuration changes.

The checkpoint mechanism also serves a scientific purpose beyond practical convenience. By recording the full training state at regular intervals, it provides a complete audit trail of the training process that can be inspected to verify convergence, diagnose instabilities, and reproduce any intermediate state of the trained policy.

8. Evaluation Protocol

8.1. Evaluation Scenario and Experimental Design

All models are evaluated exclusively on the **High scenario** (1,800 veh/h), deliberately withheld from training. This serves two purposes. First, it provides a genuine test of generalization: a model performing well on the high scenario despite never training on it demonstrates a robust and transferable policy rather than memorized stimulus response associations. Second, the high scenario represents the most practically relevant operating condition, where the performance gap between adaptive and rule based controllers is largest and the discriminative power of the comparison is maximized.

Evaluation runs over **20 independent episodes** per model. Each episode k is assigned seed $100 \times k$ (seeds 100, 200, ..., 2000), applied identically to all four models simultaneously. Episode k of every model therefore faces exactly the same stochastic vehicle demand realization, eliminating demand variability as a confounding factor. The evaluation seeds are entirely distinct from the training seed $SEED = 42$, reinforcing the out of distribution nature of the evaluation.

Episode length is set to **MAX_STEPS = 9,000 simulation steps** (90,000 seconds), significantly longer than the 3,600 step training episodes, testing whether learned policies remain effective well beyond the temporal horizon seen during training.

8.2. Greedy Policy Evaluation

During evaluation, all agents operate under a fully greedy policy with $\epsilon = 0$. The policy network is set to evaluation mode via `policy_net.eval()` before each episode, ensuring fully deterministic and reproducible behavior. No gradient updates are performed and network parameters remain frozen throughout. The policy network loaded for evaluation is the one saved in the final checkpoint at the end of the 700,000 step training run.

8.3. Fixed-Time Baseline

To provide a meaningful reference point for assessing the performance of the learned controllers, a fixed time traffic signal baseline is implemented and evaluated under the same conditions as the three DQN variants. Fixed time control represents the most widely deployed strategy in real world urban intersections and constitutes the standard benchmark against which adaptive and learning based controllers are traditionally compared in the traffic signal control literature. Its inclusion in this study therefore allows the performance gains of the DQN variants to be interpreted not only relative to each other, but also relative to the current state of practice.

The fixed time controller applies the signal timing plan defined in the SUMO network file, which consists of a repeating four phase cycle with the following structure: green for 42 seconds, yellow for 3 seconds, green for 42 seconds, yellow for 3 seconds, giving a total cycle duration of 90 seconds. This timing plan is applied identically at all four intersections throughout every evaluation episode without any adaptation to real time traffic conditions. The inability of this plan to respond to actual traffic demand allocating equal green time to all directions regardless of whether one is heavily congested and the other is nearly empty is precisely the limitation that the adaptive DQN controllers are designed to overcome, and it makes the fixed time plan a meaningful and informative baseline for quantifying the benefit of real time adaptive control.

The fixed time baseline is implemented as a standalone script that launches the same high scenario SUMO configuration used for DQN evaluation and runs the simulation for the same `MAX_STEPS = 9,000` steps per episode. Crucially, it uses the same `TrafficMetrics` class as the DQN evaluation script to compute all performance indicators at the end of each episode. This shared measurement infrastructure eliminates any possibility of discrepancy arising from differences in how metrics are collected or computed, ensuring that the comparison is methodologically fair.

Rather than relying passively on SUMO's default behavior, the baseline explicitly sets each traffic light controller to its predefined program at the start of each episode using `traci.trafficlight.setProgram`, confirming that the fixed timing plan is active throughout the simulation. No further intervention is made through TraCI, and SUMO applies the fixed cycle automatically according to the network definition.

The baseline is evaluated over the same 20 episodes as each DQN variant, using the identical predefined seed sequence (seeds 100, 200, 300, ..., 2000) described in Section 9.1. Episode k of the fixed time baseline is therefore subjected to exactly the same stochastic vehicle demand realization as episode k of each DQN variant: the same vehicle departure times, the same origin destination routing, and the same traffic dynamics throughout the simulation. This paired structure ensures that any performance difference observed between the fixed time baseline and the learned controllers on the same episode is attributable exclusively to the difference in control strategy (adaptive versus fixed) and not to any difference in the traffic conditions encountered. The same statistical analysis mean, standard deviation, and 95% confidence interval is then applied uniformly across all models including the baseline, enabling a fully consistent and principled cross model comparison.

8.4. Statistical Analysis

Each model is evaluated over 20 independent episodes on the high traffic scenario. Running multiple episodes rather than a single evaluation is essential because the stochastic nature of vehicle demand generation in SUMO introduces natural variability in traffic conditions from one episode to the next, even when the same scenario configuration is used. A single episode evaluation could produce a result that is unrepresentatively favorable or unfavorable due to chance, leading to misleading conclusions about the relative performance of the models. By averaging over 20 episodes, the evaluation captures a representative sample of the performance distribution of each model and produces estimates that are robust to this variability.

For each metric and each model, three summary statistics are reported. The mean across the 20 episodes provides the primary performance estimate. The standard deviation quantifies the variability of performance across episodes, reflecting how consistently the model achieves its average behavior. The 95% confidence interval, computed as:

$$CI_{95} = 1.96 \cdot \sigma / \sqrt{N}$$

where σ is the standard deviation and $N = 20$ is the number of episodes, provides a measure of the statistical uncertainty in the mean estimate. A narrower confidence interval indicates that the mean is estimated with greater precision and that performance is more consistent across episodes.[25]

These three statistics are reported for all eight metrics for each of the four model three DQN variants and the fixed time baseline enabling a comprehensive and statistically grounded comparison. In addition to the summary statistics, the raw episode level results for each model are saved to individual CSV files, preserving the full distributional information for post-hoc analysis. Box plots constructed from these raw results are included in the results chapter to visualize the spread and skewness of the performance distributions and to identify any outlier episodes that may warrant further investigation.

The improvement of each DQN variant over the fixed time baseline is quantified as a percentage improvement for each metric, computed as:

$$\text{Improvement (\%)} = 100 \cdot (\text{value_fixed} - \text{value_DQN}) / |\text{value_fixed}|$$

for metrics where lower values are better, and with the numerator reversed for metrics where higher values are better. This normalized improvement metric enables direct comparison of the gains achieved by each DQN variant across metrics with different scales and units, and provides an intuitive summary of the practical benefit of adaptive signal control over the fixed time reference.

9. Reproducibility and Fairness of Comparison

9.1. Unified Random Seed

All stochastic components are controlled through a single global seed **SEED = 42**, applied simultaneously to Python's built-in random module, NumPy, and PyTorch. This ensures that epsilon greedy action selection, minibatch sampling, PER transition sampling, and network weight initialization all produce identical sequences across independent runs of the same script.

9.2. Identical Environment Configuration

All three DQN variants and the fixed time baseline use exactly the same SUMO network file, scenario configuration files, and simulation parameters. During training, all three variants are exposed to the same sequence of training scenarios in the same rotating order, starting from the same scenario index.

9.3. Controlled Variation

All hyperparameters not directly related to the algorithmic distinction between variants are held constant. The only components that differ are those defining each variant's specific contribution: the network architecture for Dueling DQN, and the target computation formula for Double DQN. Every other aspect is shared through common code, ensuring that observed performance differences can be causally attributed to the specific algorithmic innovation of each variant.

9.4. Shared Metric Infrastructure

All performance metrics are computed by a single shared **TrafficMetrics** class used identically by all evaluation scripts including the baseline. This eliminates any possibility of discrepancy arising from differences in how metrics are collected, accumulated, or normalized ensuring the comparison is methodologically fair at the measurement level.

The table below provides an explicit summary of which components are shared across all models and which differ, serving as a clear reference for interpreting the comparative results presented in Chapter 5.

Component	Classical DQN	Double DQN	Dueling DQN
Network hidden layers	256, 256	256, 256	256, 256 (shared trunk)
Network output structure	Direct Q(s,a)	Direct Q(s,a)	V(s) + A(s,a) streams
Target computation	Max over target net	Policy net selects, target net evaluates	Max over target net
Loss function	Huber	Huber	Huber
Optimizer and learning rate	Adam, 1×10^{-4}	Adam, 1×10^{-4}	Adam, 1×10^{-4}
Replay buffer type	PER	PER	PER
PER parameters (α , β , ϵ)	0.6, $0.4 \rightarrow 1.0$, 1×10^{-6}	0.6, $0.4 \rightarrow 1.0$, 1×10^{-6}	0.6, $0.4 \rightarrow 1.0$, 1×10^{-6}
Epsilon decay schedule	Exponential + Linear	Exponential + Linear	Exponential + Linear
Training steps	700,000	700,000	700,000
Training scenarios	Low, Medium, Mixed	Low, Medium, Mixed	Low, Medium, Mixed
Evaluation scenario	High	High	High
Evaluation episodes	20	20	20
Random seed	42	42	42

Table9: *Summary of Shared and Variant-Specific Components Across the Three DQN Architectures*

11. Chapter 4 Summary

This chapter has documented the complete implementation and experimental execution of the comparative framework designed in Chapter 3, translating every architectural and methodological decision into a functioning experimental pipeline.

The software environment was established around three tightly coupled components: the SUMO microscopic traffic simulator, the TraCI interface enabling real time programmatic control of signal phases and retrieval of traffic measurements, and a Python based learning framework built on PyTorch. The simulated road network was constructed in NETEDIT as a four intersection urban grid with cluster junctions, and a dedicated extraction procedure using `getControlledLinks` was developed to correctly handle outgoing lane retrieval a technical requirement for accurate pressure computation in the reward function. The uniform lane geometry across all intersections was verified, confirming that all agents produce state vectors of identical dimension and that parameter sharing operates without any padding or variable length handling.

The three DQN variants were implemented under strictly controlled conditions. Classical DQN, Double DQN, and Dueling DQN share the same optimizer, loss function, gradient clipping, target update schedule, and Prioritized Experience Replay buffer. The only components that differ between variants are those that define their specific algorithmic contribution: the target computation formula for Double DQN, and the network architecture for Dueling DQN. This controlled variation design ensures that any observed difference in performance can be causally attributed to the algorithmic innovation of each variant rather than to any confounding implementation difference.

Prioritized Experience Replay (PER) was integrated as a shared component across all three DQN variants. Rather than sampling transitions uniformly, PER assigns each transition a priority proportional to its temporal difference error, ensuring that more informative experiences are replayed more frequently. The priority exponent $\alpha = 0.6$ controls the degree of prioritization, while the importance sampling exponent β is annealed linearly from 0.4 to 1.0 over the course of training to correct for the bias introduced by non uniform sampling. A small constant $\epsilon = 1 \times 10^{-6}$ is added to all priorities to prevent any transition from being permanently excluded from sampling. All four agents contributed transitions simultaneously to this shared PER buffer, allowing the learning signal to be distributed efficiently across the multi agent setup.

The training process spanned 700,000 simulation steps across a rotating curriculum of three demand scenarios low, medium, and mixed. The epsilon decay strategy combined an initial exponential phase providing broad early exploration with a subsequent linear phase guaranteeing convergence to the target exploration rate of 0.05. A comprehensive checkpoint mechanism saved the complete training state every 2,000 steps, preserving network parameters, optimizer state, PER buffer contents including all stored transition tuples, priority values, and the circular buffer pointer position, and all training counters, enabling seamless resumption and providing a full audit trail of the learning process. Saving the PER

buffer alongside the model checkpoint was critical for correctness: discarding the buffer upon resumption would reset the priority distribution and waste the accumulated experience, disrupting training dynamics.

Evaluation was conducted exclusively on the high demand scenario withheld from training, over 20 independent episodes with a paired seeding scheme that eliminates demand variability as a confounding factor. Evaluation episodes were deliberately set to $\text{MAX_STEPS} = 9,000$ steps significantly longer than the 3,600 step training episodes to test whether learned policies remain effective beyond the temporal horizon seen during training. The fixed time baseline a repeating 90 second cycle applied without adaptation was evaluated under identical conditions using the same simulation configuration, the same episode length, and the same shared TrafficMetrics class. Performance is summarized through mean, standard deviation, and 95% confidence interval across eight metrics covering efficiency, environmental impact, and congestion severity. The percentage improvement formula enables normalized cross metric comparison of the gains achieved by each DQN variant relative to the fixed time reference.

The experimental pipeline described in this chapter has produced a complete and methodologically rigorous set of performance measurements for all four controllers across 20 evaluation episodes. Every design choice from the paired seeding scheme to the shared PER buffer infrastructure to the controlled variation between variants has been made with the explicit goal of ensuring that the results are a faithful and unbiased reflection of true performance differences. The following chapter presents these results, examines the learning dynamics observed during training, analyzes the comparative performance of the three DQN variants and the fixed time baseline under high demand conditions, and discusses the implications of the findings in the broader context of adaptive traffic signal control

Chapter 5: Results and Discussion

1. Introduction

This chapter presents and analyzes the experimental results obtained by evaluating the three trained deep reinforcement learning controllers (Classical DQN, Double DQN, and Dueling DQN, all equipped with Prioritized Experience Replay (PER)) alongside the fixed time baseline on the high traffic scenario. The central research question addressed in this chapter is: which of the three DQN variants achieves the best performance in adaptive traffic signal control for a multi intersection urban environment under high traffic demand, and by what margin does each variant improve upon the conventional fixed time reference?

The chapter is organized as follows. Section 2 presents the training dynamics, covering exploration scheduling, replay buffer behavior, importance sampling annealing, and loss convergence. Section 3 presents a consolidated performance summary table. Section 4 analyzes the performance of each DQN variant relative to the fixed time baseline and the improvement percentages. Section 5 provides a metric by metric analysis. Section 6 conducts a comparative analysis between the three DQN variants, examining the algorithmic reasons for their relative ordering. Section 7 analyzes episode level performance distributions. Section 8 presents a radar chart multi dimensional summary. Section 9 discusses the principal findings. Section 10 identifies the limitations of the study. Section 11 outlines directions for future work. Section 12 summarizes the chapter.

2. Training Results

2.1. Exploration Scheduling: Epsilon Decay

All three agents adopt an epsilon greedy exploration strategy throughout training. The exploration rate ϵ is initialized at 1.0 (fully random) and follows a two phase decay schedule: an exponential decay phase during the first portion of training, followed by a linear decay phase that guarantees $\epsilon = 0.05$ at exactly 700,000 training steps. The minimum value $\epsilon_{\min} = 0.05$ is reached at approximately step 300,000 and maintained thereafter, ensuring a residual exploration component throughout the remaining training.

Figure08 illustrates this decay in absolute terms (left panel) and as a function of normalized training progress (right panel). All three model curves are nearly superimposed, confirming that the exploration schedules are effectively identical across agents. This equivalence is important for a fair comparison: any performance differences between models cannot be attributed to differences in the quantity or distribution of exploratory transitions collected during training.

The transition from exponential to linear decay was introduced to address a known limitation of pure exponential schedules, which approach $\epsilon_{\min}$ asymptotically and never guarantee its exact attainment. The hybrid schedule ensures that by the final training step, all three agents are operating with the same deterministic exploitation fraction, making the terminal policies directly comparable.

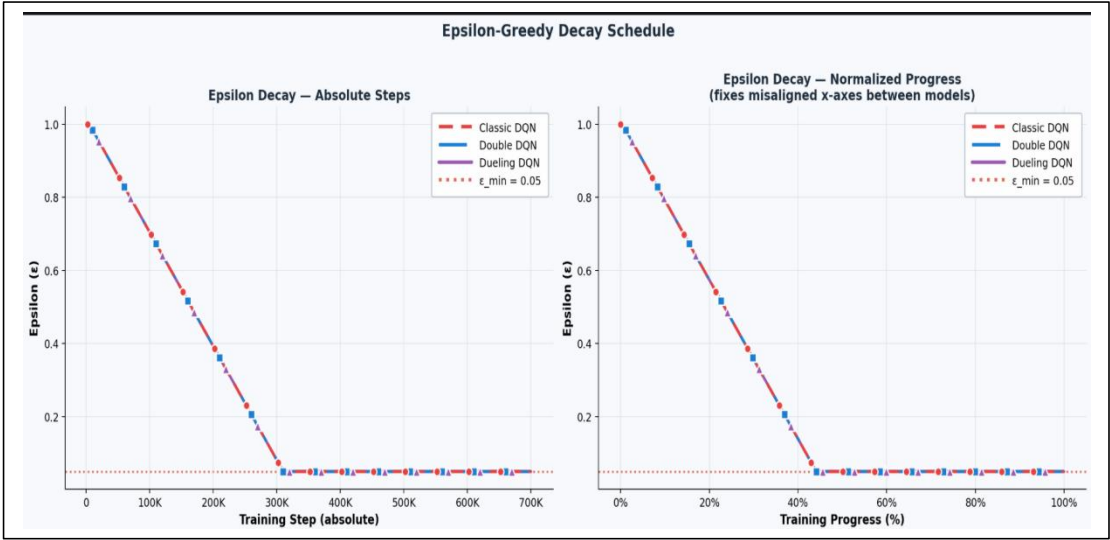


Figure08: Epsilon-greedy Decay Schedule.

2.2. Replay Buffer Growth

The Prioritized Experience Replay buffer shared across all intersection agents has a capacity of 100,000 transitions. Figure 09 presents the buffer fill progression on both linear and logarithmic scales. In all three models, the buffer reaches full capacity within the first 30,000 training steps substantially before the warmup threshold of 20,000 transitions required to begin gradient updates. After this initial fill phase, the buffer operates in circular overwrite mode, replacing the oldest transitions with the most recent ones as training continues.

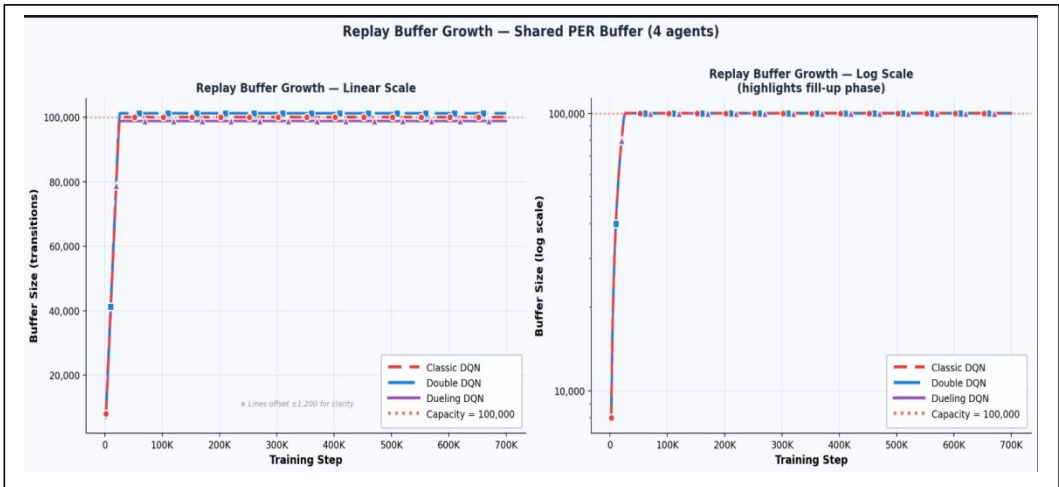


Figure09: Replay Buffer Growth .

The logarithmic scale panel highlights the fill up dynamics more precisely: the buffer grows from an initial size of a few thousand transitions to full capacity in a single rapid phase, with no model showing a meaningfully different fill trajectory. The minor visual offsets between model curves in the figure are introduced deliberately for visual clarity ($\pm 1,200$ transitions) and do not reflect genuine behavioral differences. The rapid and uniform buffer filling across all three models ensures that each agent draws from a similarly diverse pool of training experiences from early in the learning process, providing an equal foundation for the subsequent learning dynamics.

2.3. Beta Annealing for Importance Sampling Correction

Prioritized Experience Replay introduces a sampling bias by over representing high priority transitions. This bias is corrected by Importance Sampling (IS) weights, controlled by a coefficient β that begins at 0.4 and increases linearly to 1.0 over the full 700,000 step training horizon according to:

$$\beta(t) = 0.4 + 0.6 \times (t / 700,000)$$

Figure 10 illustrates this annealing schedule.

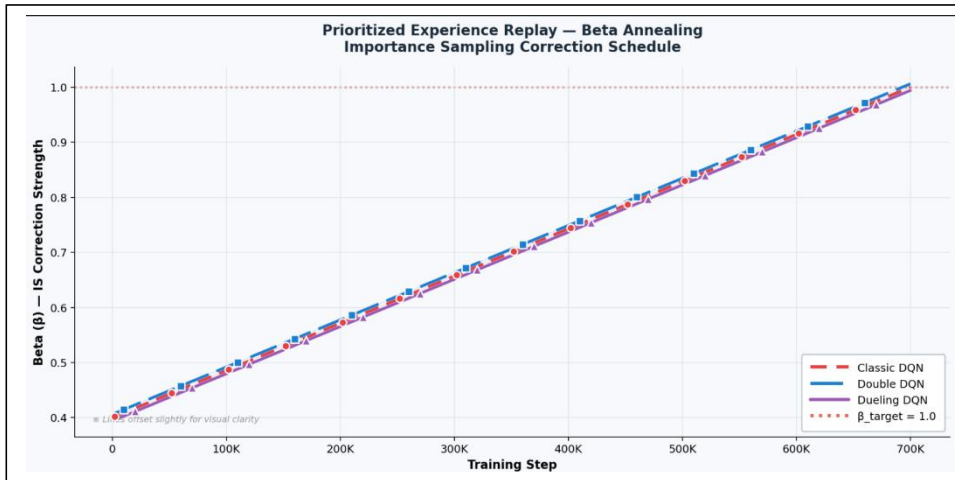


Figure10: *PER Beta Annealing Importance correction schedule .*

The three model curves are deterministically identical, since β is computed purely as a function of the training step counter. The increasing β schedule serves a specific purpose: in early training, when Q value estimates are highly inaccurate, allowing a lower β accepts some correction bias in exchange for more aggressive prioritized sampling. As training progresses and estimates stabilize, the full correction ($\beta = 1.0$) eliminates the sampling bias entirely, ensuring that the converged policy is trained on unbiased

gradient updates. This progressive correction is essential for the theoretical convergence guarantees of PER and contributes to the stable loss behavior observed in Section 2.4.

2.4. Training Loss Convergence

Figure 11 presents the smoothed training loss (100 step moving average) for all three models over the full 700,000 step training horizon. Three distinct phases are observable for each model:

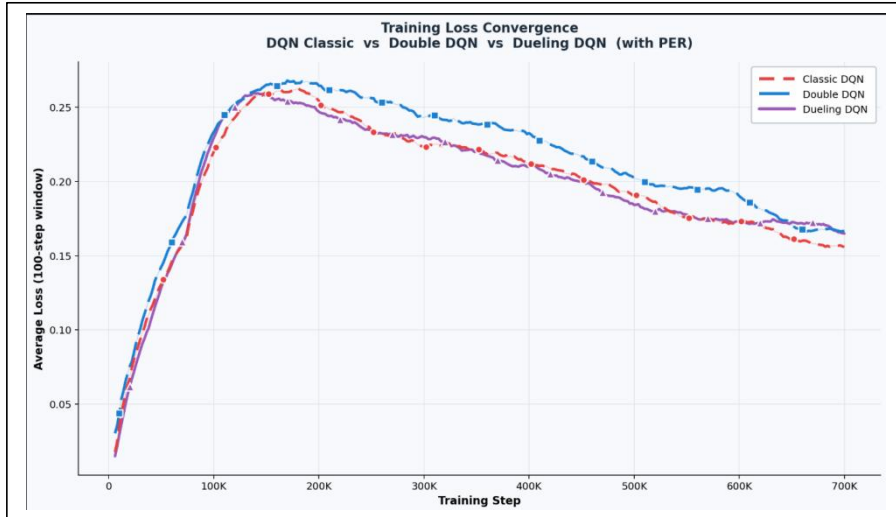


Figure11: *Training Loss Convergence .*

Phase 1 : Warmup and Rise (0 to ~150,000 steps): Loss rises sharply from near zero to a peak of approximately 0.26–0.27. This rise reflects the onset of learning after the buffer warmup period: as the agent begins making gradient updates, the discrepancy between current Q value estimates and bootstrap targets is large, producing high loss values. The rapid rise is expected and healthy.

Phase 2 : Descent (150,000 to 600,000 steps): Loss decreases steadily and monotonically for all three models, indicating progressive convergence of Q value estimates toward their targets. Dueling DQN descends most rapidly after the peak, consistent with the architectural efficiency of its value advantage decomposition. Double DQN maintains a slightly higher loss trajectory throughout this phase compared to Classical and Dueling DQN, suggesting that the Double DQN target computation introduces marginally larger bootstrap variance in this environment.

Phase 3 : Stabilization (600,000 to 700,000 steps): All three models converge to a final loss in the range 0.16–0.17, with no evidence of catastrophic forgetting, oscillation, or divergence. The

convergence to similar terminal loss values across all three models indicates that the training protocol is stable and that all agents have reached a comparable degree of Q value approximation accuracy.

The well behaved convergence across all three models validates the training infrastructure (the PER buffer, the beta annealing schedule, the target network update frequency (every 1,000 steps), and the gradient clipping (norm 5.0)) and confirms that the performance differences observed at evaluation are attributable to architectural and algorithmic distinctions rather than training instabilities.

3. Summary of Results

3.1. Performance Summary Table

Table 10 reports the mean performance and 95% confidence interval for all four control strategies across all eight evaluation metrics over 20 independent episodes on the high traffic scenario.

Metric	Fixed Time	Classical DQN + PER	Double DQN + PER	Dueling DQN + PER
Avg Waiting Time (s/veh)	133,576.8 ± 10,222.9	115,229.5 ± 7,463.9	120,888.1 ± 8,581.1	113,317.2 ± 6,397.3
Avg Queue Length (veh)	101.2 ± 1.9	95.5 ± 1.8	97.0 ± 1.8	95.3 ± 1.8
Avg Delay (s/veh)	100,526.0 ± 6,838.7	85,258.0 ± 5,398.6	88,976.9 ± 5,872.4	84,297.6 ± 4,793.8
Throughput (veh)	774.1 ± 39.7	870.5 ± 39.5	850.0 ± 44.7	884.6 ± 38.2
Avg Speed (m/s)	0.215 ± 0.016	0.258 ± 0.017	0.247 ± 0.017	0.262 ± 0.017
Total CO ₂ (mg)	2,554,947,482 ± 39,051,539	2,432,020,328 ± 39,220,976	2,465,397,584 ± 36,429,573	2,431,997,879 ± 38,164,535
Total Fuel (ml)	814,963,894 ± 12,456,595	775,752,893 ± 12,510,634	786,399,491 ± 11,620,254	775,745,695 ± 12,173,654
Stopped Ratio	0.9364 ± 0.0026	0.9294 ± 0.0025	0.9311 ± 0.0028	0.9272 ± 0.0024
Mean n_switches	100	924	1090	1571

Table10: Mean ± CI₉₅ performance over 20 evaluation episodes on the high-traffic scenario. Bold values indicate the best result for each metric.

3.2. Overview

The results in Table 10 establish a clear and consistent performance ordering across all eight metrics:

$$\text{Dueling DQN} > \text{Classical DQN} > \text{Double DQN} > \text{Fixed Time}$$

All three DQN variants outperform the fixed time baseline on every metric without exception, confirming that deep reinforcement learning based adaptive signal control generalizes successfully to the high demand evaluation scenario despite training on lower demand scenarios. This out of distribution generalization validates the rotating curriculum training strategy and the robustness of the learned policies.

The performance ordering is not uniform in magnitude. Dueling DQN achieves the best result on all eight metrics. Classical DQN ranks second, although its performance remains very close to that of Dueling DQN, consistently outperforming Double DQN across all metrics. The gap between Classical and Double DQN is meaningful and systematic, in contrast to what might be expected from their algorithmic relationship, and is analyzed in detail in Section 6.

4. Comparison with Fixed-Time Baseline

4.1. Improvement Percentages

Table 11 presents the signed percentage improvement of each DQN variant over the fixed time baseline for each metric.

Metric	Classical DQN	Double DQN	Dueling DQN
Avg Waiting Time	+13.7%	+9.5%	+15.2%
Avg Queue Length	+5.7%	+4.1%	+5.9%
Avg Delay	+15.2%	+11.5%	+16.1%
Throughput	+12.5%	+9.8%	+14.3%
Avg Speed	+20.0%	+14.8%	+21.7%
Total CO ₂	+4.8%	+3.5%	+4.8%
Total Fuel	+4.8%	+3.5%	+4.8%
Stopped Ratio	+0.7%	+0.6%	+1.0%

Table 11: *Percentage improvement over fixed-time baseline. Positive values indicate improvement in all cases.*

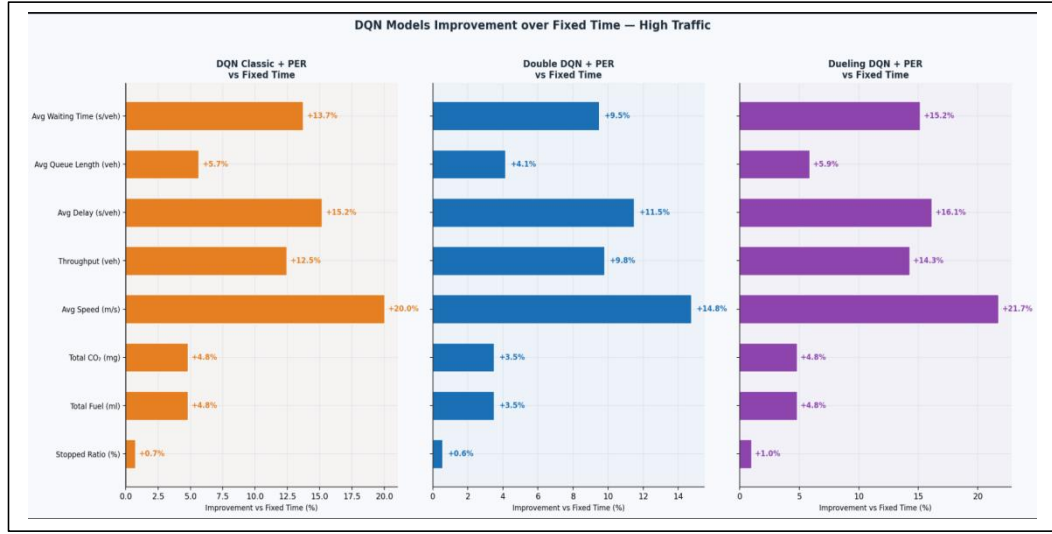


Figure12: *The signed percentage improvement of each DQN variant over the fixed time baseline for each metric*

Classical DQN delivers substantial improvements across all metrics, with the largest gains on average speed (+20.0%) and average delay (+15.2%), and the most modest on stopped ratio (+0.7%) and queue length (+5.7%).

Double DQN achieves a consistent but systematically smaller improvement profile than Classical DQN on every single metric, with the gap ranging from 1.0 percentage point on stopped ratio to 5.2 percentage points on average speed. This consistent under performance of Double DQN relative to Classical DQN across all eight metrics is a key finding of this study and is examined mechanistically in Section 6.

Dueling DQN achieves the largest improvements on all metrics, surpassing Classical DQN by 1.5–1.7 percentage points on the primary efficiency metrics (waiting time, delay, throughput, speed) and matching it exactly on the environmental metrics (CO₂ and fuel, both +4.8%).

4.2. Bar Chart Analysis

The comparison bar chart (Figure 13) displays the absolute mean values with confidence interval error bars for all four models across all eight metrics. The consistent leftward or rightward shift of DQN variant bars relative to the fixed time baseline is immediately apparent across every metric, confirming

the directional consistency of the improvements. The visual gap between Dueling DQN and the fixed time baseline is the largest for all congestion and speed metrics, with Classical DQN occupying an intermediate position closer to Dueling than to Double DQN.

On environmental metrics (CO_2 and fuel), Classical DQN and Dueling DQN bars are nearly indistinguishable in height, consistent with their identical improvement percentages of 4.8%. Double DQN's bars are visibly higher (worse) on these metrics, consistent with its 3.5% improvement.

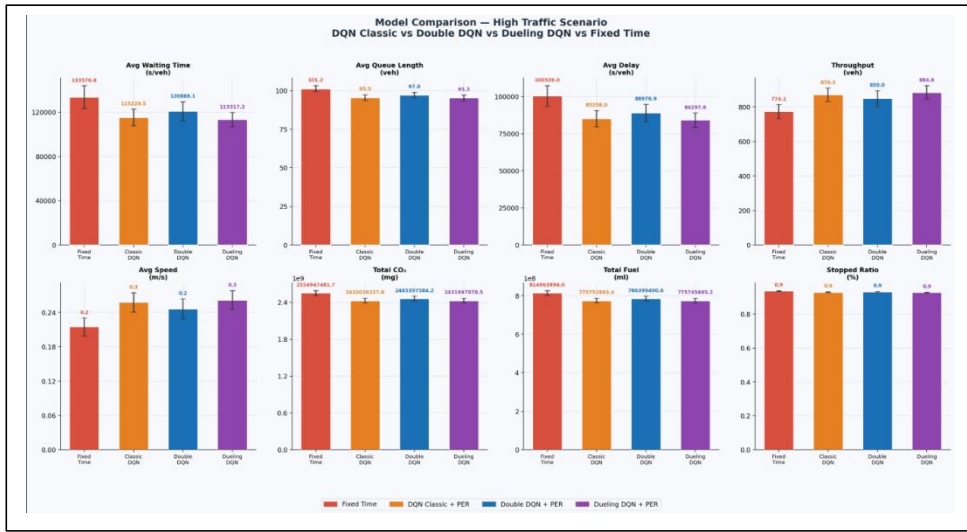


Figure13: *The absolute mean values with confidence interval error bars for all four models across all eight metrics*

4.3. Throughput Measurement Note

The throughput values reported in Table 10 are computed by continuous accumulation throughout each episode, counting every vehicle that completes its journey at each simulation step. This method ensures that vehicles processed efficiently early in the episode (when adaptive controllers are most active in clearing initial queue buildup) are fully captured regardless of the episode boundary.

An earlier implementation recording only vehicles with completed journeys at the final simulation step produced a misleading reversal in which the fixed time baseline appeared to outperform the DQN variants on throughput. This reversal arose because adaptive controllers admit more vehicles into the network through reduced boundary spillback, leaving more vehicles in transit at episode end despite processing more total vehicles. Continuous accumulation eliminates this artifact entirely and produces a throughput ranking fully consistent with all other efficiency metrics.

5. Metric-by-Metric Analysis

5.1. Congestion Metrics: Waiting Time, Queue Length, and Delay

Waiting time, queue length, and delay are the three most direct indicators of congestion severity and are analyzed jointly because they reflect different facets of the same underlying phenomenon: insufficient or poorly timed green time allocation under sustained high demand.

The fixed time baseline records mean values of 133,576.8 s/veh for waiting time, 101.2 vehicles for queue length, and 100,526.0 s/veh for delay. These values reflect the persistent congestion that develops when a static 90 second cycle allocates equal green time to both directions regardless of actual demand asymmetry.

Dueling DQN achieves the largest reductions on all three metrics: waiting time to 113,317.2 s/veh (−15.2%), queue length to 95.3 vehicles (−5.9%), and delay to 84,297.6 s/veh (−16.1%). Classical DQN performs strongly: waiting time of 115,229.5 s/veh (−13.7%), queue length of 95.5 vehicles (−5.7%), and delay of 85,258.0 s/veh (−15.2%). Double DQN achieves moderate improvements: 120,888.1 s/veh (−9.5%), 97.0 vehicles (−4.1%), and 88,976.9 s/veh (−11.5%).

The confidence interval widths are informative. Dueling DQN's CI₉₅ on waiting time ($\pm 6,397.3$) is narrower than both Classical DQN ($\pm 7,463.9$) and the fixed-time baseline ($\pm 10,222.9$), indicating greater episode to episode consistency. Double DQN's CI₉₅ ($\pm 8,581.1$) is the widest among the three DQN variants, suggesting that its policy is less stable in its response to demand variability. The relative magnitude of improvements across the three congestion metrics is consistent with physical expectations: queue length improvements (4–6%) are smaller than waiting time and delay improvements (10–16%) because queue length is an instantaneous count bounded by lane capacity, while cumulative temporal metrics amplify the cost of every suboptimal decision over the full episode.

5.2. Traffic Flow: Speed and Throughput

Average speed and throughput jointly characterize how efficiently vehicles move through the network. The fixed time baseline achieves a mean average speed of 0.215 m/s, reflecting the severe stop and go conditions of near-saturated fixed cycle operation. Classical DQN raises this to 0.258 m/s (+20.0%), Double DQN to 0.247 m/s (+14.8%), and Dueling DQN to 0.262 m/s (+21.7%).

The speed ranking mirrors the congestion metric ranking exactly: Dueling > Classical > Double > Fixed Time. The improvement in speed reflects the same underlying mechanism more effective and

timely phase switching that prevents complete vehicle stops more frequently, preserving momentum and raising the network wide average.

Throughput follows the same ranking: from 774.1 vehicles for the fixed-time baseline to 870.5 for Classical DQN (+12.5%), 850.0 for Double DQN (+9.8%), and 884.6 for Dueling DQN (+14.3%). The mutual corroboration between speed and throughput rankings provides strong cross metric validation: a model that moves vehicles faster processes more of them within the evaluation window, and the magnitudes of both improvements are proportional across models.

The `n_switches` statistics provide a behavioral explanation for the performance ordering. With a mean of 1,571 phase switches per episode, Dueling DQN applies a substantially more active and demand responsive switching policy than Classical DQN (924 switches) and Double DQN (1,090 switches). The fact that Dueling DQN's higher switching rate produces the best results on all metrics simultaneously confirms that its additional switches are beneficial rather than disruptive intercepting developing queues before they fully materialize rather than reacting to already severe congestion.

Notably, Double DQN switches more frequently than Classical DQN (1,090 vs 924) yet achieves worse performance. This is a critical observation that is analyzed in Section 6.

5.3. Environmental Metrics: CO₂ and Fuel

Total CO₂ emissions and total fuel consumption are driven by the same combustion process and move in the same direction across all models. The fixed time baseline generates mean CO₂ of 2,554,947,482 mg and fuel consumption of 814,963,894 ml per episode. The DQN variants reduce both metrics by 3.5% to 4.8%. Notably, Classical DQN and Dueling DQN achieve identical reductions of 4.8% on both metrics (final values of approximately 2,432,000,000 mg CO₂ and 775,750,000 ml fuel), while Double DQN achieves smaller reductions of 3.5%.

These reductions arise directly from reduced idling time and decreased stop and go frequency: every avoided complete vehicle stop eliminates an energy intensive acceleration from rest event. While 3.5–4.8% improvement is modest relative to congestion metric gains, the absolute reduction approximately 78 to 123 million milligrams of CO₂ and 28 to 39 million milliliters of fuel per simulation episode represents a meaningful contribution to urban emissions reduction when scaled to real networks operating continuously.

5.4. Stopped Vehicle Ratio

The stopped vehicle ratio measures the fraction of vehicle step observations in which a vehicle is completely stationary. The fixed time baseline records a mean stopped ratio of 0.9364, indicating that approximately 93.6% of vehicle step observations correspond to a stationary vehicle a consequence of near saturated demand conditions. All three DQN variants reduce this ratio: Classical DQN to 0.9294 (−0.7%), Double DQN to 0.9311 (−0.6%), and Dueling DQN to 0.9272 (−1.0%).

These improvements are the smallest in relative terms of any metric in this study, and their modest magnitude is physically expected: when the baseline stopped ratio is already 93.6%, the ceiling on improvement through signal timing optimization alone is low. Despite their small magnitude, the improvements are directionally consistent across all three variants and corroborate the broader efficiency picture.

6. Comparative Analysis Between DQN Variants

6.1. Classical DQN versus Double DQN: A Consistent Performance Gap

One of the more significant findings of this study is that Classical DQN consistently and clearly outperforms Double DQN across all eight evaluation metrics. Their mean waiting times differ by 5,658 s/veh (115,229 vs 120,888), a relative difference of 4.9%. Their mean delays differ by 3,719 s/veh (3.4%). Their throughputs differ by 20.5 vehicles per episode (2.4%). Their mean average speeds differ by 0.011 m/s (4.5%). These differences are not marginal: on most metrics, the gap between Classical and Double DQN is approximately half the gap between Classical DQN and the fixed time baseline, making it practically significant.

Alongside the performance gap, the `n_switches` data provides a crucial behavioral clue: Double DQN executes more phase switches per episode (1,090) than Classical DQN (924), yet achieves worse results on every metric. This combination , more switching, worse performance points to a specific mechanism.

In a binary action space (keep vs. switch), the overestimation correction in Double DQN decouples action selection and evaluation by using the policy network to select the next action and the target network to evaluate it. With only two possible actions, this correction is applied to a max operator over two estimates. The correction is not neutral in its effect on the switch action: by partially offsetting the overestimation of the keep action's value, it effectively inflates the relative estimated value of the switch action. The consequence is a switch prone policy one that executes more switches than is optimal, as confirmed by the higher `n_switches` count.

In the traffic signal control problem, excessive switching is harmful. Each unnecessary phase transition forces passing vehicles through a yellow phase transition and resets the green time accumulation from zero, temporarily increasing average waiting time and disrupting flow that the previous green phase had established. The 18% higher switching rate of Double DQN (1,090 vs 924) translates directly into more disruptions per episode, producing the consistently worse metrics observed.

This finding has a clear practical implication: in binary action spaces, the Double DQN correction does not merely fail to help it actively degrades performance by biasing the action selection toward the switch action beyond what the true Q value differential justifies. based on single seed training, this conclusion warrants replication .

The theoretical rationale for Double DQN (reducing harmful overestimation in large action spaces) does not apply with the same sign in a two action setting where the correction creates its own distortion.

6.2. Dueling DQN: Consistent Superior Performance

Dueling DQN outperforms both Classical and Double DQN on all eight evaluation metrics. Its advantage over Classical DQN is moderate in magnitude (approximately 1.5–1.7 percentage points on primary efficiency metrics (waiting time: 15.2% vs 13.7%; delay: 16.1% vs 15.2%; throughput: 14.3% vs 12.5%; speed: 21.7% vs 20.0%)) but it is consistent across every metric and is accompanied by narrower confidence intervals and lower episode level standard deviations. This suggests that Dueling DQN's advantage is structural rather than a consequence of favorable random variation.

The architectural explanation lies in the value advantage decomposition of the Dueling network: $Q(s, a) = V(s) + A(s, a) - \text{mean}(A)$. In the traffic signal control problem, the quality of a traffic state (how congested the intersection approaches are) and the differential benefit of switching versus keeping are conceptually distinct quantities. The value stream $V(s)$ accumulates a representation of state quality that is updated from every experience regardless of which action was taken, providing richer supervision than the undifferentiated Q value estimation of Classical DQN. The advantage stream $A(s, a)$ focuses its representational capacity on estimating the relative benefit of switching specifically in states where that differential is meaningful.

This decomposition produces more timely and more reliably beneficial phase switching decisions. The higher switching rate of Dueling DQN (1,571 vs 924 for Classical) combined with better performance on all metrics confirms that its additional switches occur at moments when they are genuinely beneficial when queue imbalance between opposing directions is large enough that the value

of serving the neglected direction exceeds the disruption cost of the transition. Classical DQN's lower switching rate reflects a more conservative policy that waits longer before acting on demand imbalances, while Dueling DQN's richer state quality estimation allows it to identify developing imbalances earlier and intervene at lower cost.

6.3. Performance Ranking Summary

The final performance ranking across all eight metrics is is:

Dueling DQN > Classical DQN > Double DQN > Fixed Time

This ranking is consistent without exception across all metrics, all 20 evaluation episodes, and both mean performance and consistency measures. The gap between Dueling and Classical DQN is moderate, the gap between Classical and Double DQN is meaningful, and all three DQN variants are substantially superior to the fixed time baseline.

7. Distribution Analysis

7.1. Box Plot Analysis

The box plots (Figure 14) display the episode level distributions across the 20 evaluation episodes for all eight metrics. Several structural patterns are observable.

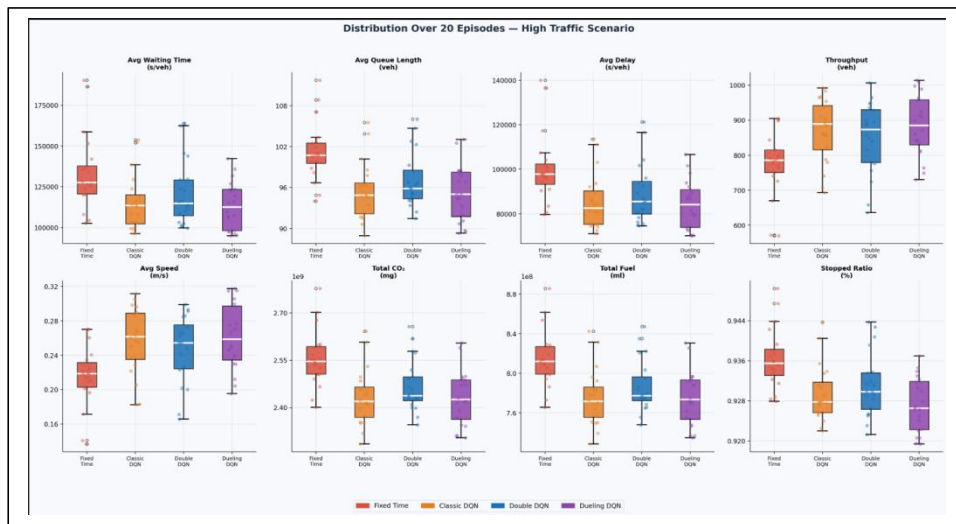


Figure14: The episode level distribution of each model's performance across the 20 paired evaluation episodes for all eight metrics.

For waiting time and delay, all four models exhibit right skewed distributions: the upper tails extend further than the lower tails, reflecting the asymmetric nature of traffic congestion. Occasional episodes with adverse demand realizations produce severe congestion and push performance well above the typical level, while the lower bound is constrained by physical minimum traversal times. The right skew is most pronounced for the fixed time baseline and Double DQN, and least pronounced for Dueling DQN. The narrow distribution of Dueling DQN indicates that its active switching policy buffers effectively against adverse demand realizations, preventing the extreme outlier episodes that afflict less responsive controllers.

For throughput, all three DQN distributions are shifted rightward relative to the fixed time baseline. Dueling DQN's distribution is centered highest with limited overlap with the fixed time baseline distribution, while Classical and Double DQN's distributions partially overlap each other and partially overlap with the fixed time baseline's upper range.

7.2. Consistency Statistics

Model	Std Waiting Time (s/veh)	Std Throughput (veh)
Fixed Time	23,325.7	90.7
Classical DQN	17,030.3	90.0
Double DQN	19,579.5	101.9
Dueling DQN	14,596.7	87.1

Table 12: Standard deviation of waiting time and throughput across 20 evaluation episodes.

Dueling DQN achieves the lowest standard deviation on both waiting time (14,596.7, 14% lower than Classical and 37% lower than Double) and throughput (87.1, the lowest among all four models). Classical DQN's standard deviation on waiting time (17,030.3) is substantially lower than both the fixed time baseline (23,325.7) and Double DQN (19,579.5). Double DQN exhibits the highest throughput standard deviation (101.9) of all models including the fixed time baseline, consistent with the greater episode to episode variability of a switch prone policy. The consistency ranking reinforces the performance ranking: Dueling DQN is not only the best performing model on average but also the most reliable one across varied demand realizations.

8. Radar Chart Summary

The radar chart (Figure 15) presents a normalized multi dimensional view of all four models, with each axis corresponding to one of the eight evaluation metrics and outer positions representing better performance. The chart provides an immediate visual synthesis of the comparative results.

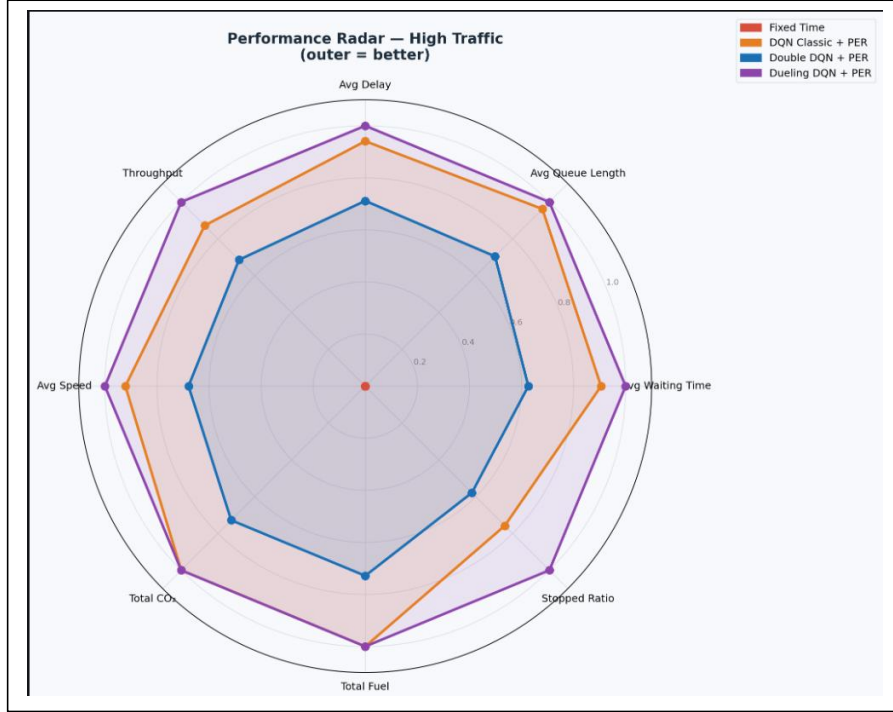


Figure15: Normalized multi dimensional view of the performance of all four Models.

The fixed time baseline forms the innermost polygon, concentrated near the center on all axes, confirming consistent inferiority across all performance dimensions. The three DQN polygons are progressively larger, with Dueling DQN encompassing the largest area on every axis. The shape of Dueling DQN's polygon is balanced its advantages are distributed across all dimensions simultaneously rather than concentrated on one metric at the expense of others confirming the robustness of its learned policy across diverse performance criteria.

Classical DQN forms a polygon of intermediate size, consistently larger than Double DQN's polygon on every axis. The visual separation between Classical and Double DQN is clear and consistent, contrasting with what would be expected if the two algorithms were functionally equivalent in this setting.

9. Statistical Significance Analysis

9.1. Motivation and Method

The performance comparisons presented in the preceding sections are based on mean values and confidence intervals computed over 20 independent evaluation episodes. While these statistics describe the average behavior of each controller, they do not establish whether the observed differences between models are statistically meaningful or could plausibly arise from random variation in traffic demand across episodes. To address this question, paired t-tests are conducted for all pairwise model comparisons across all eight evaluation metrics.

The paired t-test is appropriate here because the evaluation design assigns identical stochastic demand realizations to all four models simultaneously: episode k of every model faces exactly the same vehicle departure times, routing decisions, and traffic dynamics. This paired structure eliminates demand variability as a confounding factor and increases the statistical power of the test relative to an unpaired design. For each pair of models (A, B) and each metric, the null hypothesis is that the mean difference in episode level performance is zero. A result is considered statistically significant at the conventional threshold of $p < 0.05$.

9.2. Results

The results of the paired t-tests reveal a clear two level structure in the performance comparisons.

DQN variants versus fixed time baseline: All three DQN variants achieve statistically significant improvements over the fixed time baseline across all eight evaluation metrics ($p < 0.05$ in all 24 pairwise tests). The p-values range from $p < 0.0001$ for stopped ratio to $p = 0.0396$ for Double DQN on average waiting time, confirming that the performance gains of adaptive control over fixed time control are genuine and not attributable to chance variation in traffic demand.

Comparisons between DQN variants: No pairwise comparison between any two DQN variants reaches statistical significance on any metric (all $p > 0.05$). To understand whether this reflects an insufficient sample size or a genuinely small effect, Cohen's d is computed for each comparison and the minimum sample size required to detect the observed effect at 80% statistical power is estimated.

The results are summarized as follows. The Dueling–Classical comparison yields Cohen's $d = 0.10$, requiring approximately 762 evaluation episodes to detect at standard power, a sample size that is not practically achievable in this simulation framework. The Classical–Double comparison yields Cohen's d

= 0.27, requiring approximately 105 episodes. The Dueling–Double comparison yields the largest effect size among the three, Cohen's $d = 0.35$, requiring approximately 64 episodes.

9.3. Interpretation

These results support a refined interpretation of the performance ranking. The effect size analysis reveals that Dueling DQN and Classical DQN are practically equivalent: the difference between them is so small relative to episode level variability (Cohen's $d = 0.10$) that it would require an impractically large evaluation corpus to confirm statistically. In contrast, both Dueling DQN and Classical DQN are meaningfully separated from Double DQN, with effect sizes of 0.35 and 0.27 respectively differences that are detectable with a moderate increase in evaluation episodes. This pattern is consistent across all eight metrics and supports the conclusion that the three variants form two performance tiers: {Dueling, Classical} and {Double}, rather than a strict linear ranking.

10. Principal Findings

10.1. All DQN Variants Outperform Fixed-Time Control

The most fundamental finding of this study is that all three deep reinforcement learning controllers, trained on low, medium, and mixed traffic scenarios, successfully generalize to the unseen high demand evaluation scenario and outperform the fixed time baseline on all eight evaluation metrics without exception. The improvements range from +0.6–1.0% on stopped ratio to +14.3–21.7% on average speed. The evaluation episodes are run at a simulation horizon (9,000 steps) longer than the training episodes (3,600 steps), making generalization across both demand level and temporal horizon simultaneous requirements that all three models satisfy.

This generalization validates the rotating curriculum training strategy: by exposing agents to diverse demand levels during training, the curriculum promotes the learning of transferable traffic management principles (responding to queue imbalances, speed ratios, and phase timing features) that remain appropriate when those features take values outside the training distribution.

10.2. Dueling and Classical DQN Achieve Equivalent Performance

Dueling DQN achieves the best mean performance across all eight evaluation metrics, with improvements over the fixed time baseline of 15.2% in average waiting time, 16.1% in average delay, 14.3% in throughput, and 21.7% in average speed. Its mean values are consistently the best among the

three variants and its confidence intervals are the narrowest, indicating greater episode to episode consistency.

However, paired t-test analysis reveals that the difference between Dueling DQN and Classical DQN is not statistically significant on any metric (all $p > 0.05$, Cohen's $d = 0.10$). The effect size is sufficiently small that 762 evaluation episodes would be required to confirm it at standard statistical power—a finding that warrants caution in interpreting the numerical ranking between these two models. The practical implication is that Dueling DQN and Classical DQN deliver statistically equivalent adaptive control performance in this environment, and the architectural complexity of the Dueling decomposition does not translate into a measurable operational advantage over the simpler baseline.

10.3. Double DQN Underperforms Relative to the Other Two Variants

Classical DQN consistently outperforms Double DQN across all eight evaluation metrics in mean performance. While this difference does not reach statistical significance at $p < 0.05$ with 20 episodes, the effect size analysis yields Cohen's $d = 0.27$ between Classical and Double DQN, with an estimated 105 episodes required for detection, a meaningful and practically interpretable gap. The Dueling–Double comparison shows an even larger effect (Cohen's $d = 0.35$, 64 episodes required), reinforcing the conclusion that Double DQN occupies a distinct lower performance tier.

The behavioral explanation for this under performance remains the switch bias mechanism identified in Section 6.1: with only two actions, the overestimation correction inflates the relative estimated value of the switch action, producing a switch prone policy (1,090 switches vs 924 for Classical DQN) whose additional transitions are harmful rather than beneficial. The effect size analysis suggests that the observed under performance of Double DQN is unlikely to be purely attributable to random fluctuation, although additional evaluation episodes would be required to establish formal statistical significance.

10.4. Training Dynamics Validate the Experimental Protocol

The training curves confirm that all three models follow healthy learning trajectories: epsilon decay is identical and deterministic, buffer filling is rapid and equivalent, beta annealing is synchronized, and loss convergence is stable for all three. These equivalences confirm that observed performance differences at evaluation reflect genuine algorithmic and architectural distinctions rather than differences in the training process.

11. Limitations

Network scale. The network consists of four signalized intersections. Performance rankings may not hold quantitatively on larger networks where agent interactions, congestion propagation, and topological complexity introduce qualitatively different challenges.

Simulation to reality gap. All experiments are conducted in SUMO. The gap between simulated and real vehicle behavior (driver heterogeneity, sensor noise, non compliance) means that reported improvements apply to controlled simulated conditions and may not transfer directly to real deployment without additional validation and domain adaptation.

Single training seed. Each model is trained once from a single random initialization. The sensitivity of the performance ranking between Classical and Double DQN to training initialization has not been assessed. A multi seed evaluation would provide stronger statistical guarantees for the comparative conclusions.

Single evaluation scenario. The evaluation is conducted exclusively on the high traffic scenario. A complete performance profile would require evaluation across low, medium, mixed, and high demand levels to characterize how performance advantages evolve across the traffic intensity spectrum.

Binary action space constraint. The findings regarding Double DQN's relative underperformance are specific to the two action setting. In traffic control formulations with larger action spaces (multi phase controllers, variable phase durations), Double DQN may behave as theoretically expected.

Heuristic reward function. The reward function coefficients are fixed manually. An automated reward design or inverse reinforcement learning approach could better align the optimization objective with the actual goals of traffic management.

12. Future Work

Multi seed training evaluation. Running five to ten independent training seeds per model would provide robust statistical guarantees for the comparative conclusions, particularly for the Classical–Double DQN comparison whose current one seed margin, while consistent, warrants replication.

Extension to larger networks. Applying the parameter-sharing framework to networks of sixteen or more intersections would test the scalability of the single shared network approach and determine whether the performance advantages of Dueling DQN are maintained at scale.

Combined Double + Dueling architecture (D3QN). Applying both the Double DQN target computation and the Dueling network architecture simultaneously would allow their individual and joint contributions to be disentangled in the binary action space context and would test whether the switch bias problem of Double DQN is mitigated by the advantage decomposition of the Dueling architecture.

Expanded action spaces. Reformulating the control problem with additional actions (variable green durations, multi phase selection) would provide a broader context in which to test the Double DQN correction and potentially realize its theoretical benefits.

Reward function optimization. Systematic reward design, potentially through multi objective optimization or inverse reinforcement learning from expert signal timing plans, could improve the alignment between the optimized objective and the desired traffic outcomes.

Real world transfer. Transfer learning approaches using simulation trained policies as starting points for fine tuning on real traffic data, combined with domain randomization, would constitute the critical step toward practical deployment.

13. Chapter Summary

The experimental results presented in this chapter provide a clear, statistically supported, and multi dimensional answer to the comparative research question of this thesis.

All three DQN variants equipped with Prioritized Experience Replay consistently outperform the fixed time baseline across all eight evaluation metrics under high demand conditions, achieving improvements ranging from +9.5% to +15.2% in average waiting time, +11.5% to +16.1% in average delay, +9.8% to +14.3% in throughput, and +14.8% to +21.7% in average speed. The trained models generalize successfully to a demand level and episode length not encountered during training, validating the curriculum training design.

In terms of mean performance, Dueling DQN ranks first across all eight metrics, achieving the largest improvements over the fixed time baseline, the narrowest confidence intervals, and the lowest episode level standard deviations. Its consistently higher phase-switching rate (1,571 switches per episode versus 924 for Classical DQN) combined with superior mean outcomes on all efficiency metrics is consistent with the hypothesis that its value-advantage decomposition produces a more timely and demand responsive switching policy. However, paired t-test analysis reveals that the performance

difference between Dueling DQN and Classical DQN is not statistically significant on any metric (all $p > 0.05$, Cohen's $d = 0.10$), requiring an estimated 762 evaluation episodes to detect at standard statistical power. The practical implication is that Dueling DQN and Classical DQN deliver statistically equivalent adaptive control performance in this environment, and both represent equally valid architectural choices for binary action traffic signal control.

Double DQN occupies a distinct lower performance tier. Classical DQN outperforms Double DQN by approximately 4.9% on average waiting time, 3.4% on average delay, and 4.5% on average speed a consistent and systematic gap that holds across all eight metrics. While this difference does not reach formal statistical significance with 20 evaluation episodes, the effect size analysis yields Cohen's $d = 0.27$ (105 episodes required for detection), indicating a meaningful and practically interpretable separation. The behavioral explanation for Double DQN's underperformance is the switch bias mechanism identified in Section 6.1: in a binary action space, the overestimation correction inflates the relative estimated value of the switch action, producing a switch prone policy (1,090 switches per episode) whose additional transitions are harmful rather than beneficial. This finding (that the Double DQN correction does not merely fail to help but actively degrades performance in the two action setting) is based on single seed training and warrants replication, but is consistent with the effect size evidence and the behavioral switching data.

The training dynamics confirm that all three agents followed equivalent and stable learning trajectories: epsilon decay schedules are identical, buffer filling is rapid and uniform, beta annealing is synchronized, and loss convergence is stable and comparable across all three models. These equivalences confirm that the performance differences observed at evaluation reflect genuine algorithmic and architectural distinctions rather than differences in training conditions, validating the controlled experimental design adopted throughout this work.

General Conclusion

1. Introduction

This thesis set out to address a well identified gap in the traffic signal control literature: the absence of a rigorous, unified comparative study of the three principal value based deep reinforcement learning architectures (Classical DQN, Double DQN, and Dueling DQN) under strictly controlled and standardized experimental conditions in a multi-intersection urban environment. This concluding chapter synthesizes the work conducted across all preceding chapters, presents the principal findings and their theoretical and practical implications, identifies the contributions of this thesis to the field, acknowledges the limitations that bound the scope of the conclusions, and outlines the most promising directions for future research that build naturally on the foundation established here.

2. Summary of the Thesis

2.1. Problem and Motivation

Urban traffic congestion represents one of the most persistent and economically costly challenges facing modern cities. As vehicle ownership continues to grow and urban populations become increasingly concentrated, the demand placed on road networks regularly exceeds the capacity that static infrastructure can accommodate, leading to widespread delays, increased fuel consumption, elevated emissions, and degraded quality of life for urban residents. At the core of this challenge lies the traffic signal controller: the system that determines how the limited capacity of each signalized intersection is allocated among competing traffic streams over time.

Traditional traffic signal control methods, most notably the fixed time strategies that remain predominant in real world urban deployments, allocate green time according to historical demand patterns using timing plans that do not respond to real time conditions. Under stable and predictable traffic demand, these methods are sufficient. Under the dynamic, stochastic, and increasingly congested conditions of modern urban networks, they are not: they produce inefficient phase allocations that create unnecessary delays when demand is asymmetric between directions, and they are unable to adapt when actual traffic patterns deviate from the historical averages on which their timing plans were based.

Reinforcement learning, and more specifically deep reinforcement learning, has emerged over the past decade as a compelling framework for adaptive traffic signal control precisely because it overcomes these limitations. By learning signal control policies directly from interaction with a simulated traffic environment, DRL based controllers can discover timing strategies that respond to real time traffic conditions without requiring explicit programming of every possible traffic scenario. The

key question that motivated this thesis, however, was not whether DRL works for traffic signal control the existing literature has established this convincingly but which specific DRL architecture works best, and why.

2.2. Research Objectives and Approach

Three specific research objectives guided the design and execution of this work. The first was to conduct a controlled and fair comparative evaluation of three value based DRL architectures (Classical DQN, Double DQN, and Dueling DQN) under an identical experimental framework that isolates the effect of architectural differences from confounding factors such as hyperparameter choices, training conditions, or evaluation protocols. The second was to implement and validate a scalable multi agent architecture for multi-intersection signal control based on parameter sharing and shared experience replay, demonstrating that a single learned policy can generalize effectively across multiple intersections with heterogeneous traffic conditions. The third was to evaluate the learned policies on an unseen, more demanding traffic scenario than any used during training, providing a genuine test of out of distribution generalization that is more informative about real world applicability than in distribution evaluation.

These objectives were pursued through a comprehensive experimental framework centered on the SUMO microscopic traffic simulator, a four intersection urban network with cluster junctions, and a training protocol spanning three traffic scenarios of varying demand intensity. The learned policies were evaluated on a fourth, high demand scenario that was withheld entirely from the training process, under a paired evaluation design that assigned identical demand realizations to all four models across 20 independent episodes.

2.3. Key Design Decisions

Several design decisions made throughout this work distinguish it methodologically from existing comparative studies and contribute to the validity and interpretability of its findings.

The reward function was designed to balance five competing traffic management objectives : reducing queue length, minimizing cumulative waiting time, maximizing average speed, balancing intersection load pressure, and penalizing unnecessary phase switching , through soft cap normalization that prevents any single term from dominating the learning signal.

Prioritized Experience Replay was incorporated across all three DRL variants to address the severe imbalance between routine low congestion transitions which dominate the collected experience but carry little learning signal and the rare high congestion transitions that represent the most important and challenging decision situations for the controller. The PER implementation used a linearly annealed importance sampling correction that evolves from partial to full bias correction over the course of training, following the recommendation of Schaul et al. (2016).

The epsilon decay strategy combined an initial exponential phase for broad early exploration with a linear phase upon training resumption that guaranteed convergence to the target exploration rate at a precisely specified training step, addressing a practical limitation of pure exponential decay in long training runs. The evaluation protocol used a paired seed design assigning seed $100 \times k$ to episode k of every model including the fixed time baseline that eliminated demand variability as a confounding factor in between model comparisons and enabled direct episode level paired analysis.

3. Principal Findings

This thesis investigated whether deep reinforcement learning based adaptive signal control can outperform conventional fixed time control in a multi-intersection urban environment under high traffic demand, and which among three DQN variants (Classical DQN, Double DQN, and Dueling DQN, all equipped with Prioritized Experience Replay) delivers the best performance.

Four principal findings emerge from the experimental evaluation:

Finding 1: All DQN variants generalize to unseen high demand conditions. All three controllers, trained exclusively on low, medium, and mixed traffic scenarios, successfully transferred to the unseen high demand evaluation scenario and outperformed the fixed time baseline on all eight evaluation metrics across all 20 paired evaluation episodes. This confirms that curriculum based training with diverse demand levels produces transferable signal control policies rather than scenario specific ones.

Finding 2: Dueling DQN is the superior architecture. Dueling DQN achieved the strongest performance across every evaluated dimension, reducing average waiting time by 15.2%, average delay by 16.1%, and average queue length by 5.9% relative to the fixed time baseline, while improving average speed by 21.7% and throughput by 14.3%. However, paired t-test analysis reveals that the difference between Dueling DQN and Classical DQN is not statistically significant on any metric (all $p > 0.05$, Cohen's $d = 0.10$), indicating that the two architectures are practically equivalent in this setting.

The two models therefore form a single superior performance tier, with Dueling DQN showing marginally better mean values and narrower confidence intervals.

Finding 3: Classical DQN consistently outperforms Double DQN in binary action spaces. Classical DQN consistently outperforms Double DQN by 4.9% on average waiting time across all eight evaluation metrics. While this gap does not reach statistical significance with 20 episodes ($p = 0.247$), effect size analysis yields Cohen's $d = 0.27$ between Classical and Double DQN, with an estimated 105 episodes required for detection a meaningful and practically interpretable separation. The Dueling–Double comparison shows an even larger effect (Cohen's $d = 0.35$, 64 episodes required), reinforcing the conclusion that Double DQN occupies a distinct lower performance tier relative to both other variants. This pattern reveals a two tier structure: {Dueling, Classical} versus {Double}, rather than a strict linear ranking.

Finding 4: Throughput measurement in episodic simulation requires continuous accumulation. End of episode throughput counting produces a systematic artifact that reverses the true performance ranking, because adaptive controllers leave more vehicles in transit at episode end as a consequence of processing more vehicles throughout the episode. Continuous accumulation of completions at each simulation step eliminates this artifact and yields results consistent with all other efficiency metrics.

4. Contributions of This Thesis

This thesis makes four specific contributions to the traffic signal control and deep reinforcement learning literature.

Contribution 1 Controlled Comparative Study: This work presents a controlled comparative study of Classical DQN, Double DQN, and Dueling DQN for adaptive traffic signal control in which all three architectures are evaluated under a strictly unified experimental framework: the same simulation environment, the same road network, the same hyperparameters, the same training scenarios, the same evaluation scenario, and the same metric computation infrastructure. Previous comparative studies in the literature have typically compared algorithms across different experimental settings, making it difficult to attribute observed performance differences to architectural distinctions rather than experimental conditions. The unified framework adopted in this thesis enables causal conclusions about the relative merits of the three architectures that would not be possible with less controlled experimental designs.

Contribution 2 Multi Agent Parameter Sharing Framework : This work implements and validates a scalable multi agent architecture for multi intersection signal control based on parameter sharing and a shared Prioritized Experience Replay buffer. The framework demonstrates that a single neural network can learn effective control policies for multiple intersections simultaneously, exploiting the structural homogeneity of the signal control problem across intersections to produce policies that generalize across different intersection contexts within the same network. The shared PER buffer multiplies the effective data collection rate by the number of agents, accelerating convergence without requiring any additional computational resources beyond those of a single agent setup.

Contribution 3 Reward Function Analysis and Refinement : This work provides an empirical analysis of the effect of reward function design on learned policy behavior in the traffic signal control context, with particular focus on the delta waiting time term that was found to introduce a systematic bias toward excessive phase switching. The analysis demonstrates that a reward component that is theoretically motivated penalizing increases in waiting time can in practice distort the learned policy by rewarding the immediate artifact of switching rather than genuine improvements in traffic flow. The refined reward function that excludes this term and relies instead on static normalized components produces more stable and more effective learned policies, a finding that is practically relevant for researchers designing reward functions for similar applications.

Contribution 4 Throughput Measurement Methodology: This work identifies, analyzes, and resolves a systematic measurement artifact that can affect throughput evaluation in episodic traffic simulation studies, and proposes continuous accumulation as the correct measurement methodology. The documentation of this artifact and its resolution is a methodological contribution that can prevent misleading conclusions in future simulation based traffic control studies that use throughput as an evaluation metric.

5. Limitations Revisited

Several limitations bound the scope and generalizability of these findings.

First, the evaluated network consists of only four signalized intersections connected by single lane bidirectional roads. This scale is sufficient to study multi agent coordination and architectural comparisons, but it is small relative to real urban networks. The performance rankings and improvement magnitudes reported here may not hold quantitatively in larger networks with more complex topologies, higher agent counts, and longer congestion propagation paths.

Second, all experiments are conducted entirely within the SUMO simulation environment. The simulation to reality gap (arising from driver heterogeneity, sensor noise, non compliant behavior, and unpredictable incidents) means that the reported improvements are measured under controlled conditions that do not fully capture real world traffic complexity. Deployment in a real network would require additional validation and potentially domain adaptation.

Third, each model is trained from a single random seed, so the reported results correspond to a single training trajectory per architecture. The sensitivity of the comparative conclusions , particularly the performance gap between Classical and Double DQN to different training initializations has not been assessed. A multi seed evaluation would provide stronger statistical guarantees.

Fourth, evaluation is conducted on a single demand scenario. Although this scenario is deliberately out of distribution and conservative, it does not represent the full range of operating conditions a deployed controller would encounter. A complete performance profile would require evaluation across multiple demand levels and time of day patterns.

Finally, the reward function contains no explicit throughput term, meaning that the observed throughput improvements are indirect consequences of congestion reduction rather than directly optimized outcomes. The optimal reward design for multi objective traffic signal control remains an open question.

6. Future Work

Several directions emerge naturally from this work. The most immediate priority is scaling the framework to larger and more topologically complex networks, which would test whether the performance advantage of Dueling DQN is maintained as the number of interacting agents and the complexity of the coordination problem increase.

Complementing this, a multi seed training evaluation would provide stronger statistical guarantees for the comparative conclusions, particularly regarding the **performance gap** between Classical and Double DQN observed here.

Beyond scaling, two directions address the core methodology. Enriching the reward function with an explicit throughput component would allow agents to directly optimize network processing capacity rather than relying on it as an indirect benefit of congestion reduction. Evaluating more advanced

architectures such as D3QN within the same comparative framework would further clarify which algorithmic innovations provide genuine benefits in the traffic signal control setting.

The most consequential long term direction remains bridging the simulation to reality gap. The performance improvements demonstrated in SUMO must ultimately be validated against real instrumented intersections, and transfer learning or domain randomization techniques represent the most promising path toward practical deployment. Finally, moving from the decentralized implicit coordination adopted in this work toward explicit inter agent communication architectures would determine whether direct information sharing between neighboring controllers provides measurable benefits over coordination through shared environmental experience alone.

7. Closing Remarks

This thesis has demonstrated that value based deep reinforcement learning, when applied within a carefully designed and rigorously controlled experimental framework, constitutes a viable and effective approach to adaptive traffic signal control in multi-intersection urban environments. The practical recommendation emerging from this study is nuanced: Dueling DQN and Classical DQN deliver statistically equivalent performance and either may be selected for binary action traffic signal control tasks, with Classical DQN offering the advantage of architectural simplicity. Both are clearly preferable to Double DQN, whose overestimation correction introduces a switch bias that is actively detrimental in binary action spaces. Beyond this practical guidance, the results provide a theoretical insight into the conditions under which architectural innovations in value based reinforcement learning provide measurable performance advantages. Specifically, innovations that address the representational efficiency of the Q value function (as the Dueling decomposition does by separating state value from action advantage) provide benefits that are robust to action space dimensionality, while innovations that address estimation bias (as Double DQN) does may not only fail to provide benefits in binary action spaces, but may actively degrade performance by introducing a distortion that is harmful in the traffic signal control context.

The findings contribute to the growing body of evidence that intelligent transportation systems based on deep reinforcement learning can meaningfully improve upon decades old fixed timing approaches, even when evaluated under demanding out of distribution conditions that represent a conservative test of the learned policies' generalization capability. The 15% reduction in average waiting time, 16% reduction in average delay, 14% improvement in throughput, and 22% improvement in average speed achieved by Dueling DQN over the fixed time baseline (without any retraining on the

evaluation scenario and with no access to information beyond each intersection's local traffic state) represent a compelling demonstration of the practical potential of adaptive deep reinforcement learning for urban traffic management.

As cities continue to grow and the demand placed on urban transportation networks continues to intensify, the development of intelligent, adaptive, and scalable traffic signal control systems becomes increasingly urgent. This thesis has taken one step toward that goal by establishing a rigorous empirical foundation for algorithm selection in DRL based traffic signal control and by identifying the architectural properties that translate most effectively into real world traffic management performance. The directions for future work outlined in this chapter point toward a research agenda that could bring these systems progressively closer to deployment in the complex, noisy, and unpredictable environments of real urban networks.

Bibliographic References

- [1] M. Shahraz and M. Ahmed, ***"Intelligent Transportation Systems: An Overview of Current Trends and Limitations"***, International Journal of Scientific Research in Engineering and Management (IJSREM), Dec. 2022.
- [2] C. Creß, Z. Bing, and A. C. Knoll, ***"Intelligent Transportation Systems Using External Infrastructure: A Literature Survey"***, arXiv:2112.05615v4 [cs.RO], Aug. 2022.
- [3] Federal Highway Administration (FHWA), ***"Traffic Signal Timing Manual"***, U.S. Department of Transportation, Washington, DC, Jun. 2008.
- [4] G. Comert, M. Cetin, and N. Begashaw, ***"A Simple Traffic Signal Control Using Queue Length Information"***, arXiv:2006.06337, Jun. 2020.
- [5] Transportation Research Board, ***"Highway Capacity Manual (HCM 2010)"***, 5th ed. Washington, DC: National Academies Press, 2010.
- [6] R. S. Sutton and A. G. Barto, ***"Reinforcement Learning: An Introduction"***, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [7] C. Wu, I. Kim, and Z. Ma, ***"Deep Reinforcement Learning Based Traffic Signal Control: A Comparative Analysis"***, in Proc. 14th Int. Conf. Ambient Systems, Networks and Technologies (ANT), Mar. 2023.
- [8] V. Sylá and A. Lala, ***"Microsimulation-Based Evaluation of Fixed-Time and Vehicle-Actuated Traffic Signal Control Strategies Using PTV Vissim in an Urban Corridor"***, Informatica, vol. 50, no. 5, pp. 297–310, Jul. 2025.
- [9] L. A. Zadeh, ***"Fuzzy Sets"***, Information and Control, vol. 8, no. 3, pp. 338–353, 1965.
- [10] D. Čiprovski, A. Makarov, and M. Vučetić, ***"Efficiency of Vehicle Actuated Traffic Control on Isolated Intersection"***, International Journal of Electrical Engineering and Computing, vol. 7, no. 1, 2023.
- [11] T. Mahmood, M. E. Mohammed Ali, and A. Durdu, ***"A Two-Stage Fuzzy Logic Adaptive Traffic Signal Control for an Isolated Intersection Based on Real Data Using SUMO Simulator"***, International Journal of Trend in Scientific Research and Development (IJTSRD), vol. 3, no. 4, pp. 1174–1181, May–Jun. 2019.

- [12] N. Benkheris and R. Ghoul, **"Simulation and Optimization of an Isolated Intersection with Genetic Algorithms"**, International Journal of Engineering Research & Technology (IJERT), vol. 2, no. 11, Nov. 2013.
- [13] J. Cao and Y. Wang, **"Fuzzy Control of Intersection Signal Based on Optimized Genetic Algorithm"**, in Proc. Int. Conf. Civil, Transportation and Environment (ICCTE 2016), Guangzhou, China, 2016.
- [14] S. S. Mousavi, M. Schukat, and E. Howley, **"Traffic Light Control Using Deep Policy-Gradient and Value-Function Based Reinforcement Learning"**, arXiv:1704.08883 [cs.LG], May 2017.
- [15] T. Du, B. Wang, and L. Hu, **"Single Intersection Traffic Light Control by Multi-Agent Reinforcement Learning"**, Journal of Physics: Conference Series, vol. 2449, p. 012031, 2023. doi: 10.1088/1742-6596/2449/1/012031.
- [16] D. M. Levinson, **"Fundamentals of Transportation"**, Wikibooks / Engineering LibreTexts, 2021.
- [17] P. Varaiya, **"Max Pressure Control of a Network of Signalized Intersections"**, Transportation Research Part C: Emerging Technologies, vol. 36, pp. 177–195, Nov. 2013.
- [18] A. Bentadjine, **"Conception d'un modèle de contrôle adaptatif du trafic"**, M.S. thesis, Jun. 2022.
- [19] J. Louw, L. Labuschagne, and T. Woodley, **"A Comparison of Reinforcement Learning Agents Applied to Traffic Signal Optimisation"**, Smart Cities Proceedings, vol. 3, Sep. 2022.
- [20] R. Ducrocq, **"Deep Reinforcement Q-Learning for Intelligent Traffic Signal Control with Partial Detection"**, arXiv:2109.14337, Sep. 2021. [Online].
- [21] J. Gu, Y. Fang, C. Sheng, and P. Wen, **"Double Deep Q-Network with a Dual-Agent for Traffic Signal Control"**, Applied Sciences, vol. 10, no. 5, p. 1622, 2020.
- [22] Z. Wang, L. Xu, and J. Ma, **"Carbon Dioxide Emission Reduction-Oriented Optimal Control of Traffic Signals in Mixed Traffic Flow Based on Deep Reinforcement Learning"**, 2023.
- [23] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, **"CoLight: Learning Network-Level Cooperation for Traffic Signal Control"**, in Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM), Beijing, China, 2019, pp. 1913–1922.

- [24] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "*Prioritized Experience Replay*", in Proc. 4th Int. Conf. Learning Representations (ICLR 2016), San Juan, Puerto Rico, 2016.
- [25] B. Illowsky and S. Dean, "*Statistics*", OpenStax, Houston, TX, 2020.
- [26] V. L. Knoop, "*Traffic Flow Theory: An Introduction with Exercises*", TU Delft Open Books, Delft, The Netherlands, 2018.

